Rockchip MIPI DSI2 Software Developer Guide

ID: RK-KF-YF-705

Release Version: V2.4.0

Release Date: 2026-05-26

Security Level: ☐Top-Secret ☐Secret ☐Internal ■Public

DISCLAIMER

THIS DOCUMENT IS PROVIDED “AS IS”. ROCKCHIP ELECTRONICS CO., LTD.(“ROCKCHIP”) DOES NOT PROVIDE ANY WARRANTY OF ANY KIND, EXPRESSED, IMPLIED OR OTHERWISE, WITH RESPECT TO THE ACCURACY, RELIABILITY, COMPLETENESS, MERCHANTABILITY, FITNESS FOR ANY PARTICULAR PURPOSE OR NON-INFRINGEMENT OF ANY REPRESENTATION, INFORMATION AND CONTENT IN THIS DOCUMENT. THIS DOCUMENT IS FOR REFERENCE ONLY. THIS DOCUMENT MAY BE UPDATED OR CHANGED WITHOUT ANY NOTICE AT ANY TIME DUE TO THE UPGRADES OF THE PRODUCT OR ANY OTHER REASONS.

Trademark Statement

"Rockchip", "瑞芯微", "瑞芯" shall be Rockchip’s registered trademarks and owned by Rockchip. All the other trademarks or registered trademarks mentioned in this document shall be owned by their respective owners.

All rights reserved. ©2026. Rockchip Electronics Co., Ltd.

Beyond the scope of fair use, neither any entity nor individual shall extract, copy, or distribute this document in any form in whole or in part without the written approval of Rockchip.

Rockchip Electronics Co., Ltd.

No.18 Building, No.89, Software Boulevard, TongPan Rd, Fuzhou, Fujian, PRC

Website: www.rock-chips.com

Customer service Tel: +86-4007-700-590

Customer service Fax: +86-591-83951833

Customer service e-Mail: fae@rock-chips.com

Preface

This document primarily describes the software configuration and debugging methods for various application scenarios of Rockchip MIPI DSI2.

Product Version

Chip Models	Kernel Version
RK3576	LINUX Kernel 6.1/6.12
RK3588	LINUX Kernel 5.10/6.1/6.12

Intended Audience

This document (this guide) is primarily intended for the following engineers:

Technical Support Engineers

Software Development Engineers

Revision History

Version	Author	Date	Modification Description
V1.0.0	Huang Guochun	2022-01-07	Initial release
V2.0.0	Huang Guochun	2022-07-06	Added DSC and other content
V2.1.0	Huang Guochun	2023-08-08	Added timing details for three video modes
V2.2.0	Huang Guochun	2024-06-12	Added dynamic frame rate change/PSR, etc.
V2.3.0	Huang Guochun	2025-01-24	Added descriptions of signal SI driver strength, etc.
V2.4.0	Huang Guochun	2026-05-26	Added description of VBLANK limitation for rk3588 platform

Table of Contents

Rockchip MIPI DSI2 Software Developer Guide

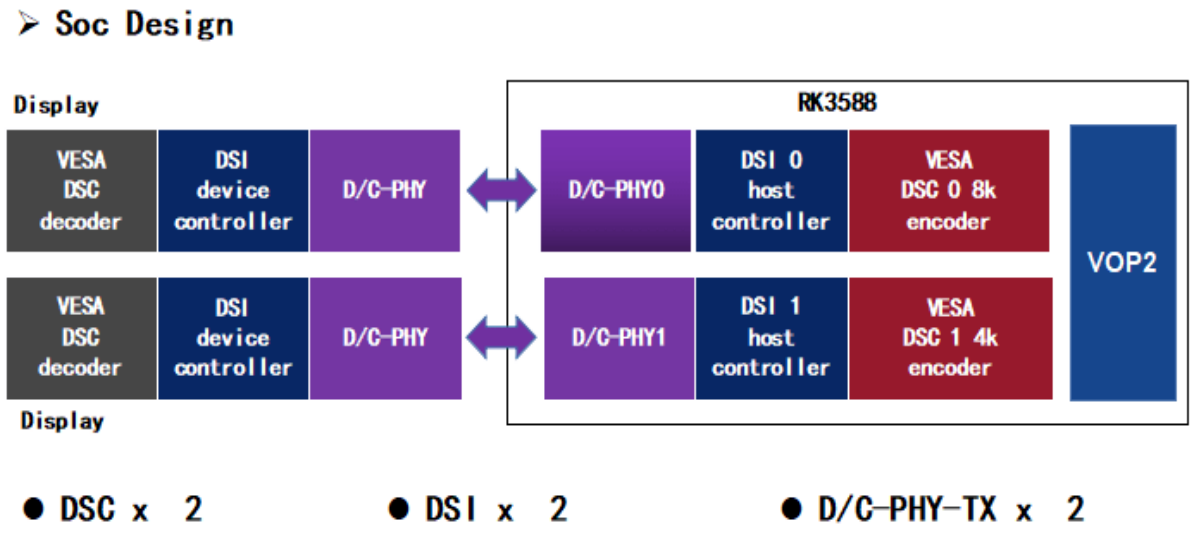
1. Introduction
2. MIPI-DSI2 Features
3. Differences Between RK3576 and RK3588 DSI Interfaces
4. Differences Between MIPI DSI-2 Host and MIPI DSI Host
5. MIPI D-PHY Differences
6. Application Areas
7. Driver Code Description
   - 7.1 U-Boot
     - 7.1.1 Driver Location
     - 7.1.2 Driver Configuration
   - 7.2 Kernel
     - 7.2.1 Driver Location
     - 7.2.2 Driver Configuration
     - 7.2.3 Reference Device Tree
8. DSI Controller and Panel Configuration
   - 8.1 DTS Configuration
   - 8.2 Configuration Instructions
     - 8.2.1 General Configuration
     - 8.2.2 Display Timing
     - 8.2.3 dsi,flags
       - 8.2.3.1 CLK Type
       - 8.2.3.2 Eotp
       - 8.2.3.3 BLANK_HS_EN
     - 8.2.4 Panel Power-On Sequence
     - 8.2.5 Panel Power-Off Sequence
     - 8.2.6 Common Data Types in Initialization Sequences
9. Timing for DSI Video Mode
10. Bandwidth
11. DSC
    - 11.1 Slice
    - 11.2 DSC Encode
    - 11.3 DSC Bandwidth
    - 11.4 PPS
    - 11.5 Example
      - 11.5.1 When to Enable DSC
      - 11.5.2 How to Enable DSC for Dual-Channel MIPI
12. Display Route
    - 12.1 MIPI with DSC
    - 12.2 MIPI with DSC Bypass
    - 12.3 DTS Configuration
13. Boot Logo
    - 13.1 route_dsi0
    - 13.2 route_dsi1
    - 13.3 route_dsi0 && route_dsi1
14. DSI HOST
    - 14.1 Single DSI
      - 14.1.1 DSI0
      - 14.1.2 DSI1
    - 14.2 Dual-Channel DSI

14.3 Dual-link DSI
14.4 DSI Application Scenarios
14.4.1 DSI + SerDes Solution
14.4.2 Multi-Display Connection Solution
15. DC-PHY
15.1 D-PHY
15.2 C-PHY
16. Dynamic Frame Rate Adjustment
17. PSR
18. Protocol Analysis
18.1 DSI Layer Definitions
18.2 D Option
18.2.1 Lane states and line levels
18.2.2 Global Operation Flow Diagram
18.2.2.1 Escape Modes
18.2.2.1.1 Escape Commands
18.2.2.1.2 LPDT
18.2.2.1.3 ULPS
18.2.2.2 HSDT
18.2.2.3 BTA
18.2.3 Endian Policy
18.2.3.1 SPa
18.2.3.1.1 LPDT-SPa
18.2.3.1.2 HSDT-SPa
18.2.3.2 LPa
18.2.3.2.1 LPDT-LPa
18.2.3.2.2 HSDT-LPa
18.2.3.3 DI
18.2.4 Video Mode Interface Timing
18.2.4.1 Vertical Display Timing (Video mode)
18.2.4.2 Horizontal Display Timing (Video Mode)
18.2.4.3 Non-Burst Mode with Sync Pulses
18.2.4.4 Burst Mode
19. Platform limitations
19.1 RK3588
19.1.1 VBLANK
20. FAQ
20.1 Check VOP Timing and Connector Information
20.2 View DSI2-related Clock Tree
20.3 How to Check the Specified DSI Lane Rate
20.4 The MIPI DSI2 HOST does not have a built-in color bar function; use the following commands to enable VOP2 to output color bars
20.4.1 RK3588
20.4.2 RK3576
20.5 How to determine whether MIPI DSI2 and the panel are communicating normally when a display anomaly occurs
20.5.1 Two Readback Waveforms for get_power_mode
20.5.1.1 Normal Panel Return of 0x08 (0x11 and 0x29 Not Issued)
20.5.1.2 Normal Panel Return of 0x9c (0x11 and 0x29 Issued)
20.6 DRM Driver Failed to Bind
20.7 Backlight driver probe failure
20.8 How to Configure TE for Command Mode Display Modules
20.8.1 Hardware TE

20.8.2 Software TE
20.9 Dual-Channel MIPI Primary/Secondary Sequence Switching
20.10 Debug Nodes
20.10.1 Patch
20.10.2 Drive Strength
20.10.2.1 Output Voltage (400mV/200mV/530mV) Selection Pin
20.10.2.2 LP Common-Mode Voltage
20.10.2.3 HS Common-Mode Voltage
20.10.2.4 HS Differential Voltage
20.10.2.4.1 High-Speed Driver Up Resistor Control
20.10.2.4.2 High-Speed Driver Down Resistor Control
20.10.2.5 Cap Peaking
20.10.3 Signal Timing
20.10.3.1 Tlpx
20.10.3.2 Ths_prepare_or_prepare_3
20.10.3.3 Ths_zero_or_prebegin_3
20.10.3.4 Ths_trail_or_post_3
20.10.3.5 Ths_exit

1. Introduction

DSI-2 is part of a set of communication protocols defined by the MIPI Alliance. The DWC-MIPI-DSI2 is a digital core controller that implements all protocol functions defined in the MIPI-DSI2 specification. It is compatible with both D-PHY and C-PHY physical interfaces and supports two channels of Display Stream Compression (DSC) data transmission.



2. MIPI-DSI2 Features

1. MIPI® Alliance Specification for Display Serial Interface 2 (DSI-2) Version 1.1
2. MIPI® Alliance Specification for Display Command Set (DCS) Version 1.4
3. MIPI® Alliance Specification for D-PHY v2.0
4. MIPI® Alliance Specification for C-PHY v1.1
5. Four data lanes on D-PHY and three data trios on C-PHY
6. Bidirectional communication and escape mode through data lane 0
7. Continuous and non-continuous clock modes on D-PHY and non-continuous clock mode on C-PHY
8. End of Transmission packet (EoTp)
9. Scrambling
10. VESA DSC 1.1/1.2a
11. Up to 4.5 Gbps per lane in D-PHY
12. Up to 2.0 Gsps per trio in C-PHY

3. Differences Between RK3576 and RK3588 DSI Interfaces

Feature	RK3576	RK3588
Dual channel	Not supported	Supported
Max resolution	2560x1600@60Hz	4096x2304@60Hz
Data lanes	1/2/4 lanes	1/2/4/8 lanes
Max lane rate	D-PHY: 2.5Gbps/lane C-PHY: 1.7Gsps/trio	D-PHY: 4.5Gbps/lane C-PHY: 2.0Gsps/trio
Color Format	RGB	RGB
Max Color Depth	10 bit	10 bit
DSC	Not supported	Supports VESA DSC 1.1/1.2a
C-PHY	Supported	Supported

4. Differences Between MIPI DSI-2 Host and MIPI DSI Host

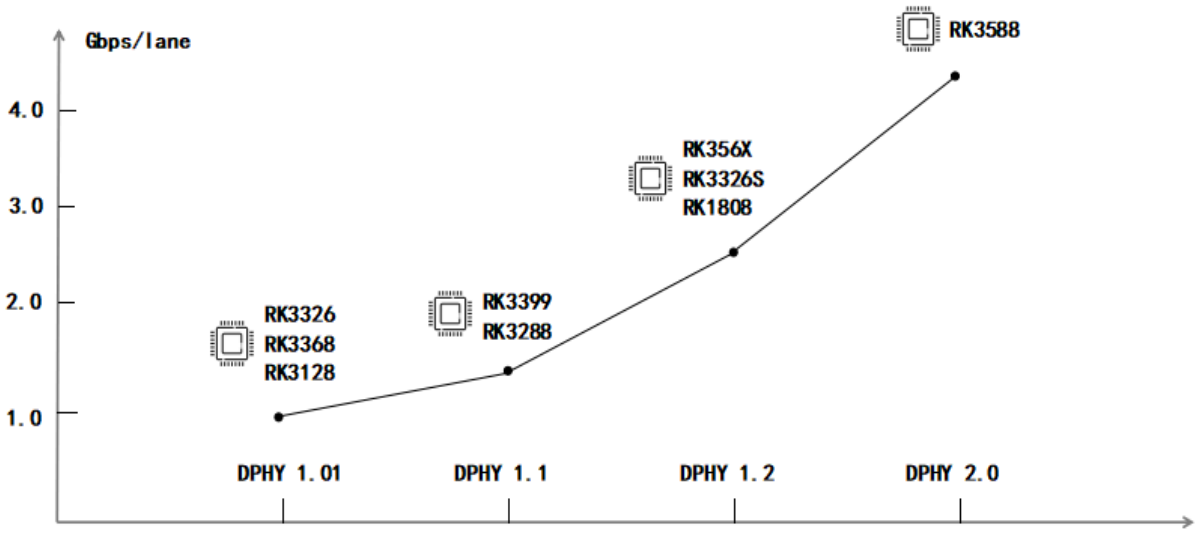
In addition to being fully compatible with all MIPI DSI protocol features, MIPI DSI-2 adds support for MIPI C-PHY.

	D-PHY 1.01	D-PHY 1.1	D-PHY 1.2	D-PHY 2.0	C-PHY
DSI 1.0	yes	yes	yes	yes	no
DSI 1.1	yes	yes	yes	yes	no
DSI 1.2	yes	yes	yes	yes	no
DSI 1.3	yes	yes	yes	yes	no
DSI-2 1.0	yes	yes	yes	yes	yes

MIPI DSI 兼容性

5. MIPI D-PHY Differences

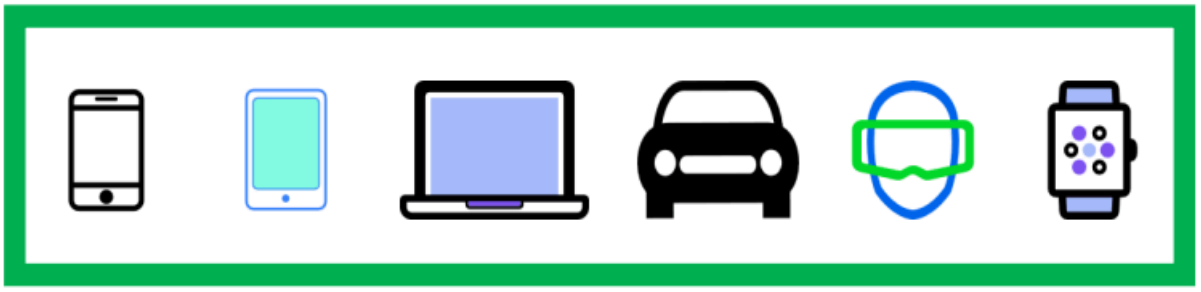
The MIPI D-PHY on the RK3588 platform differs from previous platforms, supporting a maximum bandwidth of 4.5 Gbps.



6. Application Areas

MIPI DSI utilizes differential signaling to reduce pin count and hardware design complexity while maintaining excellent hardware compatibility. Additionally, IP based on the MIPI DSI protocol features low power consumption and low EMI.

Its application areas are shown below:



- 超高清设备
- 嵌入式显示设备
- 智能座舱
- VR
- 穿戴设备

7. Driver Code Description

7.1 U-Boot

7.1.1 Driver Location

```
drivers/video/drm/dw_mipi_dsi2.c
drivers/video/drm/samsung_mipi_dcphy.c
```

7.1.2 Driver Configuration

```
CONFIG_DRM_ROCKCHIP_DW_MIPI_DSI2=y
CONFIG_DRM_ROCKCHIP_SAMSUNG_MIPI_DCPHY=y
```

7.2 Kernel

7.2.1 Driver Location

```
MIPI DSI-2 host controller:
drivers/gpu/drm/rockchip/dw-mipi-dsi2-rockchip.c

MIPI DCPHY:
drivers/phy/rockchip/phy-rockchip-samsung-dcphy.c
```

7.2.2 Driver Configuration

```
CONFIG_ROCKCHIP_DW_MIPI_DSI=y
CONFIG_PHY_ROCKCHIP_SAMSUNG_DCPHY=y
```

7.2.3 Reference Device Tree

DTS path:

```
arch/arm64/boot/dts/rockchip/rk3588-evb.dtsi
arch/arm64/boot/dts/rockchip/rk3588-evb1-lp4.dtsi
arch/arm64/boot/dts/rockchip/rk3588-evb2-lp4.dtsi
arch/arm64/boot/dts/rockchip/rk3588-evb3-lp5.dtsi
arch/arm64/boot/dts/rockchip/rk3588-evb4-lp4.dtsi
arch/arm64/boot/dts/rockchip/rk3588s-evb.dtsi
arch/arm64/boot/dts/rockchip/rk3588s-evb1-lp4x.dtsi
arch/arm64/boot/dts/rockchip/rk3588s-evb2-lp5.dtsi
arch/arm64/boot/dts/rockchip/rk3588s-evb4-lp4x.dtsi
```

DTS Configuration Use Case Scenarios:

```
rk3588-evb1: dsi0->dphy->1080p_panel && dsi1->dphy->1080p_panel;
rk3588-evb2: dsi1->dphy->1080p_panel;
rk3588-evb3: dsi0->dphy->1080p_panel && dsi1->cphy->cphy_panel;
rk3588-evb4: dsi0->dphy->1080p_panel;
rk3588s-evb1: dsi0->dphy->1080p_panel && dsi1->dphy->cmd_no_dsc_panel;
rk3588s-evb2: dsi0->cphy->cphy_panel && dsi1->dphy->1080p_panel;
rk3588s-evb4: dsi0->dphy->1080p_panel && dsi1->dphy->cmd_dsc_panel;
```

8. DSI Controller and Panel Configuration

8.1 DTS Configuration

```
&dsi0 {
    status = "okay";
    //rockchip,lane-rate = <1000>;
    //auto-calculation-mode;
    //disable-hold-mode;
    //support-psr;
};

&dsi0_panel {
    status = "okay";
    compatible = "simple-panel-dsi";
    reg = <0>;
    power-supply = <&vcc3v3_lcd_n>;
    backlight = <&backlight>;
    reset-gpios = <&gpio2 RK_PB4 GPIO_ACTIVE_LOW>;
    reset-delay-ms = <10>;
    enable-delay-ms = <10>;
    prepare-delay-ms = <10>;
    unprepare-delay-ms = <10>;
    disable-delay-ms = <60>;
    dsi,flags = <(MIPI_DSI_MODE_VIDEO | MIPI_DSI_MODE_VIDEO_BURST |
        MIPI_DSI_MODE_LPM | MIPI_DSI_MODE_EOT_PACKET)>;
    dsi,format = <MIPI_DSI_FMT_RGB888>;
    dsi,lanes  = <4>;
    //phy-c-option;
    //compressed-data;
    //slice-width = <720>;
    //slice-height = <65>;
    //version-major = <1>;
    //version-minor = <1>;

    panel-init-sequence = [
        ...
        05 78 01 11
        05 00 01 29
    ];
```

```
    panel-exit-sequence = [
      05 00 01 28
      05 00 01 10
    ];

    disp_timings0: display-timings {
      native-mode = <&dsi0_timing0>;
      dsi0_timing0: timing0 {
        clock-frequency = <132000000>;
        hactive = <1080>;
        vactive = <1920>;
        hfront-porch = <15>;
        hsync-len = <4>;
        hback-porch = <30>;
        vfront-porch = <15>;
        vsync-len = <2>;
        vback-porch = <15>;
        hsync-active = <0>;
        vsync-active = <0>;
        de-active = <0>;
        pixelclk-active = <0>;
      };
    };
};
```

8.2 Configuration Instructions

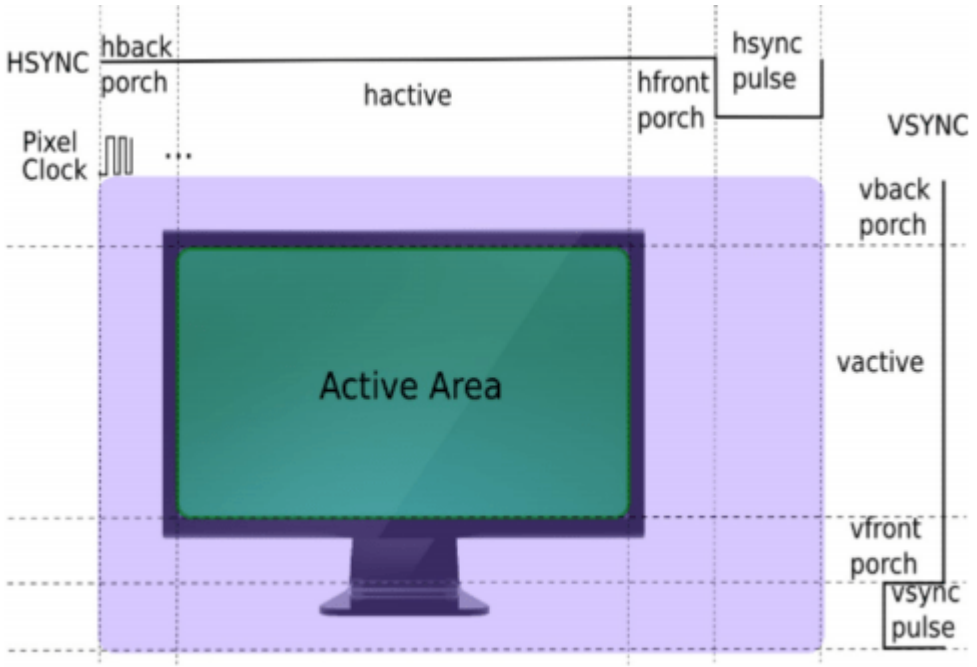
8.2.1 General Configuration

Property	Description	Value
rockchip,lane-rate	Manually specify the MIPI lane rate	Supported units: D-PHY: Mbps/lane or Kbps/lane C-PHY: Msps/lane or Ksps/lane Example for 800Mbps: rockchip,lane-rate = <800>; or rockchip,lane-rate = <800000>;
auto-calculation-mode	Enable Auto-Calculation mode	Boolean string
disable-hold-mode	Enable frame refresh without configuring TE	Boolean string
support-psr	Enable PSR function	Boolean string
compatible	compatible	simple-panel-dsi
power-supply	Panel power supply [option]	Relevant regulator reference

backlight	Backlight	Backlight reference
enable-gpios	Panel enable GPIO [option]	GPIO reference description
reset-gpios	Panel reset GPIO	GPIO reference description
reset-delay-ms	panel sequence delay	Refer to panel spec
enable-delay-ms		
prepare-delay-ms		
unprepare-delay-ms		
disable-delay-ms		
dsi,flags	DSI2 operation mode	cmd mode: MIPI_DSI_MODE_LPM | MIPI_DSI_MODE_EOT_PACKET
		video mode: MIPI_DSI_MODE_VIDEO | MIPI_DSI_MODE_VIDEO_BURST | MIPI_DSI_MODE_LPM | MIPI_DSI_MODE_EOT_PACKET
dsi,format	Pixel Data Format	MIPI_DSI_FMT_RGB888
		MIPI_DSI_FMT_RGB666
		MIPI_DSI_FMT_RGB666_PACKED
		MIPI_DSI_FMT_RGB565
dsi,lanes	Number of MIPI data lanes	1/2/3 trios [cphy]
		6 trios [cphy dual-channel]
		1/2/3/4 lanes [dphy]
		8 lanes [dphy dual-channel]
phy-c-option	C-PHY panel [option]	Boolean string
compressed-data	Panel with DSC [option]	Boolean string
slice-width	Defines DSC slice width [option]	Refer to panel spec
slice-height	Defines the DSC slice height [option]	

version-major version-minor	Defines the DSC version [option]	Refer to panel spec
panel-init-sequence	Panel power-on initialization sequence	[hex] data_type delay_ms payload_length payload
panel-exit-sequence	Panel power-off initialization sequence	
display-timings	Panel timing	Refer to panel spec

8.2.2 Display Timing



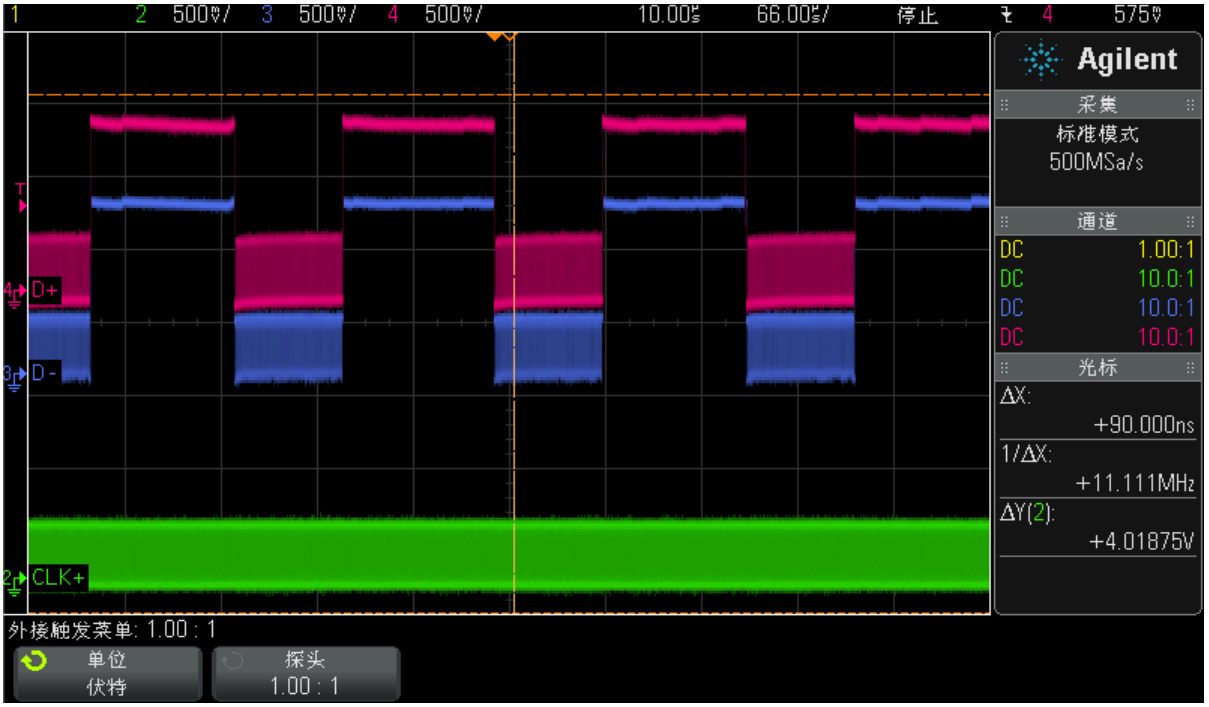
Display Timing

8.2.3 dsi,flags

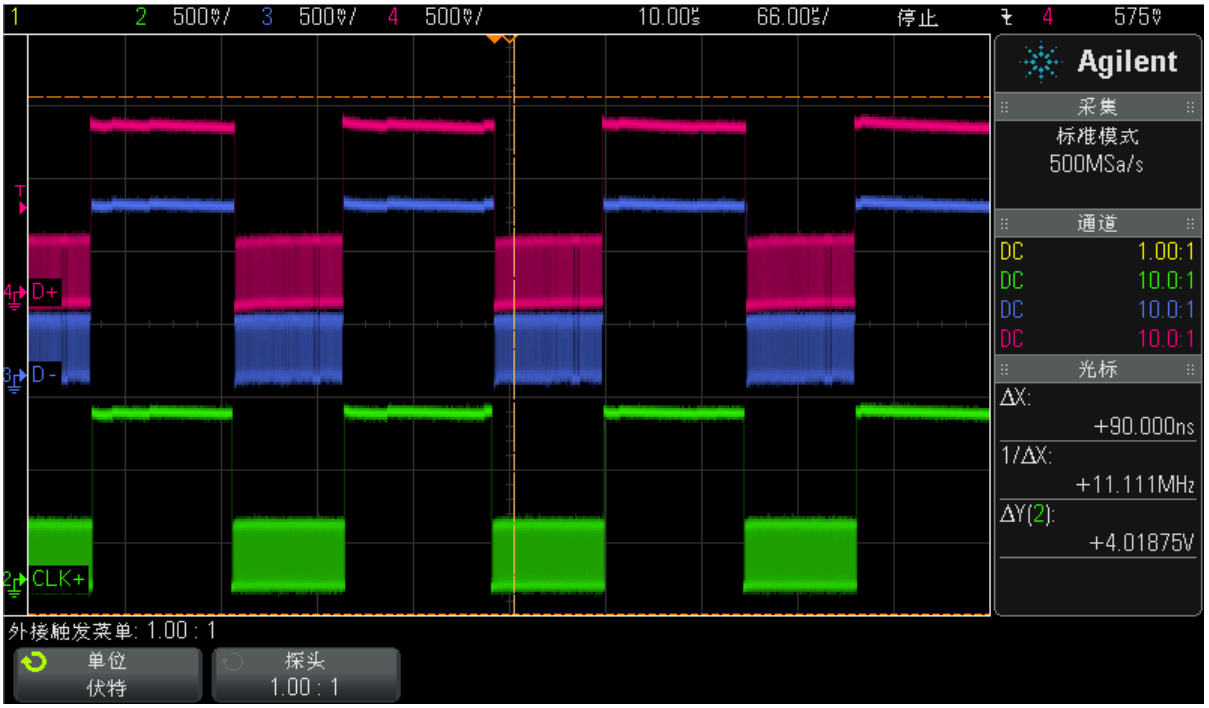
8.2.3.1 CLK Type

As described in Section 5.6.1 (Clock Requirements) of the MIPI DSI specification, all DSI transmitters and receivers shall support continuous clock behavior on the clock lane and may optionally support non-continuous clock behavior. Therefore, whether the clock lane operates in continuous or non-continuous mode depends on the requirements of the display peripheral.

By default, the MIPI D-PHY clock lane operates in continuous mode, which is supported by both the host and slave in a DSI display system. Additionally, if the DSI peripheral relies on the host-side clock for operation, the clock lane must operate in continuous mode, as shown below:



When power saving is a system consideration, especially during transmission intervals, the clock lane can be configured for non-continuous mode to avoid unnecessary power consumption when no data transmission is required. By appending `MIPI_DSI_CLOCK_NON_CONTINUOUS` to `dsi,flags`, the MIPI D-PHY clock lane will be configured for non-continuous mode, as shown below:



8.2.3.2 Eotp

EoTp (End of Transmission Packet) is not supported in DSI specification version V1.0 and earlier. Only devices compliant with DSI specifications later than V1.0 are required to support EoTp generation and detection. The DSI version on RK3588/RK3576 is V2.0. To ensure interoperability with legacy devices, the host should be capable of enabling and disabling EoTp generation and detection. In summary, whether EoTp is enabled depends on whether the panel supports it.

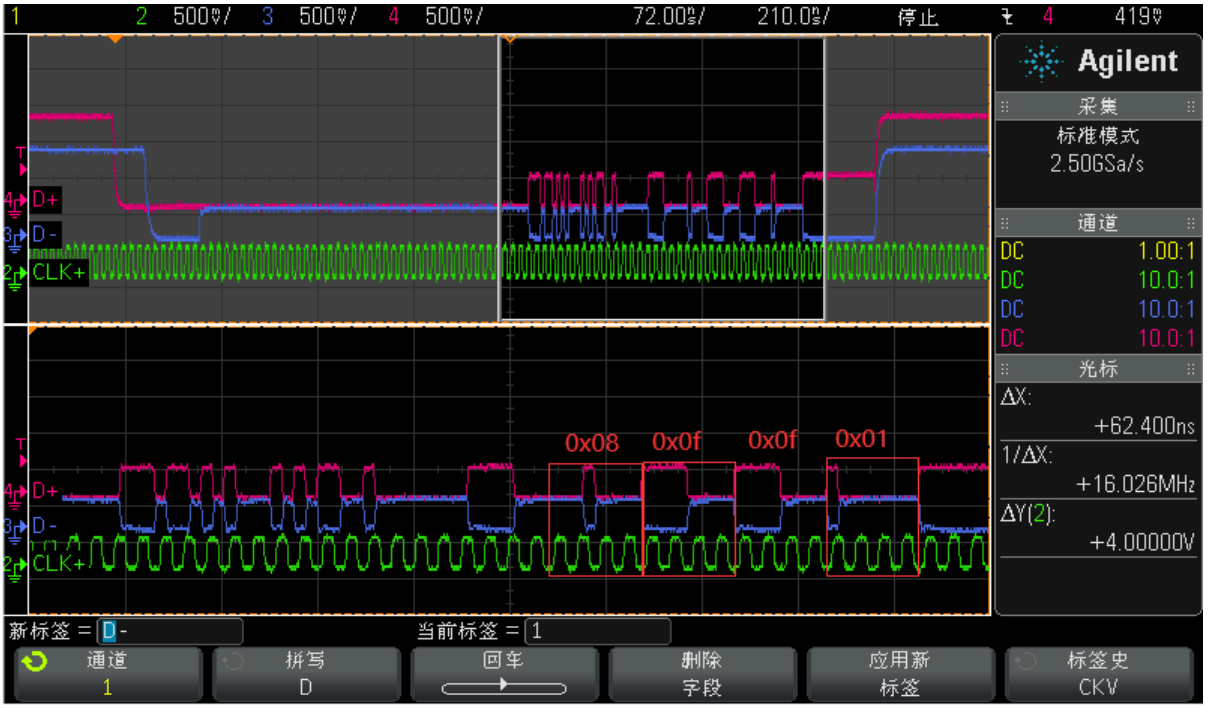
EoTp is a short packet used to indicate the end of high-speed transmission on the data lane. Its primary function is to enhance the robustness of high-speed communication; therefore, DSI does not require EoTp transmission in LP mode.

Unlike other DSI packets, EoTp has a fixed format:

Data Type = DI [5:0] = 0b001000
Virtual Channel = DI [7:6] = 0b00
Payload Data [15:0] = 0x0F0F
ECC [7:0] = 0x01

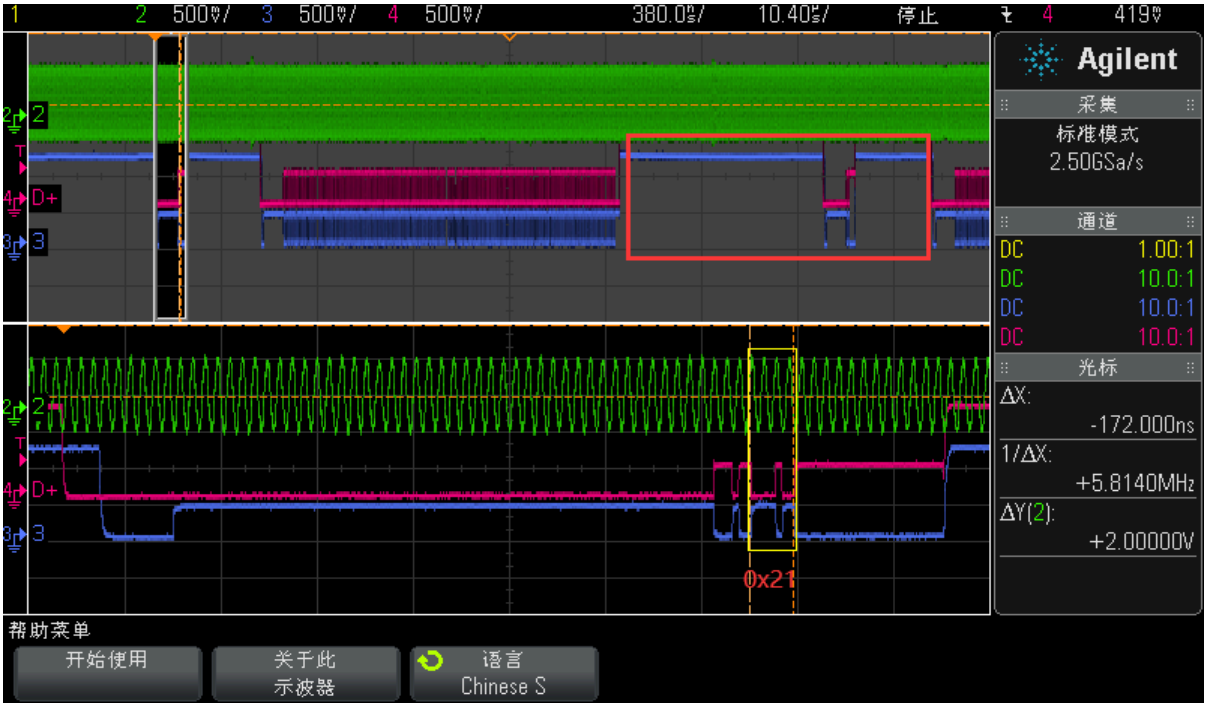
Appending `MIPI_DSI_MODE_EOT_PACKET` to the `dsi,flags` property enables or disables the SoC MIPI DSI TX from sending EoTp in High-Speed mode.

The following figure shows the EoTp waveform captured in HSDT mode:

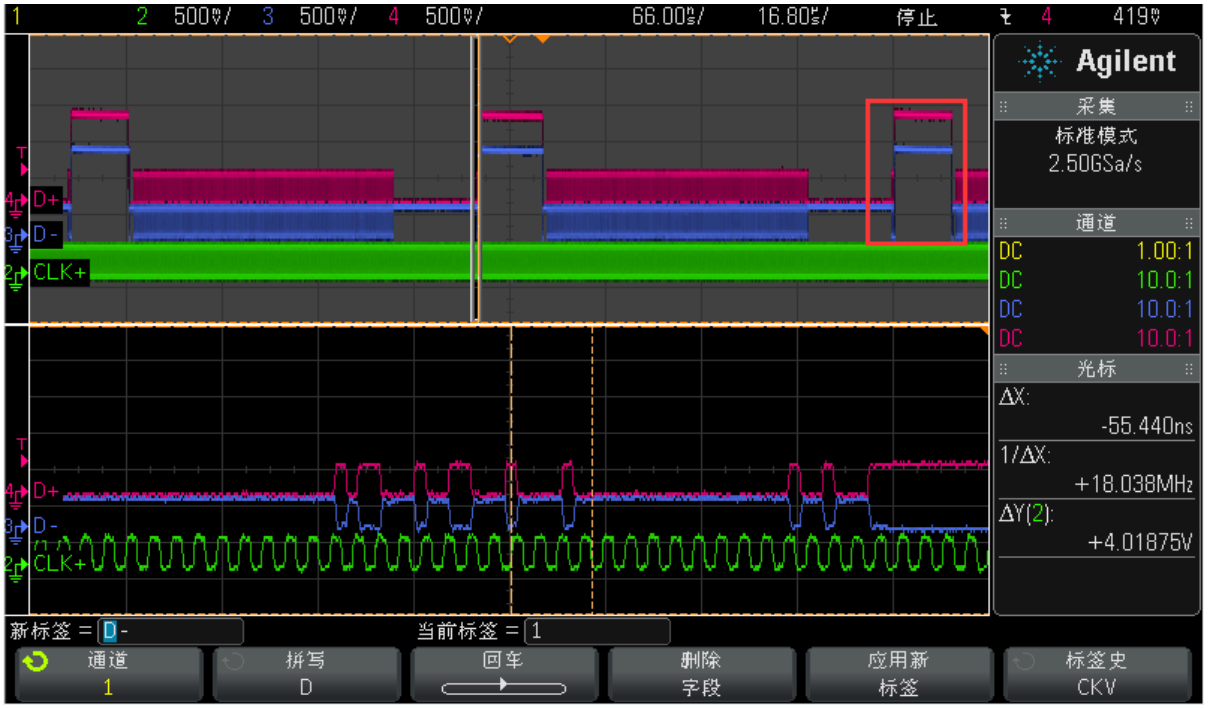


8.2.3.3 BLANK_HS_EN

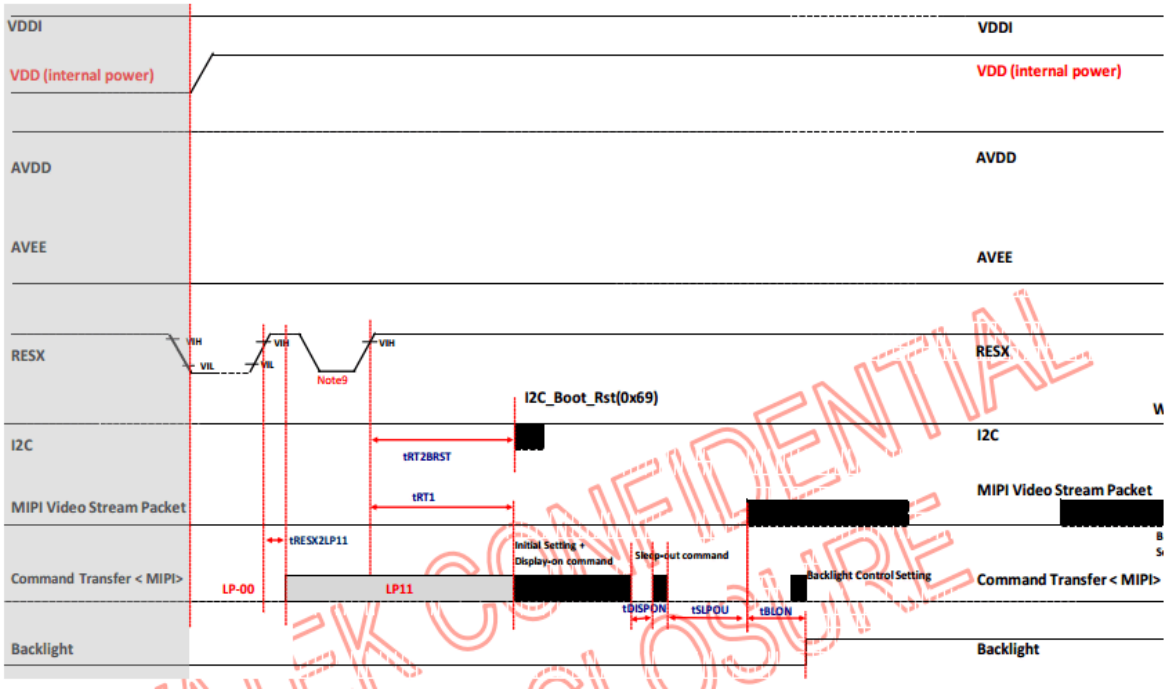
In the data lane, there are typically two LP11 blanking intervals per line, as shown below:



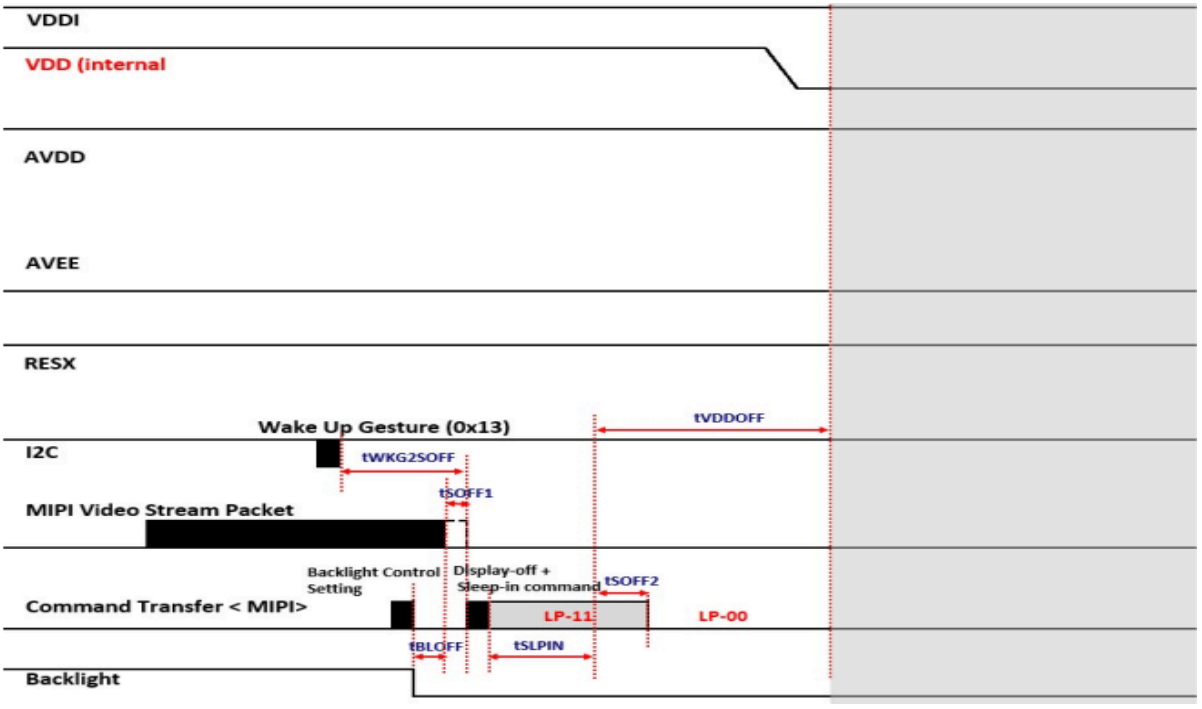
However, some display modules or external MIPI bridge chips do not support two LP-11 intervals during the Hblank phase. In such cases, append `BLK_HFP_HS_EN` or `BLK_HBP_HS_EN` to the `dsi,flags` property to configure HFP or HBP to operate in High-Speed mode.



8.2.4 Panel Power-On Sequence



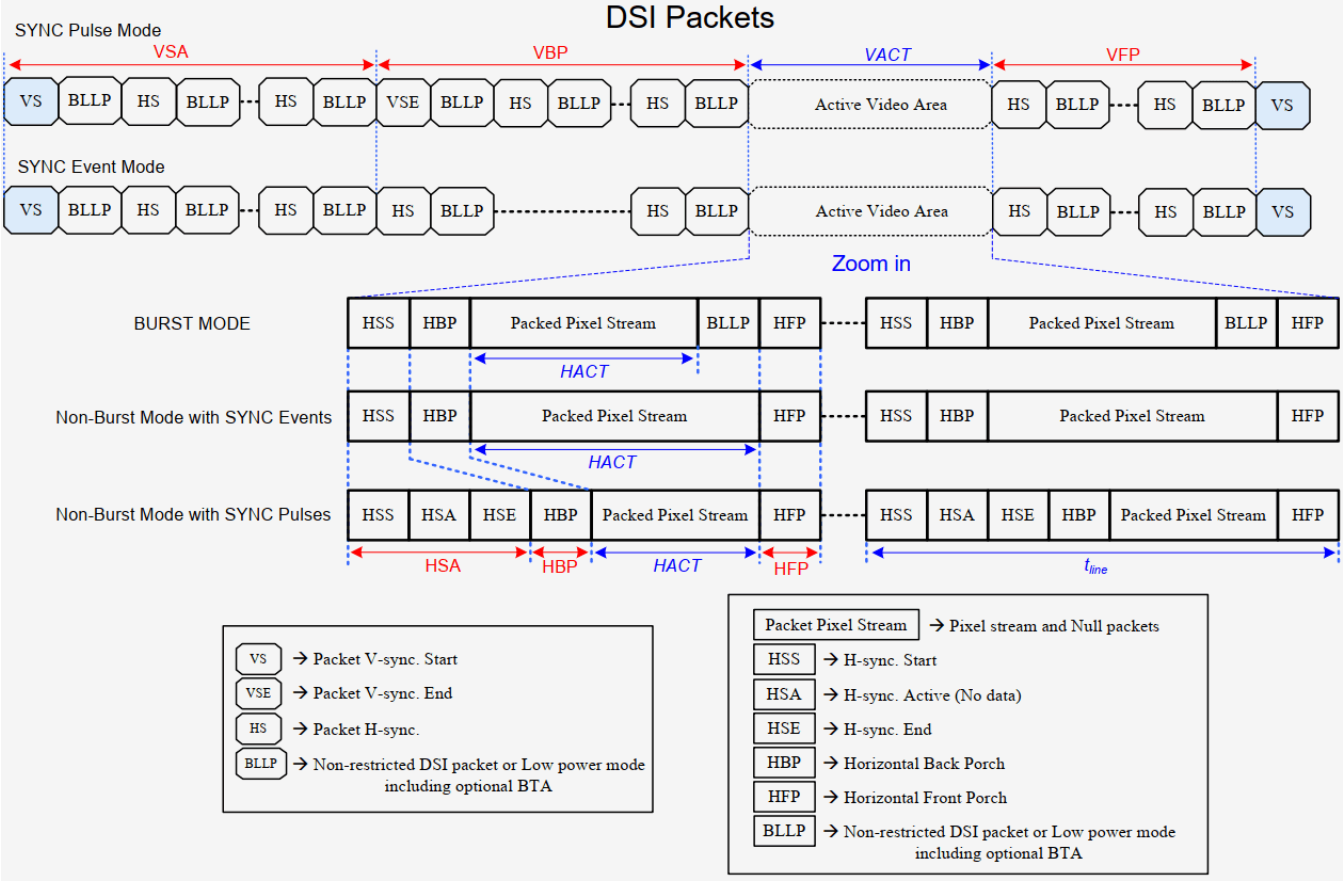
8.2.5 Panel Power-Off Sequence



8.2.6 Common Data Types in Initialization Sequences

Data Type	Description	Packet Size
0x03	Generic Short WRITE, no parameters	short
0x13	Generic Short WRITE, 1 parameter	short
0x23	Generic Short WRITE, 2 parameters	short
0x29	Generic Long WRITE	long
0x05	DCS Short WRITE, no parameters	short
0x15	DCS Short WRITE, 1 parameter	short
0x07	DCS Short WRITE, 1 parameter, DSC EN	short
0x0a	DCS Long WRITE, PPS, 128 bytes	long

9. Timing for DSI Video Mode



10. Bandwidth

The MIPI DSI Driver automatically calculates bandwidth according to the following formula based on different operating modes. During debugging, if fine-tuning of the calculated result is required, it can be specified via the `rockchip,lane-rate` property under the DTS dsi node. The unit can be Kbps/Mbps (D-PHY) or Ksps/Msps (C-PHY). As observed in the following bandwidth calculation examples, the RK3588 MIPI DSI D-PHY can fully support 4K@60 resolution without DSC, whereas the C-PHY slightly exceeds its maximum bandwidth.

resolution	hact	hfp	hsync	hbp	vact	vfp	vsync	vbp	bpp	lanes
3840x2160	3840	176	88	296	2160	8	10	72	24	4

```
Htotal = hact + hfp + hsync + hbp
Vtotal =  vact + vfp + vsync + vbp
Pixel-CLK = Htotal x Vtotal x 帧率

D-Option:
video burst:
BW = Htotal x Vtotal x 帧率 x BPP x 10 / lanes / 9 (Gbps)

no video burst sync pulse/event:
BW = Htotal x Vtotal x 帧率 x BPP  / lanes (Gbps)

C-Option:
video burst:
BW = Htotal x Vtotal x 帧率 x BPP x 10 / lanes / 9 / 2.28
(Gsps)

no video burst sync pulse/event:
BW = Htotal x Vtotal x 帧率 x BPP  / lanes / 2.28 (Gsps)
```

```
Htotal = 4400
Vtotal = 2250
Pixel-CLK = 4400 x 2250 x 60

D-Option:
video burst:
BW=4400x2250x60x24x10/4/9=3.96 Gbps

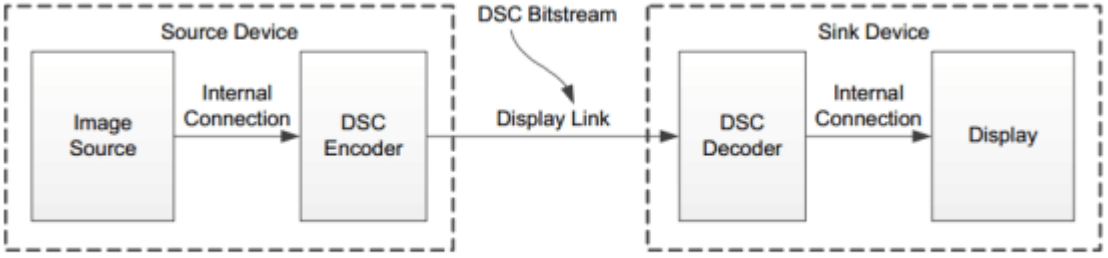
no video burst sync pulse/event:
BW=4400x2250x60x24/4=3.57 Gbps

C-Option:
video burst:
BW = 4450x2250x60x24x10/3/9/2.28=2.32 Gbps

no video burst sync pulse/event:
BW = 4450x2250x60x24/3/2.28=2.11 Gbps
```

11. DSC

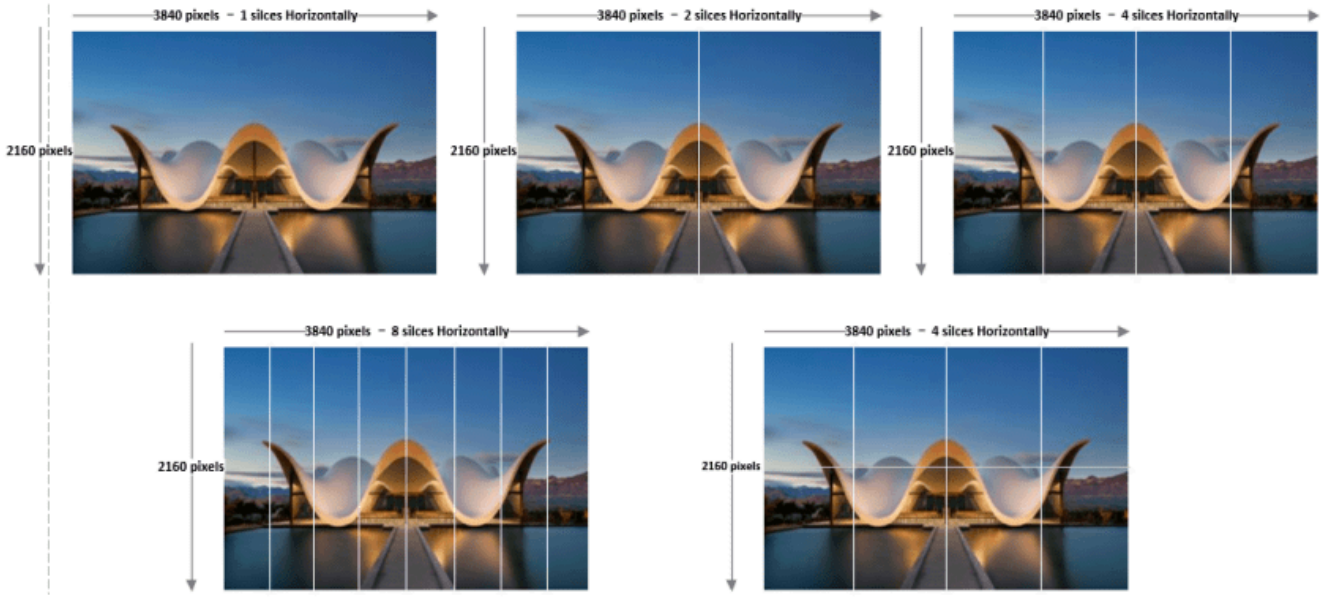
DSC stands for Display Stream Compression. The following diagram illustrates a complete DSC system architecture. The entire system operates in real-time: uncompressed raw video pixel data enters the encoder in raster scan order and outputs a bitstream, which is transmitted in real-time via the display link to the decoder. The decoder reconstructs the received bitstream into raw video pixel data and forwards it to the display module. The image format output by the decoder is identical to the image data format input to the encoder. The RK3588 features two DSC encoders, enabling high-resolution content to be transmitted with low bandwidth and low latency, thereby allowing MIPI DSI to support higher resolution displays.



DSC Use in End-to end System

11.1 Slice

To accelerate the encoding process and minimize compression artifacts, DSC introduces the concept of a slice, dividing each frame into multiple sections that are encoded simultaneously. DSC supports 1, 2, 4, 8, or even more slices. Note that the unit is slices/line, where a line refers to a row of pixels arranged in raster-scan order during image formation. In addition to varying the number of slices, DSC also supports slices with different aspect ratios. As shown in the top-right and bottom-right images below, both configurations use four slices per line; however, the top-right image uses elongated rectangular slices, while the bottom-right image uses narrower rectangular slices. The choice of slice dimensions depends on the number of slices supported by the Source Device and Sink Device, as well as the DSC compression or decompression ratio.
RK3588 DSC0 supports up to 8 slices, and DSC1 supports up to 2 slices.



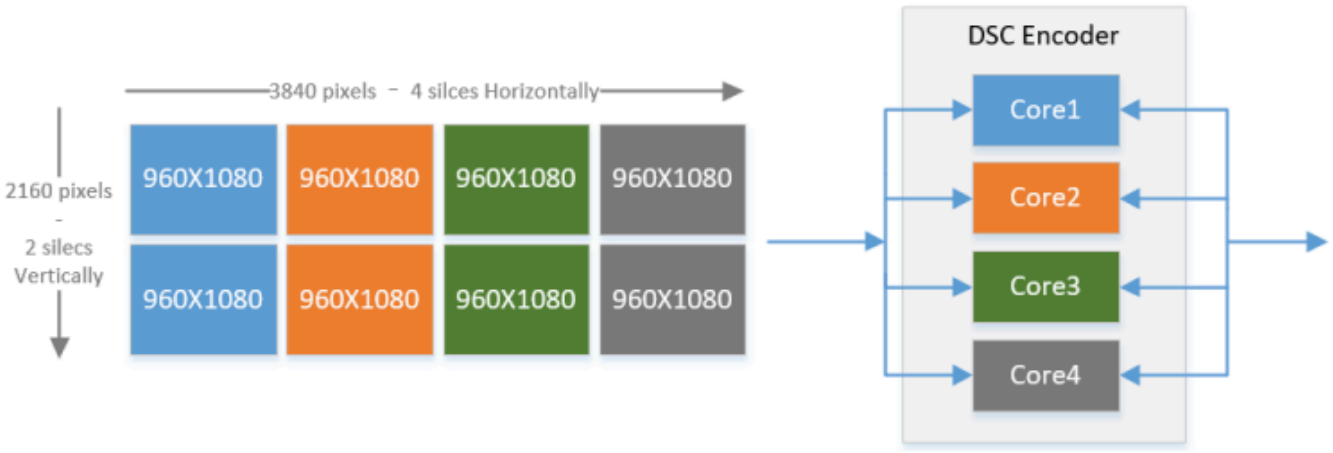
Slice

When verifying whether the RK3588 DSC supports a specific resolution with a given number of Slices in an actual display solution, refer to the following formula:

```
DSC_8K: active_slice_num * slice_width <= 960 * 8
DSC_4K: active_slice_num * slice_width <= 2048 * 2
```

11.2 DSC Encode

Prior to image data compression, the Source Device and Sink Device negotiate a mutually supported number of slices to partition the image into an equal grid. Taking 3840x2160 as an example, if both the Source Device and Sink Device support 8 slices for DSC, the image can be divided equally into 4 parts horizontally and 2 parts vertically. If the DSC encoder comprises 4 independent cores capable of parallel image data compression, the 8 slices on the left side of the diagram can be grouped into 4 sets for simultaneous parallel compression. This effectively enables the transmission of 4K content with low bandwidth and low latency.



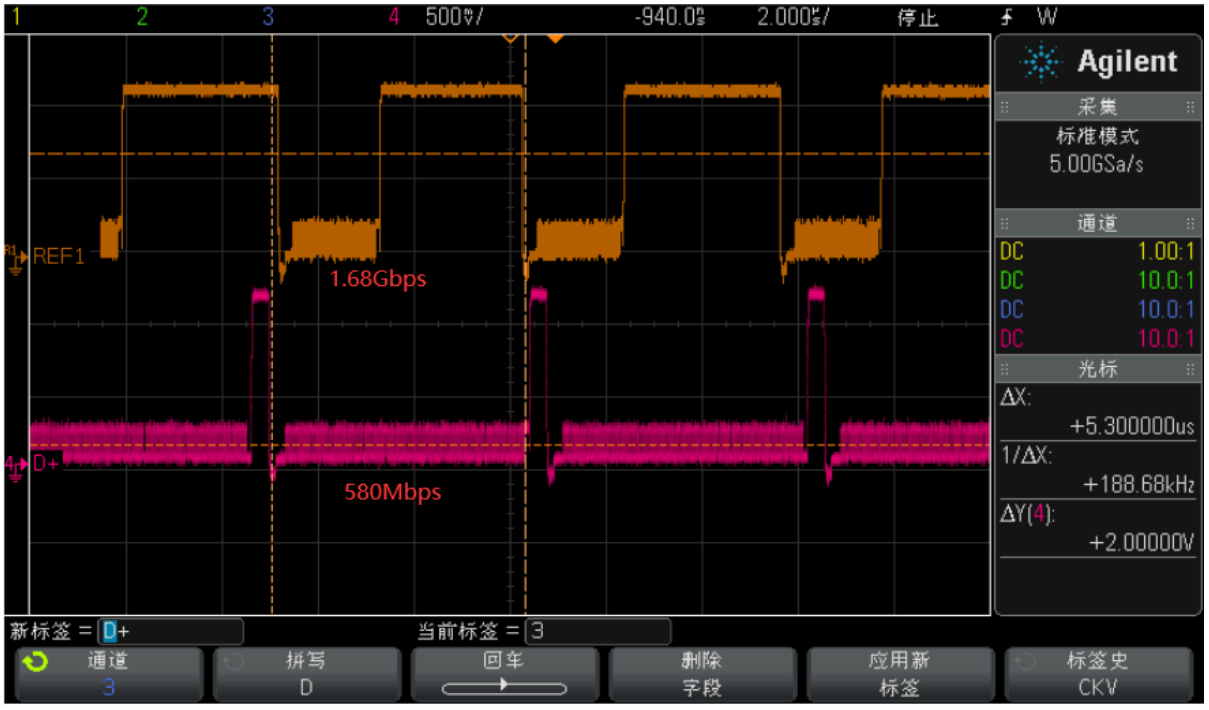
DSC-Encode

11.3 DSC Bandwidth

Taking 4Kp60 resolution as an example, the MIPI D/C-PHY bandwidth changes before and after enabling DSC are as follows. After enabling DSC, the link bandwidth can be reduced to one-third to one-half of the original bandwidth.

resolution	PHY-TYPE	带宽（no dsc）	带宽 （dsc 8bits rgb@60）
3840x2160: 24bits rgb@60	D-PHY （vdieo burst）	3.96 x 4 Gbps	1.32 x 4 Gbsp
	C-PHY （vdieo burst）	2.32 x 3 Gsps	0.774 x 3 Gsps

The following figure shows the signal waveforms measured at the original bandwidth and one-third of the original bandwidth for the RK3588 MIPI DSI Driver driving a 1440x3120p60 4-lane display module with DSC enabled.



11.4 PPS

The PPS has a total length of 128 bytes. The table below describes key information required by the DSC system:

1. DSC version;
2. BPC before DSC compression encoding;
3. BPP after DSC compression encoding;
4. Width and height of the original input image before DSC compression encoding;
5. Width and height of the DSC compression encoding Slice.

1600 2560 slice 40	Dec	Hex
force_reg_dsc_version_major[3:0]	1	1
force_reg_dsc_version_minor[3:0]	1	1
force_reg_pps_identifier[7:0]	0	00
force_reg_bits_per_component[3:0]	8	8
force_reg_linebuf_depth[3:0]	9	9
force_reg_block_pred_enable	1	1
force_reg_convert_rgb	1	1
force_reg_simple_422	0	0
force_reg_vbr_enable	0	0
force_reg_bits_per_pixel[9:0]	128	80
force_reg_pic_height[15:0]	2560	0A00
force_reg_pic_width[15:0]	1600	0640
force_reg_slice_height[15:0]	40	28
force_reg_slice_width[15:0]	1600	0640
force_reg_chunk_size[15:0]	1600	0640
force_reg_initial_xmit_delay[9:0]	512	0200
force_reg_initial_dec_delay[15:0]	1057	0421
force_reg_initial_scale_value[5:0]	32	0020
force_reg_scale_inc_interval[15:0]	1488	05D0
force_reg_scale_dec_interval[11:0]	22	0016
force_reg_first_line_bpg_offset[4:0]	12	0C
force_reg_nfl_bpg_offset[15:0]	631	0277
force_reg_slice_bpg_offset[15:0]	218	00DA
force_reg_initial_offset[15:0]	6144	1800
force_reg_final_offset[15:0]	4320	10E0
force_reg_flatness_min_qp[4:0]	3	03
force_reg_flatness_max_qp[4:0]	12	0C
	PPS	

	PPS setting	
PA1	11	PPS1
PA2	00	PPS2
PA3	00	PPS3
PA4	89	PPS4
PA5	30	PPS5
PA6	80	PPS6
PA7	0A	PPS7
PA8	00	PPS8
PA9	06	PPS9
PA10	40	PPS10
PA11	00	PPS11
PA12	28	PPS12
PA13	06	PPS13
PA14	40	PPS14
PA15	06	PPS15
PA16	40	PPS16
PA17	02	PPS17
PA18	00	PPS18
PA19	04	PPS19
PA20	21	PPS20
PA21	00	PPS21
PA22	20	PPS22
PA23	05	PPS23
PA24	D0	PPS24
PA25	00	PPS25
PA26	16	PPS26
PA27	00	PPS27
PA28	0C	PPS28
PA29	02	PPS29
PA30	77	PPS30
PA31	00	PPS31

11.5 Example

11.5.1 When to Enable DSC

This section describes the conditions for enabling DSC in MIPI DSI display solutions for specific projects, using the following display module parameters provided by a customer as an example:

1. The module features a D-PHY interface with 4 data lanes, supporting a maximum rate of 1 Gbps/lane.
2. The module operates in Video Burst mode with a raw bandwidth requirement of 1.866 Gbps (24-bit RGB at 60 fps).
3. The module supports DSC with 1 slice.
4. The target rate for the module is 676 Mbps/lane.

4.5.1 Speed Setting

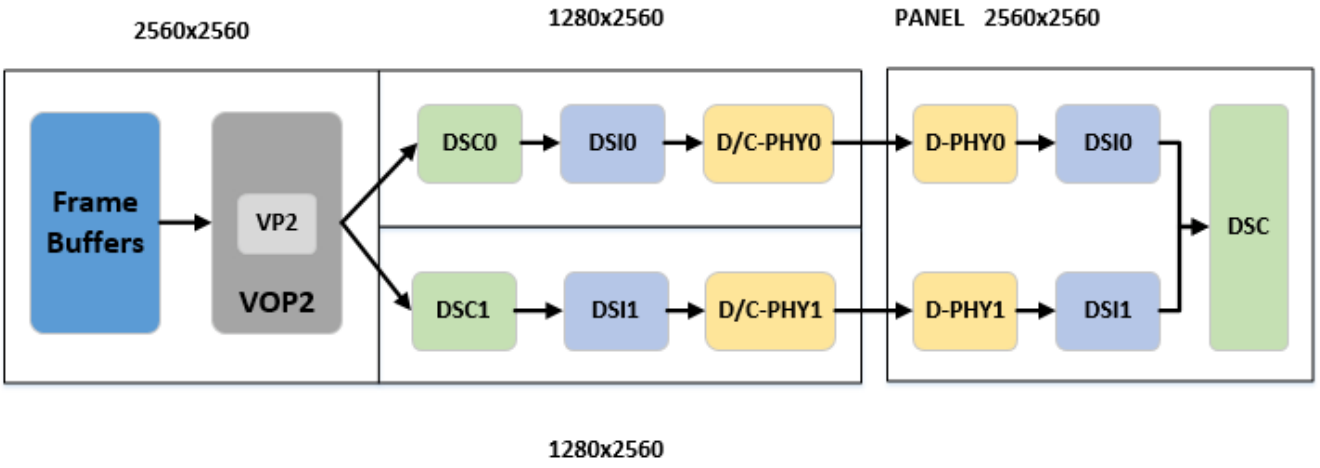
Item	Unit	Value
Frame Rate	Hz	60
Pixel Clock	MHz	84.5
MIPI Lane	Lane	1port* 4Lane
MIPI Speed	sps	676M

4.5.2 Porch Setting

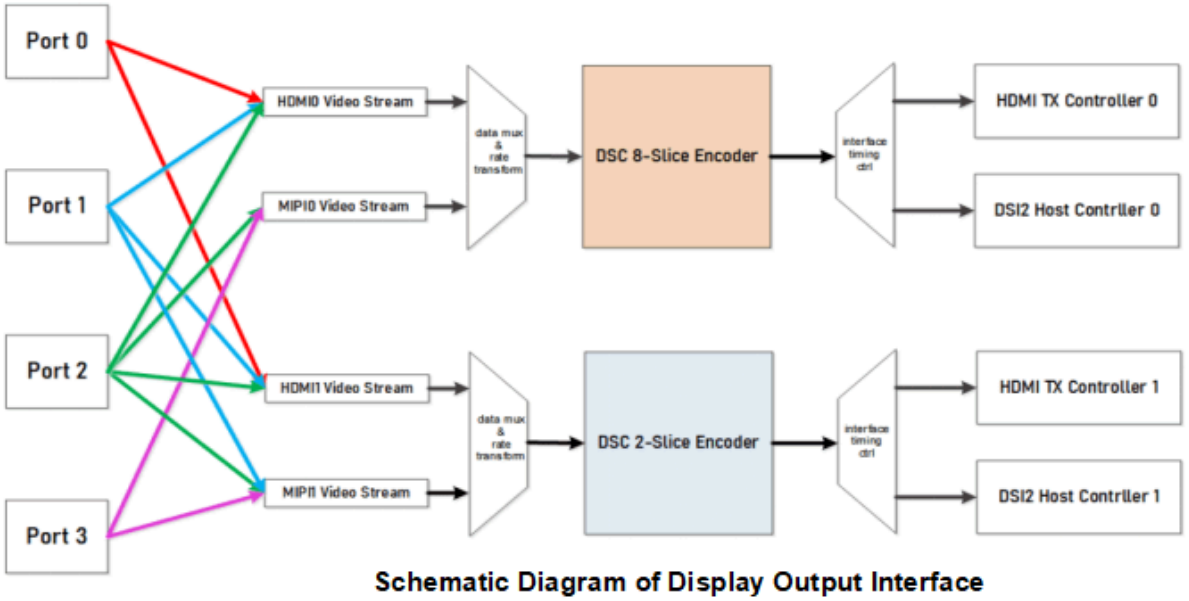
Item	Symbol	Value
Horizontal Sync Width	HSW	20
Horizontal Back Porch	HBP	40
Horizontal Active	HACT	1600
Horizontal Front Porch	HFP	118
Vertical Sync Width	VSW	4
Vertical Back Porch	VBP	18
Vertical Active	VACT	2560
Vertical Front Porch	VFP	60

11.5.2 How to Enable DSC for Dual-Channel MIPI

The figure below illustrates a customer solution featuring a dual-MIPI display module with DSC support, operating at 2560x2560p120. The corresponding display link block diagram is shown as follows.

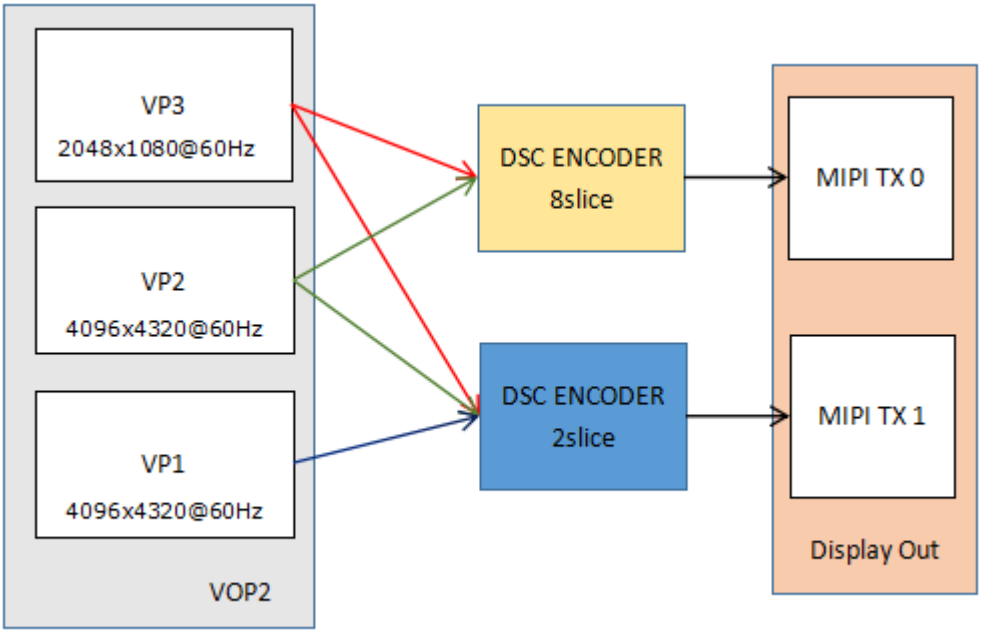


12. Display Route



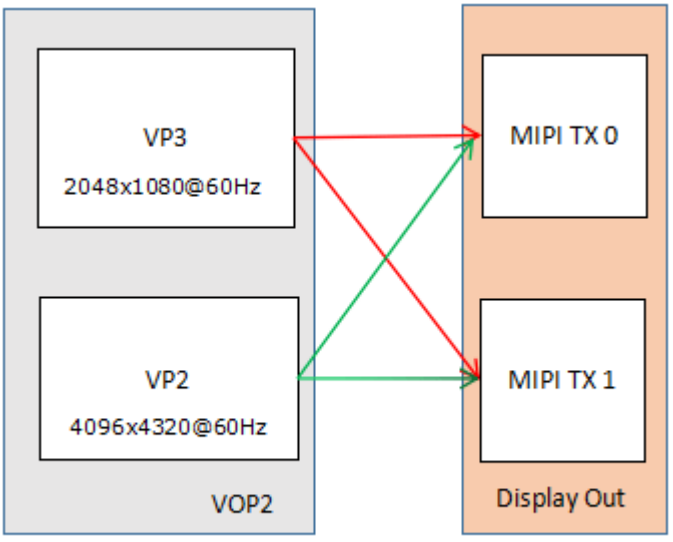
Schematic Diagram of Display Output Interface

12.1 MIPI with DSC



Schematic Diagram of Display Output Interface

12.2 MIPI with DSC Bypass



Schematic Diagram of Display Output Interface

12.3 DTS Configuration

For example, if DSI0 is attached to VP3:

```
&dsi0_in_vp2 {
    status = "disabled";
};

&dsi0_in_vp3 {
    status = "okay";
};
```

For example, mounting DSI1 to VP2:

```
&dsi1_in_vp2 {
    status = "okay";
};

&dsi1_in_vp3 {
    status = "disabled";
};
```

13. Boot Logo

13.1 route_dsi0

For example, vp3->dsi0 or vp3->dsc0->dsi0:

```
&route_dsi0 {
    status = "okay";
    connect = <&vp3_out_dsi0>;
};
```

13.2 route_dsi1

For example, vp2->dsi1 or vp2->dsc1->dsi1:

```
&route_dsi1 {
    status = "okay";
    connect = <&vp2_out_dsi1>;
};
```

13.3 route_dsi0 && route_dsi1

For example (vp3->dsi0 or vp3->dsc0->dsi0) && (vp2->dsi1 or vp2->dsc1->dsi1):

```
&route_dsi0 {
    status = "okay";
    connect = <&vp3_out_dsi0>;
};

&route_dsi1 {
    status = "okay";
    connect = <&vp2_out_dsi1>;
};
```

14. DSI HOST

The DSI can be configured in Manual or Auto-Calculation mode. By default, the Driver configures it in Manual mode. Users may optionally configure it in Auto-Calculation mode, which offers the following features:

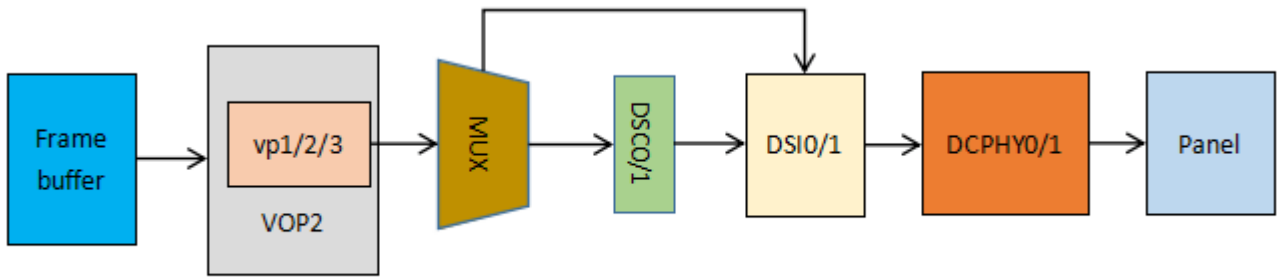
The host is configured to automatically extract all necessary information and timing parameters from the IPI and PPI interfaces to meet frame timing requirements in video mode. The Auto-Calculation mode provides the following advantages:

- Eliminates the need for users to configure a set of Registers
- Simplifies the configuration process and reduces the likelihood of unexpected controller behavior
- Allows dynamic changes to IPI frames while the controller adapts to output frame variations

ports
In the following examples, ports are used to associate the Display Interface with the panel.
For detailed configuration instructions, refer to the following document:

```
Documentation/devicetree/bindings/graph.txt
```

14.1 Single DSI



单DSI Display Route

14.1.1 DSI0

```
&dsi0 {
    status = "okay";
    //rockchip,lane-rate = <1000>;
    //auto-calculation-mode;
    //disable-hold-mode;
    //support-psr;

    dsi0_panel: panel@0 {
        status = "okay";
        compatible = "simple-panel-dsi";
        ...

        ports {
            #address-cells = <1>;
            #size-cells = <0>;

            port@0 {
                reg = <0>;
                panel_in_dsi: endpoint {
                    remote-endpoint = <&dsi_out_panel>;
                };
            };
        };
    };

    ports {
        #address-cells = <1>;
        #size-cells = <0>;

        port@1 {
            reg = <1>;
            dsi_out_panel: endpoint {
                remote-endpoint = <&panel_in_dsi>;
            };
        };
    };

};

&mipi_dcphy0 {
    status = "okay";
};
```

14.1.2 DSI1

```
&dsi1 {
    status = "okay";
    //rockchip,lane-rate = <1000>;
    //auto-calculation-mode;
    //disable-hold-mode;
    //support-psr;

    dsi1_panel: panel@0 {
        status = "okay";
        compatible = "simple-panel-dsi";
        ...

        ports {
            #address-cells = <1>;
            #size-cells = <0>;

            port@0 {
                reg = <0>;
                panel_in_dsi1: endpoint {
                    remote-endpoint = <&dsi1_out_panel>;
                };
            };
        };
    };

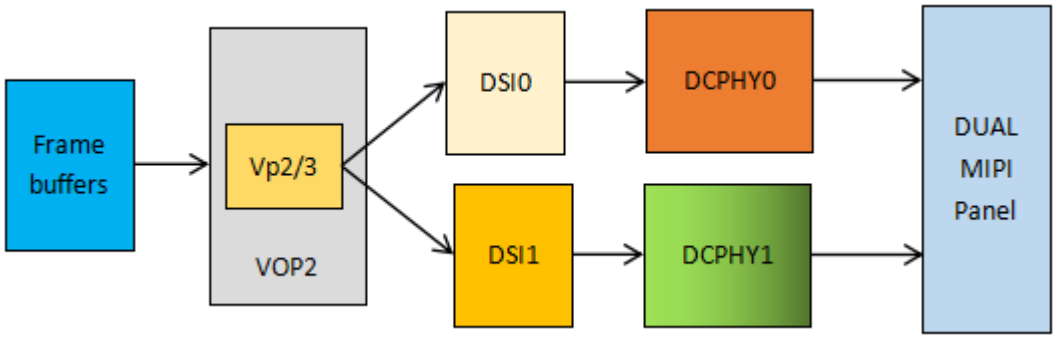
    ports {
        #address-cells = <1>;
        #size-cells = <0>;

        port@1 {
            reg = <1>;
            dsi1_out_panel: endpoint {
                remote-endpoint = <&panel_in_dsi1>;
            };
        };
    };
};

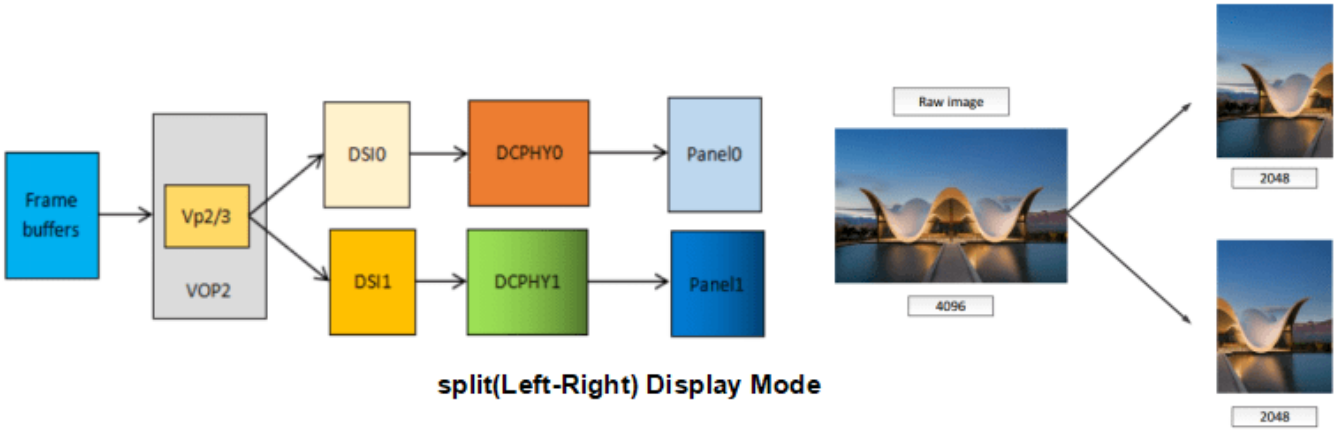
&mipi_dcphy1 {
    status = "okay";
};
```

14.2 Dual-Channel DSI

MODE1:



MODE2:



Note the following properties highlighted in red for dual-channel configuration:

rockchip,dual-channel = <&dsi1>

dsi,lanes = <8>;//For DPHY panels; change to 6 for CPHY panels

```
&dsi0 {
    status = "okay";
    rockchip,dual-channel = <&dsi1>;
    //auto-calculation-mode;
    //disable-hold-mode;
    //support-psr;

    dsi0_panel {
        status = "okay";
        compatible = "simple-panel-dsi";
        dsi,lanes  = <8>;
        ...

        display-timings {
            native-mode = <&timing0>;

            timing0: timing0 {
                clock-frequency = <260000000>;
                hactive = <1440>;
                vactive = <2560>;
                hfront-porch = <150>;
                hsync-len = <30>;
                hback-porch = <60>;
                vfront-porch = <8>;
```

```
            vsync-len = <4>;
            vback-porch = <4>;
            hsync-active = <0>;
            vsync-active = <0>;
            de-active = <0>;
            pixelclk-active = <0>;
          };
        };

        ports {
          #address-cells = <1>;
          #size-cells = <0>;

          port@0 {
            reg = <0>;
            panel_in_dsi0: endpoint {
              remote-endpoint = <&dsi0_out_panel>;
            };
          };
        };
      };

      ports {
        #address-cells = <1>;
        #size-cells = <0>;

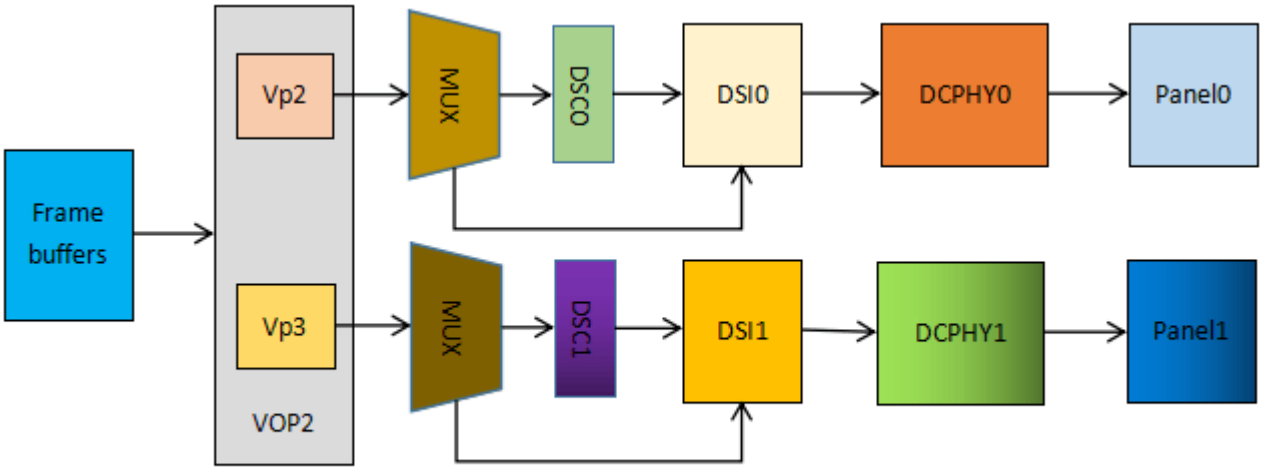
        port@1 {
          reg = <1>;
          dsi0_out_panel: endpoint {
            remote-endpoint = <&panel_in_dsi0>;
          };
        };
      };
    };

    &dsi1 {
      status = "okay";
    };

    &mipi_dcphy0 {
      status = "okay";
    };

    &mipi_dcphy1 {
      status = "okay";
    };
```

14.3 Dual-link DSI



Dual Link DSI Display Route

```
&dsi0 {
    status = "okay";
    //rockchip,lane-rate = <1000>;
    //auto-calculation-mode;
    //disable-hold-mode;
    //support-psr;

    dsi0_panel: panel@0 {
        status = "okay";
        compatible = "simple-panel-dsi";
        ...

        ports {
            #address-cells = <1>;
            #size-cells = <0>;

            port@0 {
                reg = <0>;
                panel_in_dsi: endpoint {
                    remote-endpoint = <&dsi_out_panel>;
                };
            };
        };
    };

    ports {
        #address-cells = <1>;
        #size-cells = <0>;

        port@1 {
            reg = <1>;
            dsi_out_panel: endpoint {
                remote-endpoint = <&panel_in_dsi>;
            };
```

```
        };
    };

};

&dsi1 {
    status = "okay";
    //rockchip,lane-rate = <1000>;
    //auto-calculation-mode;
    //disable-hold-mode;
    //support-psr;

    dsi1_panel: panel@0 {
        status = "okay";
        compatible = "simple-panel-dsi";
        ...

        ports {
            #address-cells = <1>;
            #size-cells = <0>;

            port@0 {
                reg = <0>;
                panel_in_dsi1: endpoint {
                    remote-endpoint = <&dsi1_out_panel>;
                };
            };
        };
    };

    ports {
        #address-cells = <1>;
        #size-cells = <0>;

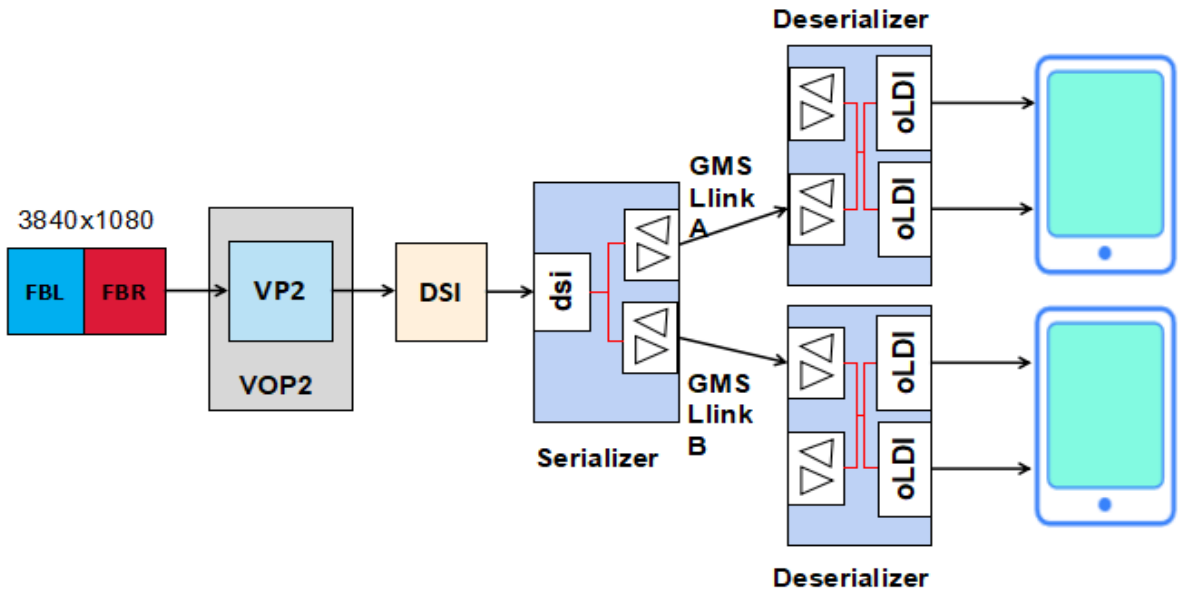
        port@1 {
            reg = <1>;
            dsi1_out_panel: endpoint {
                remote-endpoint = <&panel_in_dsi1>;
            };
        };
    };
};

&mipi_dcphy0 {
    status = "okay";
};

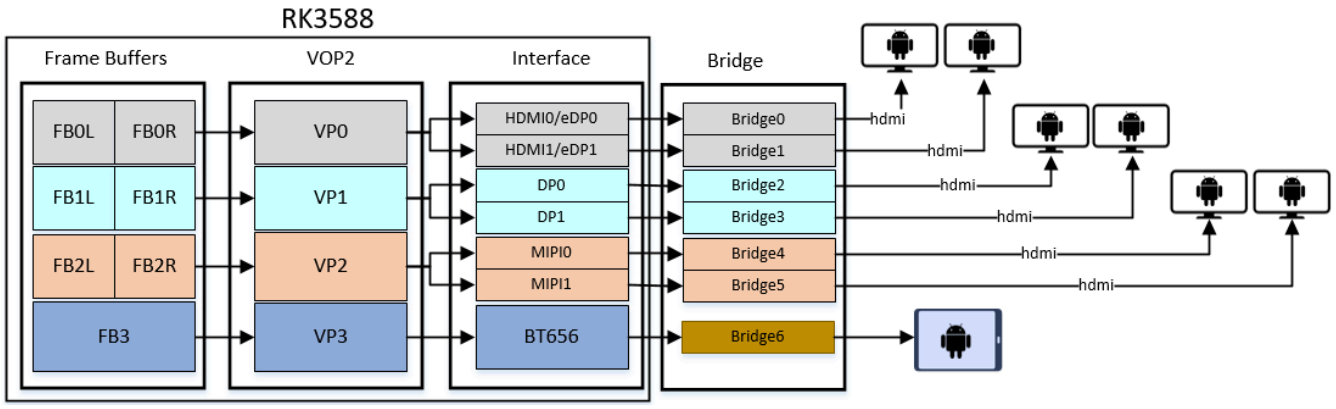
&mipi_dcphy1 {
    status = "okay";
};
```

14.4 DSI Application Scenarios

14.4.1 DSI + SerDes Solution



14.4.2 Multi-Display Connection Solution



15. DC-PHY

In practical application configurations, the default setting is D-PHY. As described in the panel configuration section, it can be configured to C-PHY as follows:

```
dsi0_panel: panel@0 {
    ...
    phy-c-option;
    ...
};
```

15.1 D-PHY

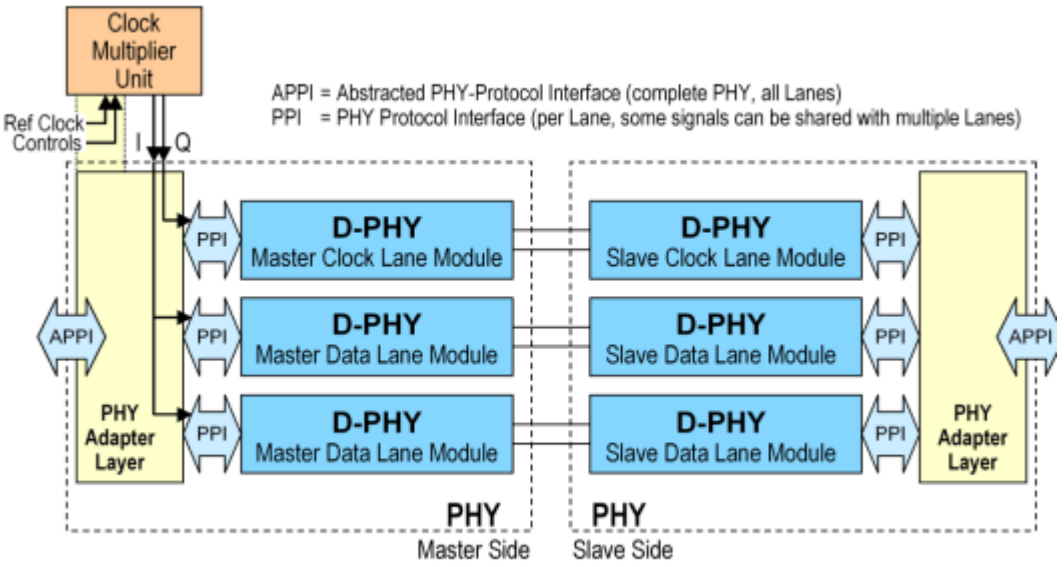


Figure 2 Two Data Lane PHY Configuration

1. Up to 4.5 Gbps per lane in D-PHY;
2. One D-PHY port supports up to 4 lanes, with each lane composed of a differential pair;
3. "D" originally referred to the Roman numeral 500 or the letter "D". When D-PHY was first introduced, the maximum data rate was 1 Gbps (DDR), corresponding to a clock rate of 500 MHz. Current D-PHY rates exceed these values, so "D" no longer carries specific meaning.

15.2 C-PHY

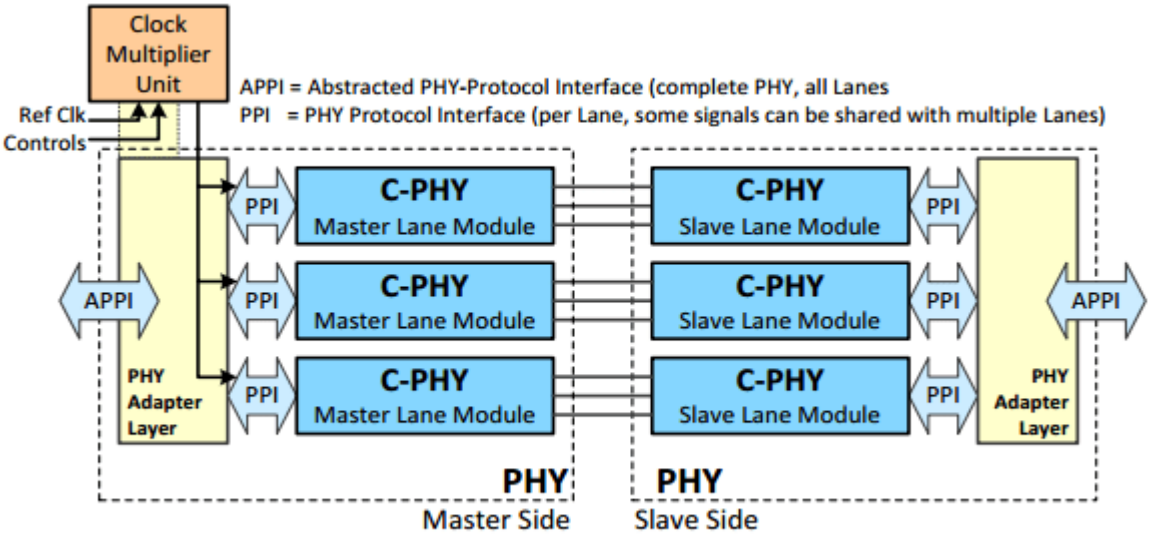


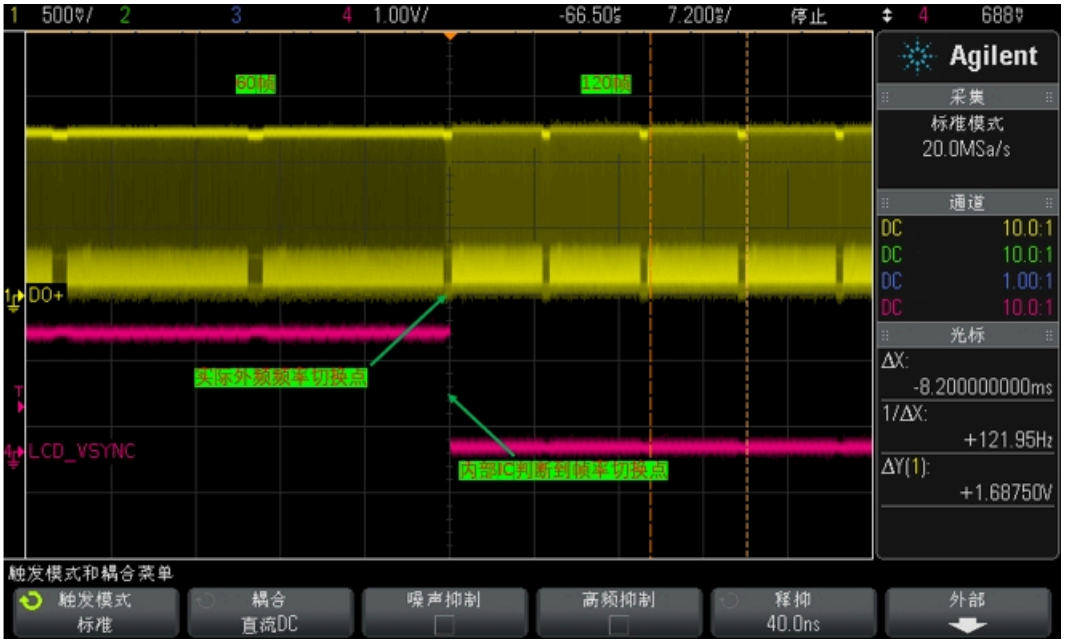
Figure 5 Three Lane PHY Configuration

1. Up to 2.0 Gsps per trio in C-PHY;
2. A C-PHY port supports up to 3 lanes, with each lane composed of three-wire trios;
3. C-PHY does not have a dedicated clock channel; its clock is embedded within the communication timing;

4. "C" stands for Channel-limited, where the channel count is limited to 3. C-PHY comprises a total of 3 lanes, with each lane utilizing a 3-wire transmission mode.

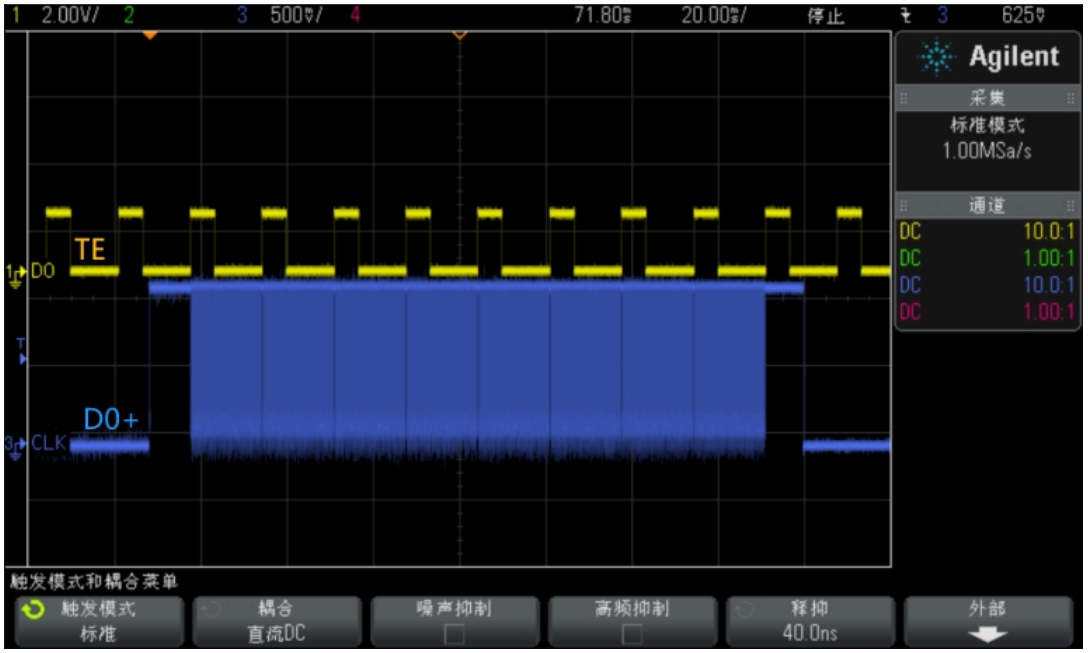
16. Dynamic Frame Rate Adjustment

MIPI DSI supports dynamic frame rate switching, such as seamless transitions between 120fps and 60fps. For further details, refer to the document "Rockchip_Developer_Guide_VRR_EN".



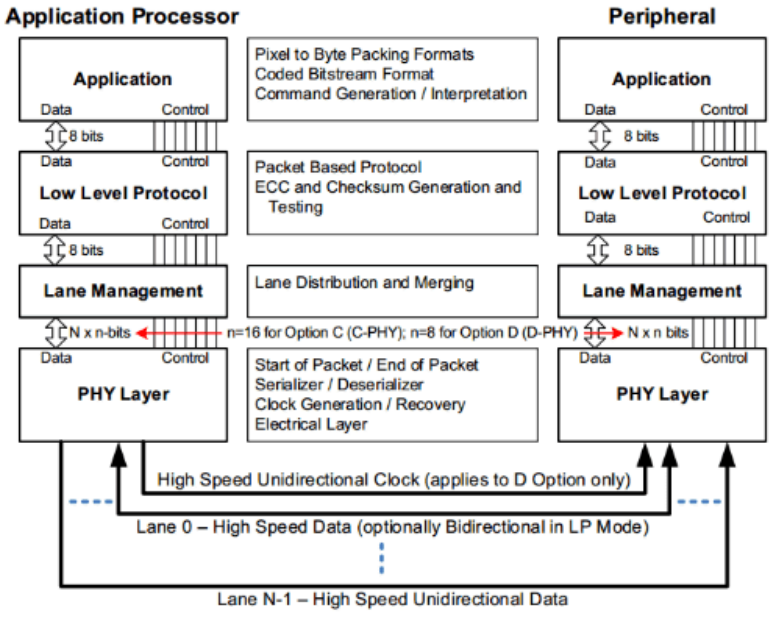
17. PSR

When MIPI DSI operates in COMMAND mode and the display panel includes built-in GRAM, the host controller's DSI output can be disabled during static frame display to reduce power consumption. In this state, the panel continuously refreshes static frames from its GRAM. When any content changes in a new frame, the host controller must re-enable the output and refresh the panel's GRAM to update the display.



18. Protocol Analysis

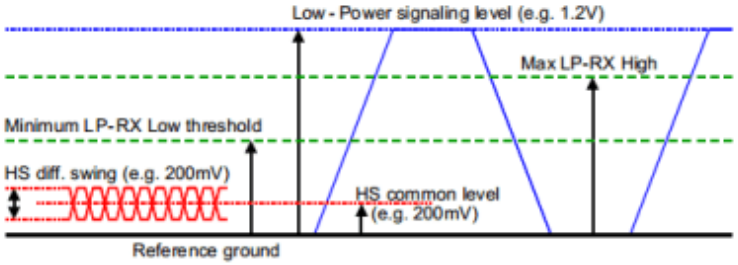
18.1 DSI Layer Definitions



- 像素数据打包，编码比特流
- 穿插ECC和CRC
- 像素数据在各个lane上分发和合并
- 物理链路上传输

18.2 D Option

18.2.1 Lane states and line levels



Line Levels

- **高速差分信号幅值：200mV**
- **Low-Power 信号幅值：1.2V**

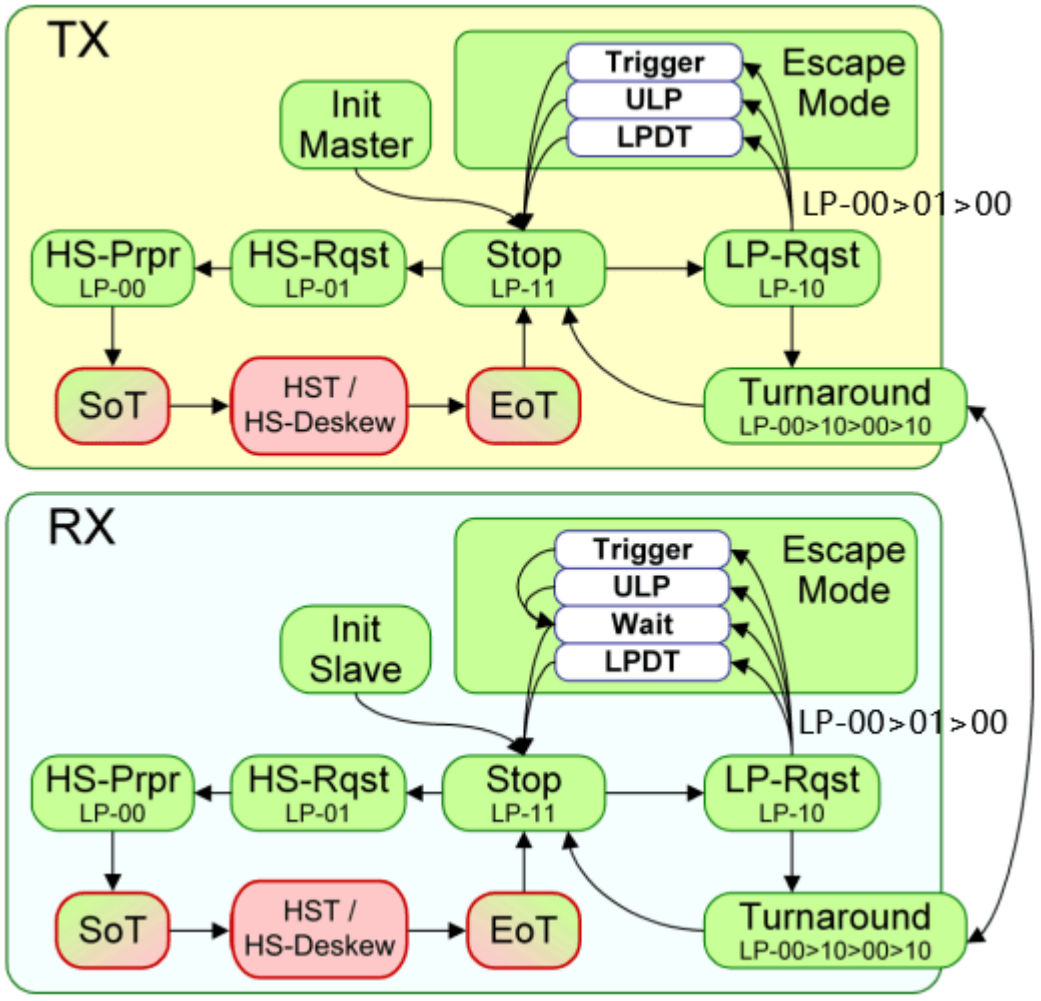
The following table lists all channel states that may occur during normal DPHY Lane operation.

State Code	Line Voltage Levels		High-Speed	Low-Power	
	Dp-Line	Dn-Line	Burst Mode	Control Mode	Escape Mode
HS-0	HS Low	HS High	Differential-0	N/A, Note 1	N/A, Note 1
HS-1	HS High	HS Low	Differential-1	N/A, Note 1	N/A, Note 1
LP-00	LP Low	LP Low	N/A	Bridge	Space
LP-01	LP Low	LP High	N/A	HS-Rqst	Mark-0
LP-10	LP High	LP Low	N/A	LP-Rqst	Mark-1
LP-11	LP High	LP High	N/A	Stop	N/A, Note 2

Note:

1. *During High-Speed transmission the Low-Power Receivers observe LP-00 on the Lines.*
2. *If LP-11 occurs during Escape mode the Lane returns to Stop state (Control Mode LP-11).*

18.2.2 Global Operation Flow Diagram



The DSI data lane can operate in the following three modes:

1. Escape Mode (only Dp0/Dn0 operates in this mode);
2. Bus Turnaround Request (only Dp0/Dn0 operates in this mode);
3. High-Speed Data Transmission.

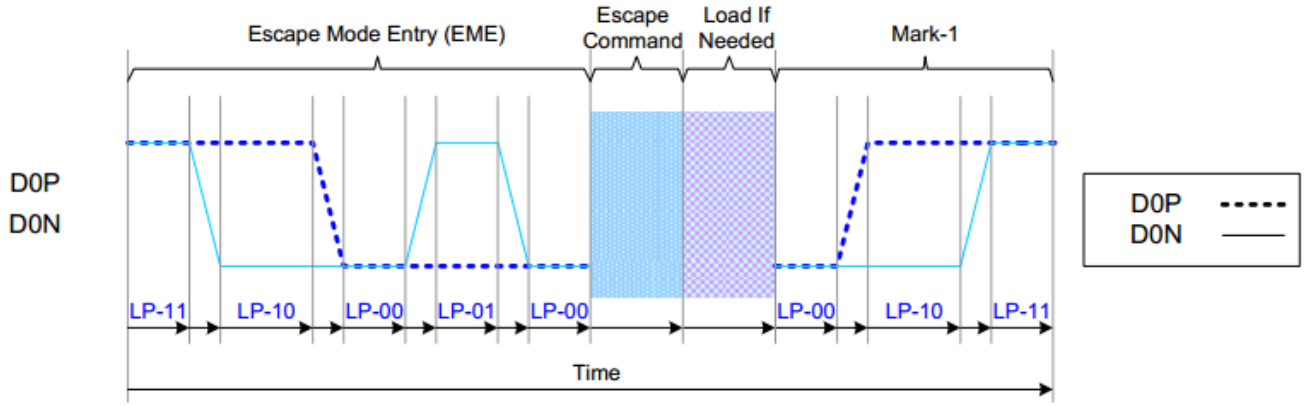
The definitions of these three modes and their corresponding entry sequences are as follows:

Mode	Entering Mode Sequence	Leaving Mode Sequence
Escape Mode[1]	LP-11 → LP-10 → LP-00 → LP-01 → LP-00	LP-00 → LP-10 → LP-11 (Mark-1)
High-Speed Data Transmission[2]	LP-11 → LP-01 → LP-00 → HS-0	(HS-0 or HS-1) → LP-11
Bus Turnaround Request[3]	LP-11 → LP-10 → LP-00 → LP-10 → LP-00	Hi-Z

Entering and Leaving Sequences

18.2.2.1 Escape Modes

When the data lane is in LP mode, data lane 0 is used for Escape Mode. The data lane shall enter Escape Mode via the sequence LP-11 -> LP-10 -> LP-00 -> LP-01 -> LP-00, and exit Escape Mode via the sequence LP-00 -> LP-10 -> LP-11.



General Escape Mode Sequence

18.2.2.1.1 Escape Commands

Once the data lane enters Escape mode, the transmitter shall send 8-bit Escape Commands to indicate the requested operation. The Escape Commands are as follows:

Escape Commands

Escape command	Command Type Mode/Trigger	Entry command Pattern (First Bit ➔ Last Bit Transmitted)	Dn	D0
Low-Power Data Transmission	Mode	1110 0001 b	-	X
Ultra-Low Power Mode	Mode	0001 1110 b	X	X
Undefined-1, Note 1	Mode	1001 1111 b	-	-
Undefined-2, Note 1	Mode	1101 1110 b	-	-
Remote Application Reset	Trigger	0110 0010 b	-	X
Acknowledge	Trigger	0010 0001 b	-	X
Unknown-5, Note 1	Trigger	1010 0000 b	-	-

Notes:

1. This Escape command support is not implemented on the display module.

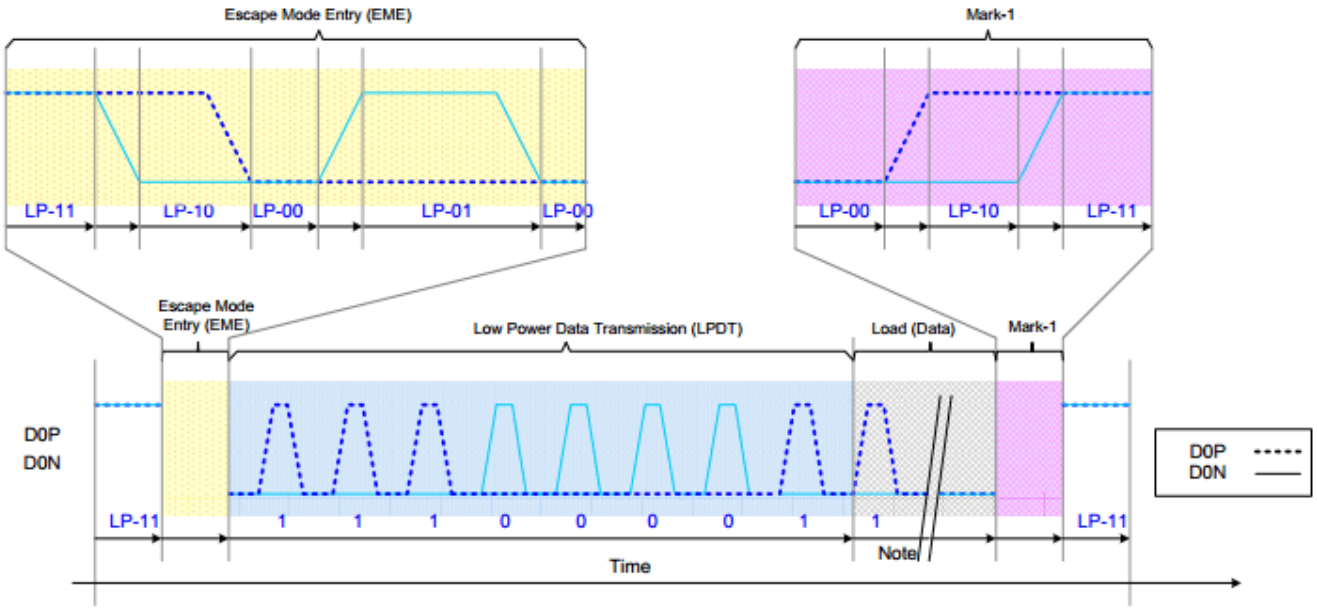
2. n = 1

3. x = Supported

4. - = Not Supported

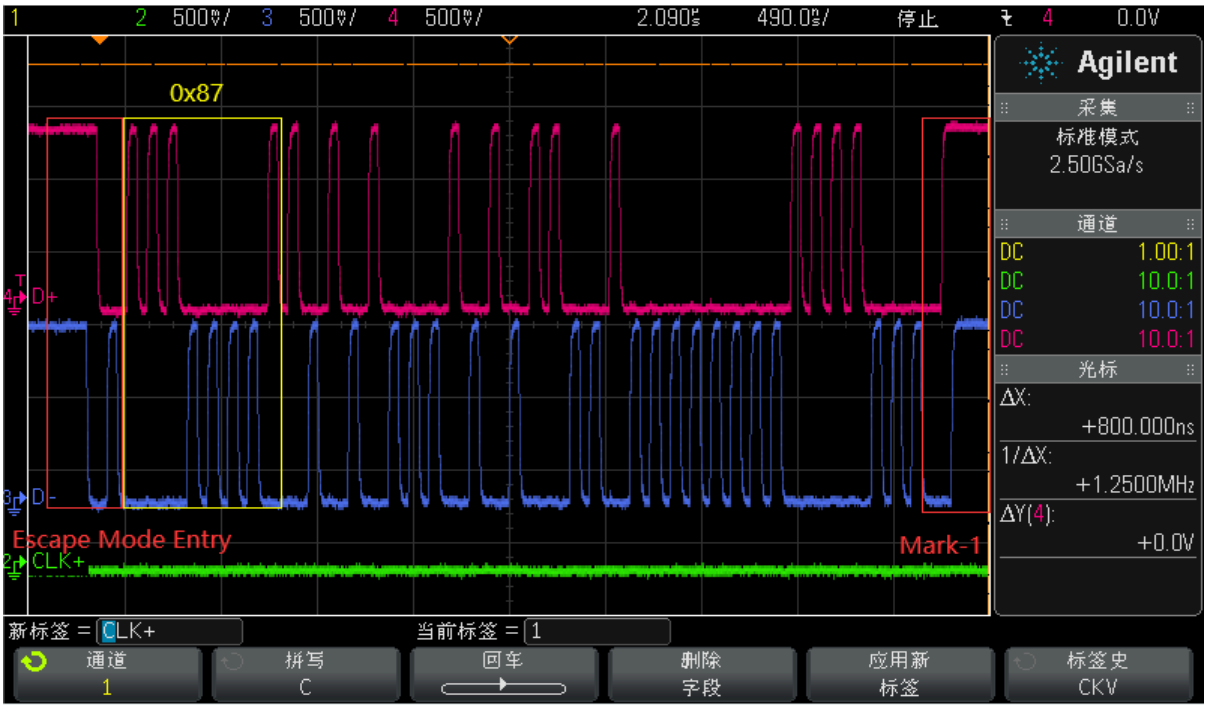
18.2.2.1.2 LPDT

When the data lane enters Escape mode and transmits a Low-Power Data Transmission (LPDT) sequence to the display module, the SoC's MIPI DSI TX can send data to the display module in LPDT mode. This method is typically used to download initialization sequences to the MIPI display module.

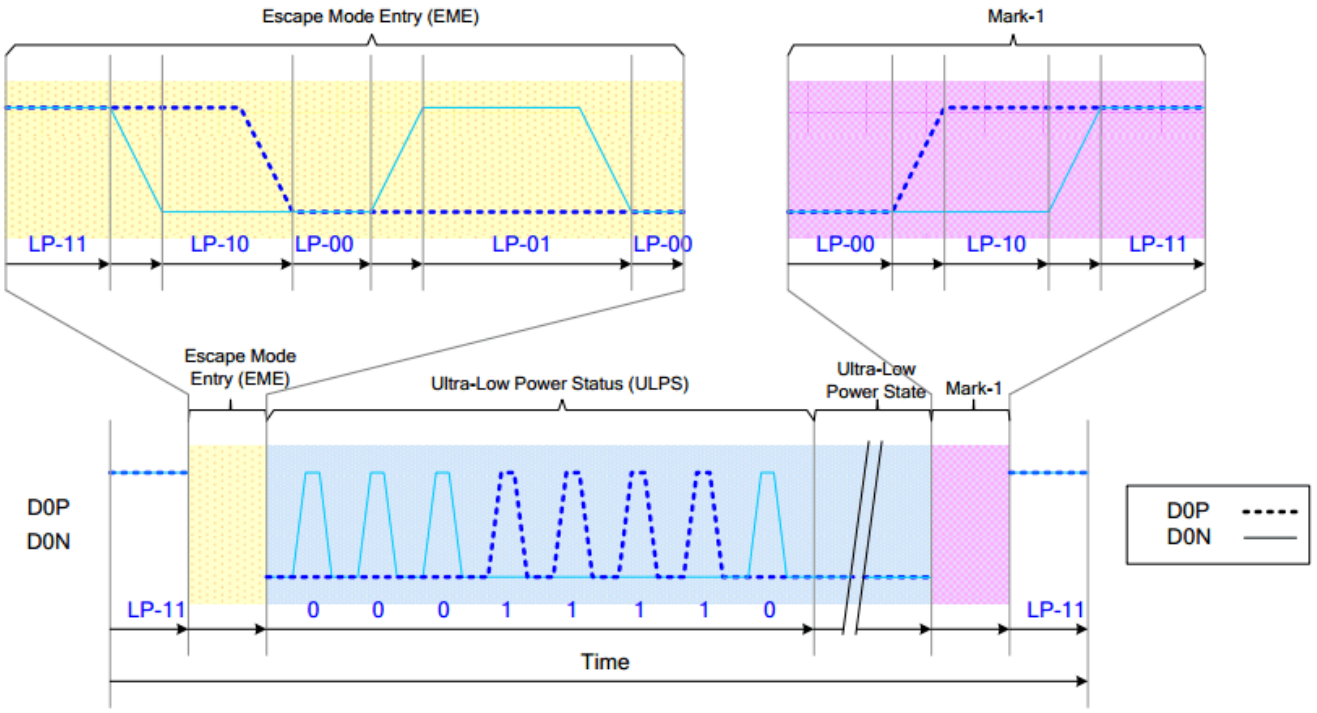


Low-Power DATA Transmission(LPDT)

The LPDT waveform captured by the oscilloscope is shown below:



18.2.2.1.3 ULPS



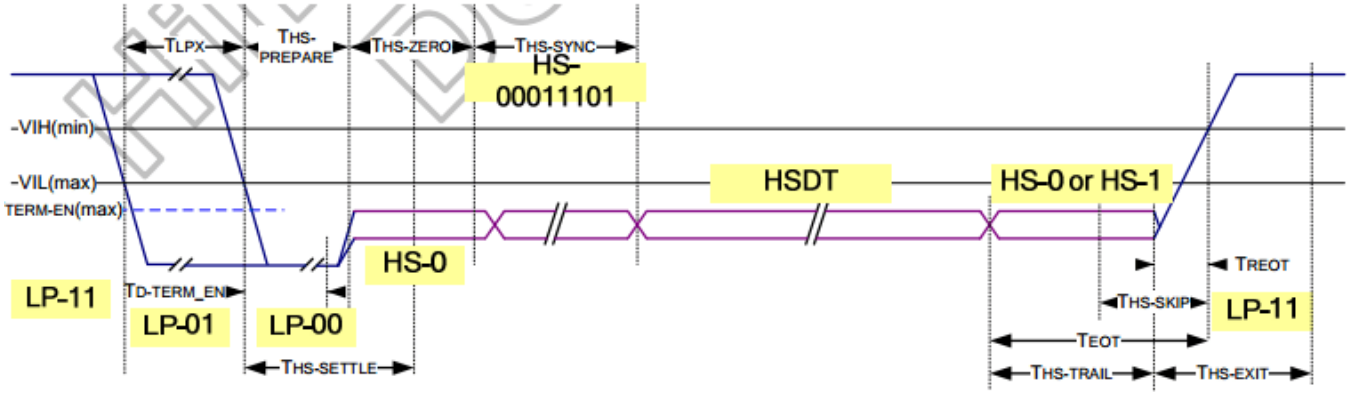
Ultra-Low Power State(ULPS)

18.2.2.2 HSDT

When the D-PHY clock lane operates in high-speed clock mode, the display module can enter high-speed data transmission mode. All data lanes simultaneously enter high-speed data transmission mode but may exit high-speed mode at different times. Data lanes enter high-speed mode via the following sequence:
LP-11 -> LP-01 -> LP-00 -> HS-0 -> SoT (0001_1101).

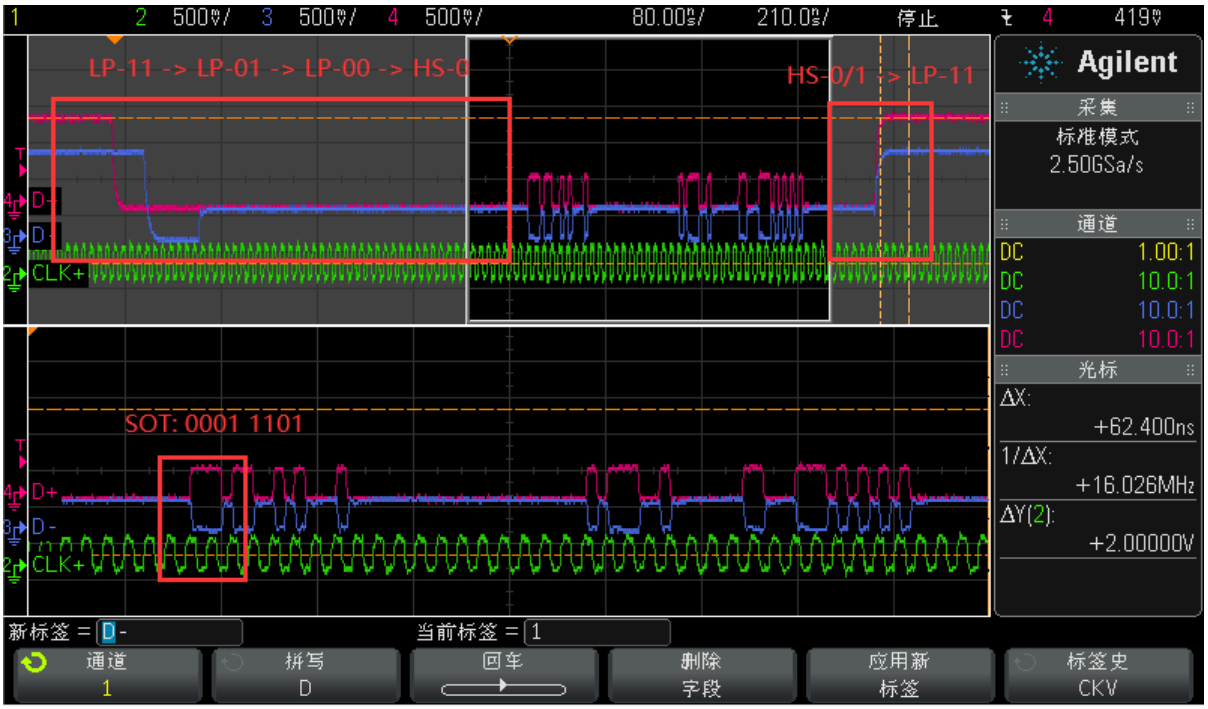
1. Start: LP-11
2. HS-Request: LP-01
3. HS-Settle: LP-00 -> HS-0 (RX: Lane Termination Enable)
4. Rx Synchronization: SoT(0001_1101)
5. End: High-Speed Data Transmission (HSDT) - Ready to receive High-Speed Data Load

Procedure for exiting high-speed data transmission mode on the data lane: Immediately after the last payload data, switch the differential state bit and maintain this state for a duration of Ths-trail.



High-Speed DATA Transmission (HSDT)

The HSDT waveform captured by the oscilloscope is shown below:

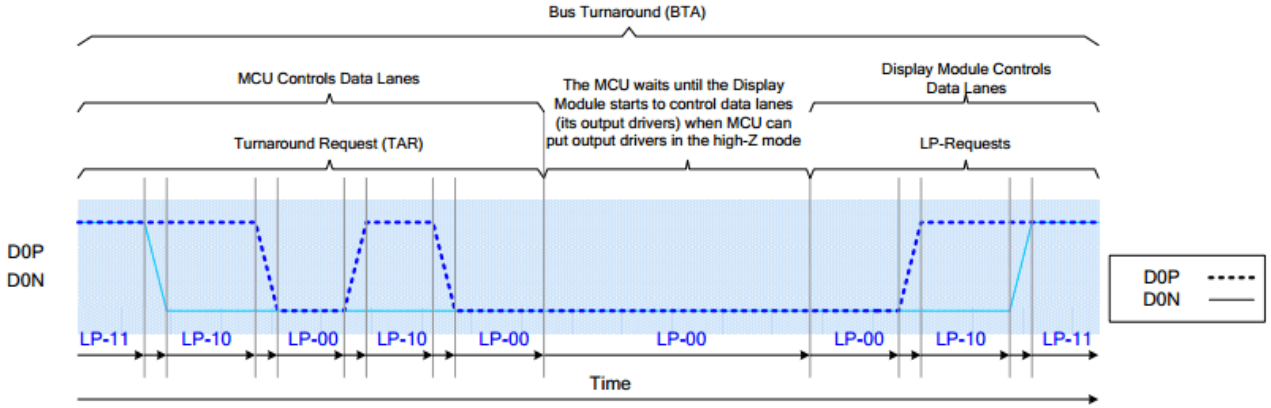


18.2.2.3 BTA

When information needs to be retrieved from the display module, the first data channel of the SoC DPHY can perform a bus turn-around procedure. The operation steps are as follows:

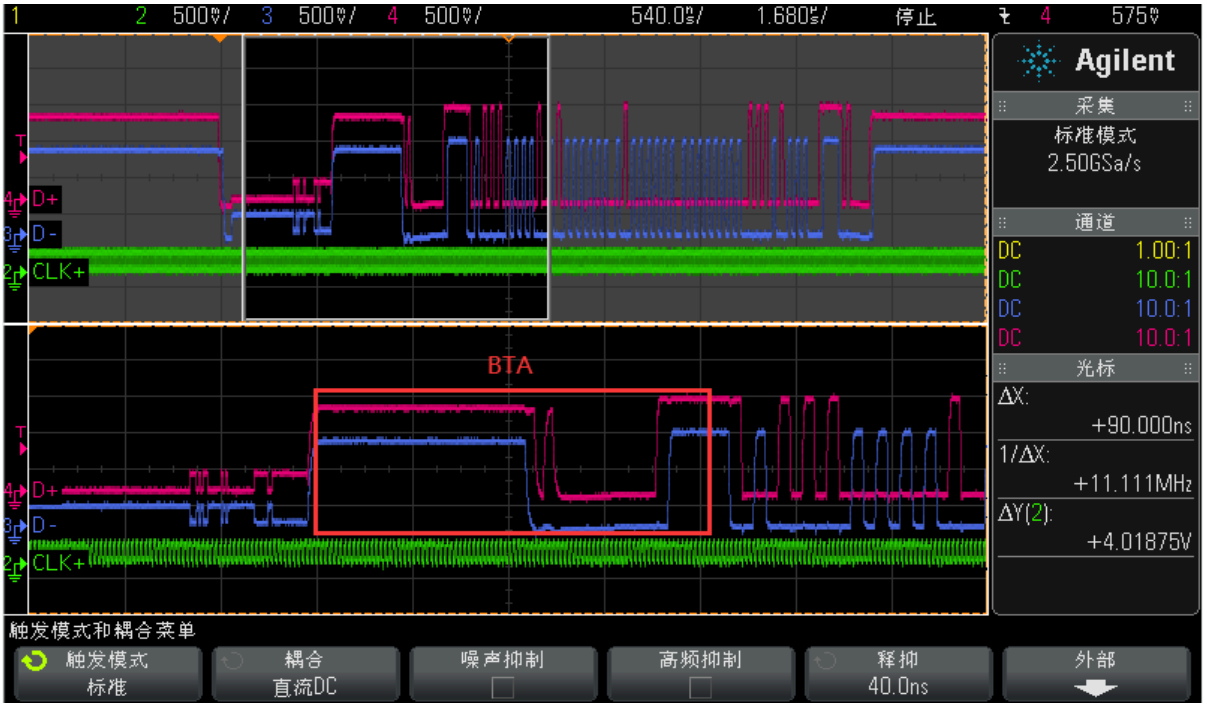
- Start (MCU): LP-11
- Turnaround Request (MCU): LP-11 => LP-10 => LP-00 => LP-10 => LP-00
- The MCU waits until the display module starts to control D0P/N data lanes and the MCU stops to control D0P/N data lanes (= High-Z)
- The display module changes to the stop mode: LP-00 => LP-10 => LP-11

The bus turnaround procedure (from the MCU to the display module) is illustrated below:



Bus Turnaround Procedure

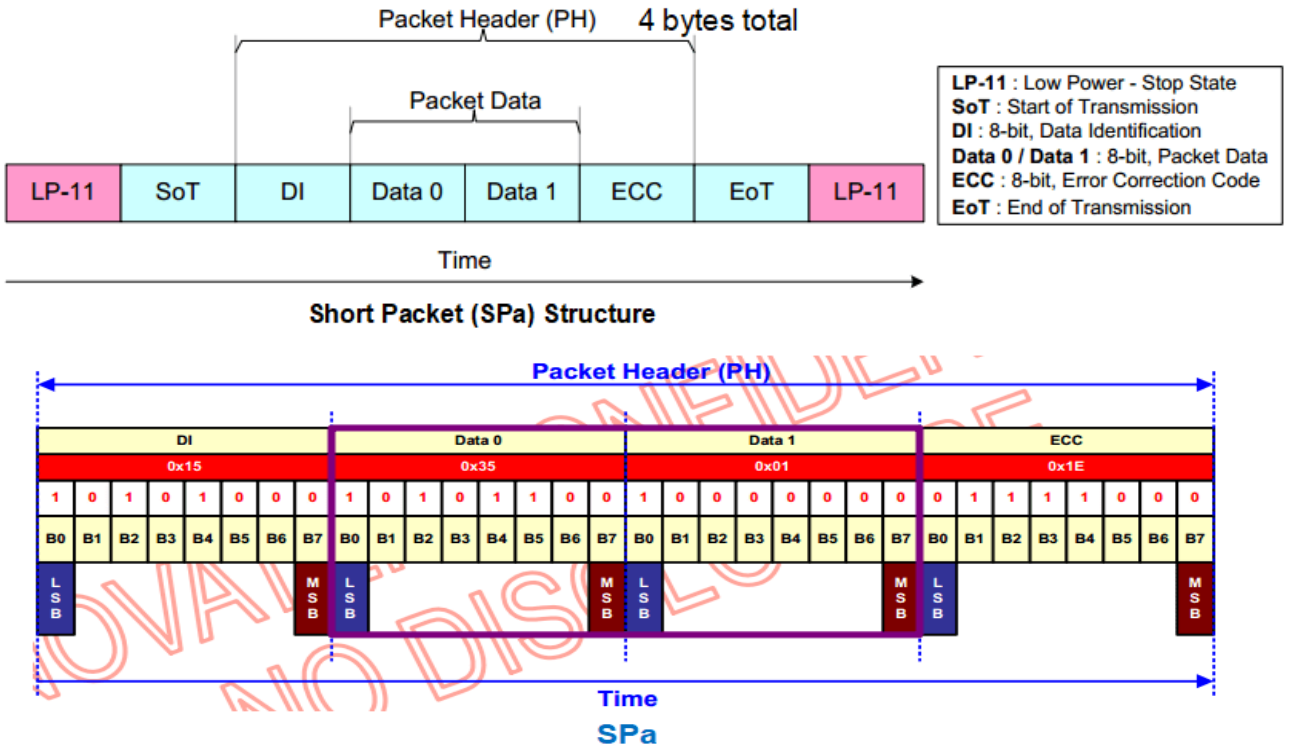
Capture the BTA waveform by reading back from the display module using an oscilloscope during HSDT mode as shown below:



18.2.3 Endian Policy

In both LP and HS data transmission modes, data is packetized into long packets and short packets before being transmitted to the display module.

18.2.3.1 SPa



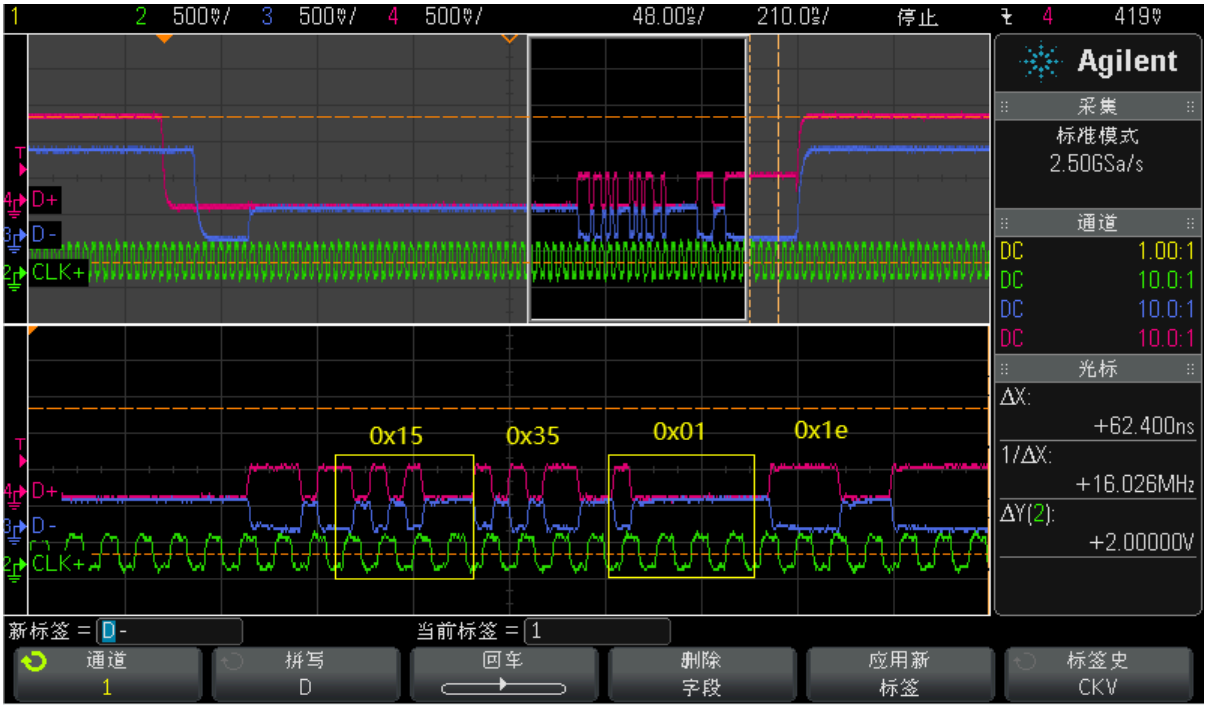
18.2.3.1.1 LPDT-SPa

The following waveform was captured using an oscilloscope while transmitting the above SPa to the display module in LPDT mode:

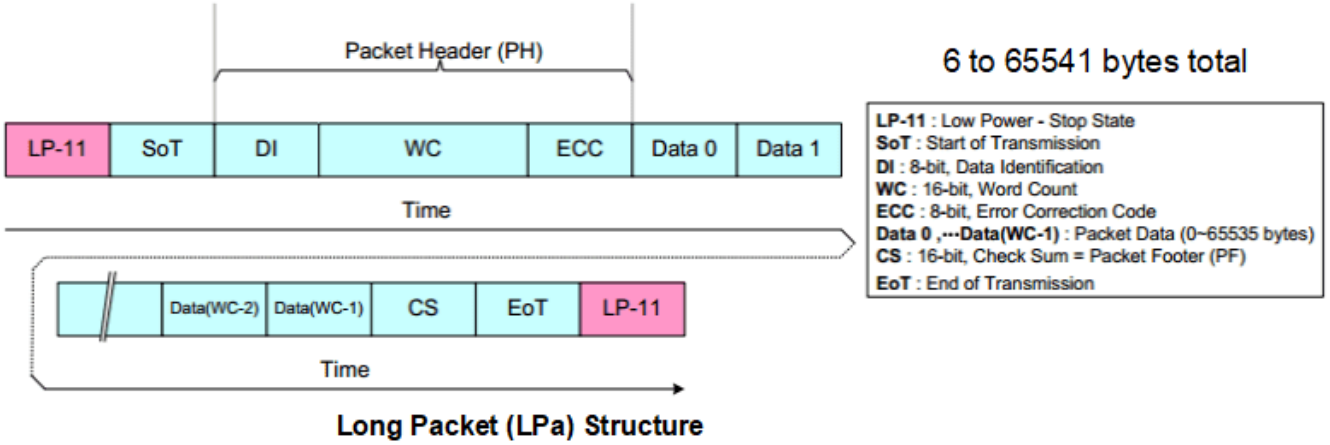


18.2.3.1.2 HSDT-SPa

The following waveform was captured using an oscilloscope while transmitting the above SPa to the display module in HSDT mode:

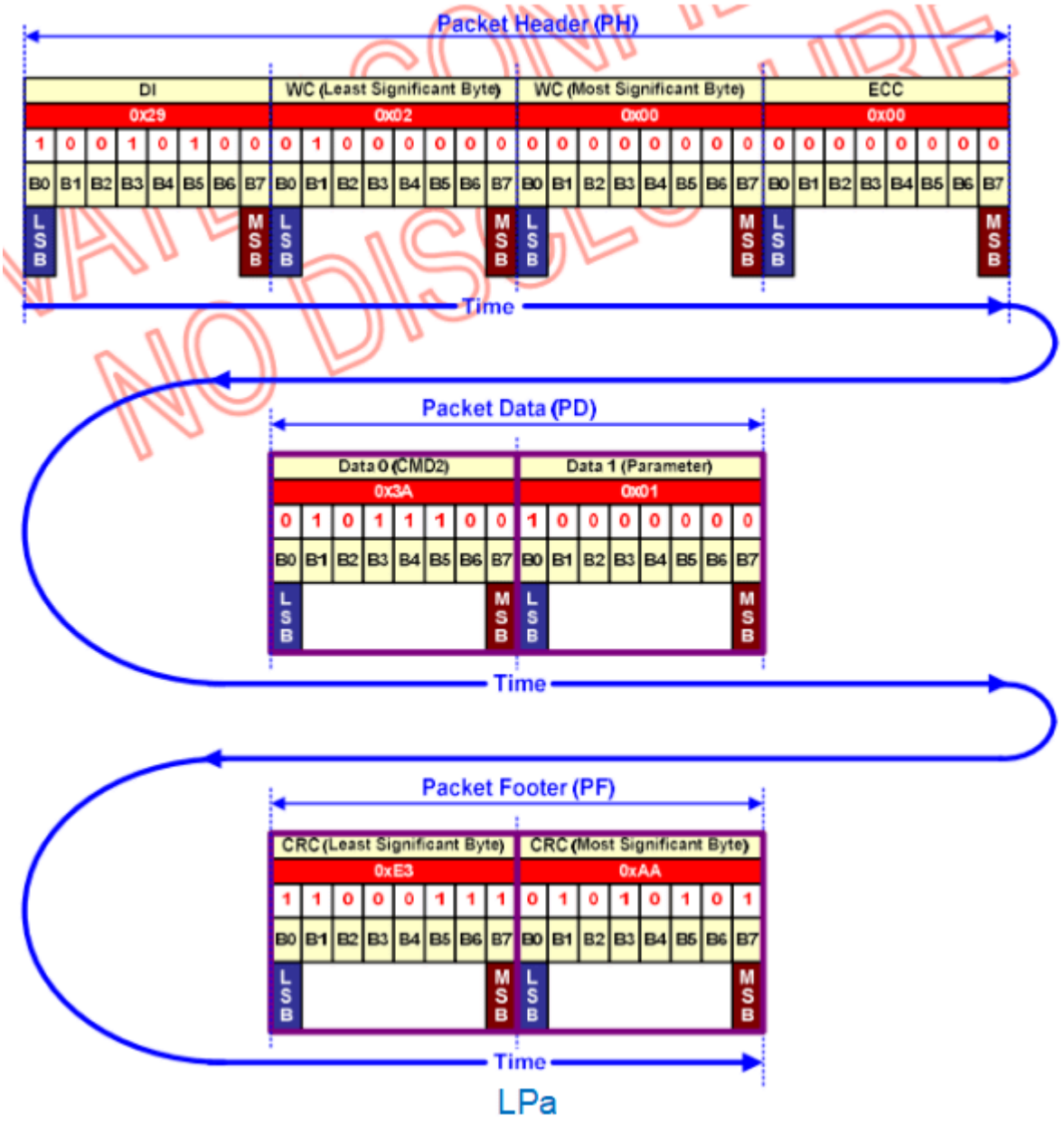


18.2.3.2 LPa



Long Packet (LPa) Structure

Example:



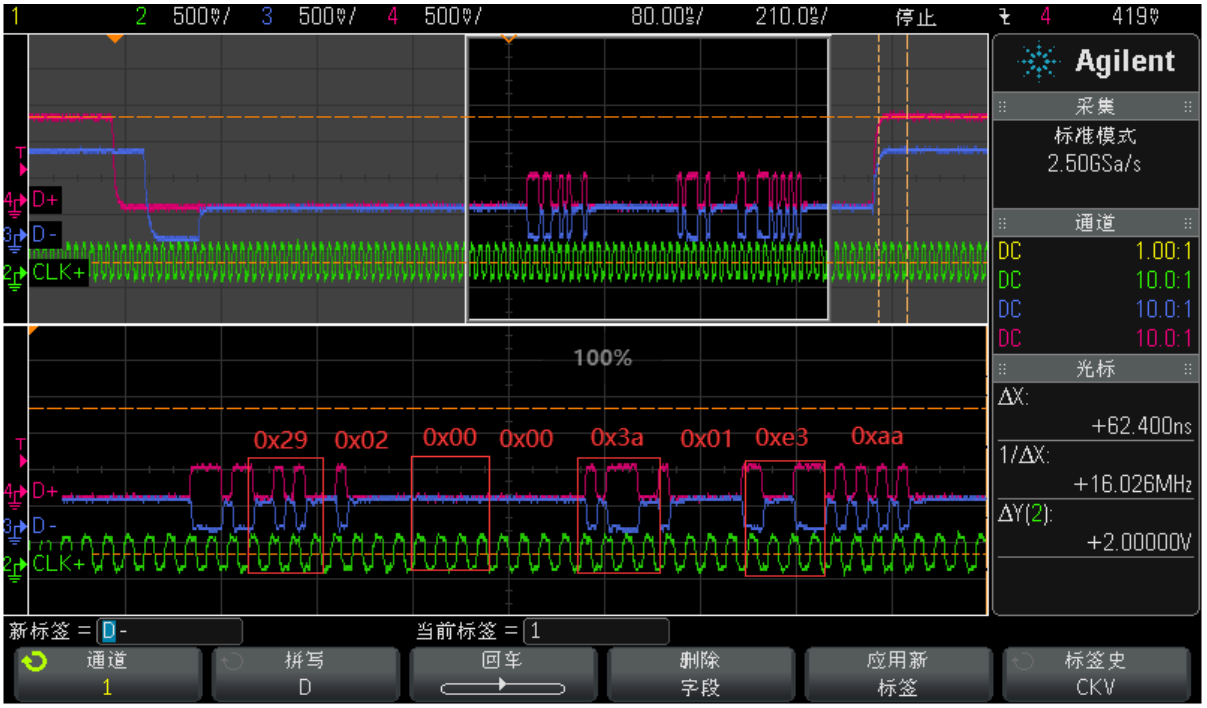
18.2.3.2.1 LPDT-LPa

The following waveform was captured using an oscilloscope while transmitting the above LPa to the display module in LPDT mode:



18.2.3.2.2 HSDT-LPa

The following waveform was captured using an oscilloscope while transmitting the above LPa to the display module in HSDT mode:

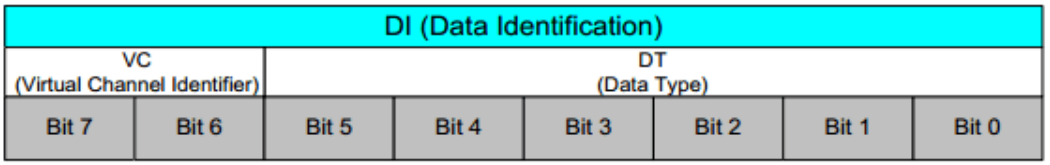


18.2.3.3 DI

Both the SPa and LPa mentioned above contain a Data Identification (DI) field. The DI determines whether a packet is a short or long packet. The DI is part of the packet header and consists of two components:

- Virtual Channel (VC), 2 bits, DI [7...6]
- Data Type (DT), 6 bits, DI [5…0]

The Data Identification (DI) structure is illustrated, see the figure below.



Data Identification (DI) Structure

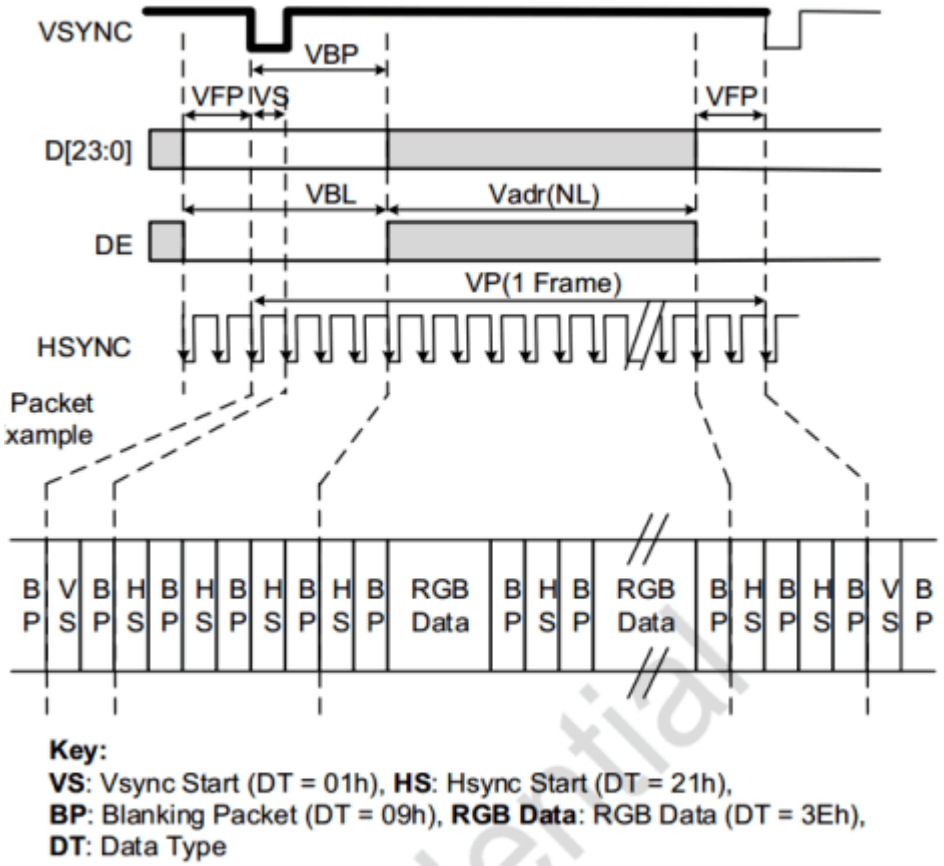
The majority of data types currently defined in the MIPI protocol are as follows:

Table 16 Data Types for Processor-Sourced Packets

Data Type (hex)	Data Type (binary)	Description	Packet Size
0x01	00 0001	Sync Event, V Sync Start	Short
0x11	01 0001	Sync Event, V Sync End	Short
0x21	10 0001	Sync Event, H Sync Start	Short
0x31	11 0001	Sync Event, H Sync End	Short
0x07	00 0111	Compression Mode Command	Short
0x08	00 1000	End of Transmission packet (EoTp)	Short
0x02	00 0010	Color Mode (CM) Off Command	Short
0x12	01 0010	Color Mode (CM) On Command	Short
0x22	10 0010	Shut Down Peripheral Command	Short
0x32	11 0010	Turn On Peripheral Command	Short
0x03	00 0011	Generic Short WRITE, no parameters	Short
0x13	01 0011	Generic Short WRITE, 1 parameter	Short
0x23	10 0011	Generic Short WRITE, 2 parameters	Short
0x04	00 0100	Generic READ, no parameters	Short
0x14	01 0100	Generic READ, 1 parameter	Short
0x24	10 0100	Generic READ, 2 parameters	Short
0x05	00 0101	DCS Short WRITE, no parameters	Short
0x15	01 0101	DCS Short WRITE, 1 parameter	Short
0x06	00 0110	DCS READ, no parameters	Short
0x16	01 0110	Execute Queue	Short
0x37	11 0111	Set Maximum Return Packet Size	Short
0x27	10 0111	Scrambling Mode Command	Short
0x09	00 1001	Null Packet, no data	Long
0x19	01 1001	Blanking Packet, no data	Long
0x29	10 1001	Generic Long Write	Long
0x39	11 1001	DCS Long Write/write_LUT Command Packet	Long
0x0A	00 1010	Picture Parameter Set	Long
0x0B	00 1011	Compressed Pixel Stream	Long
0x0C	00 1100	Loosely Packed Pixel Stream, 20-bit YCbCr, 4:2:2 Format	Long

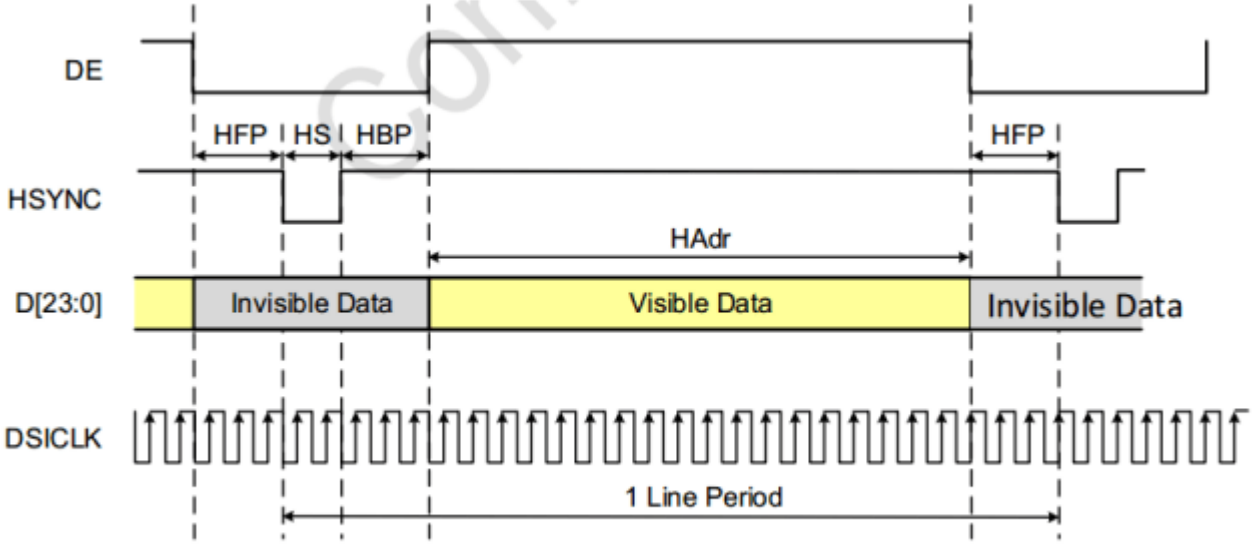
18.2.4 Video Mode Interface Timing

18.2.4.1 Vertical Display Timing (Video mode)



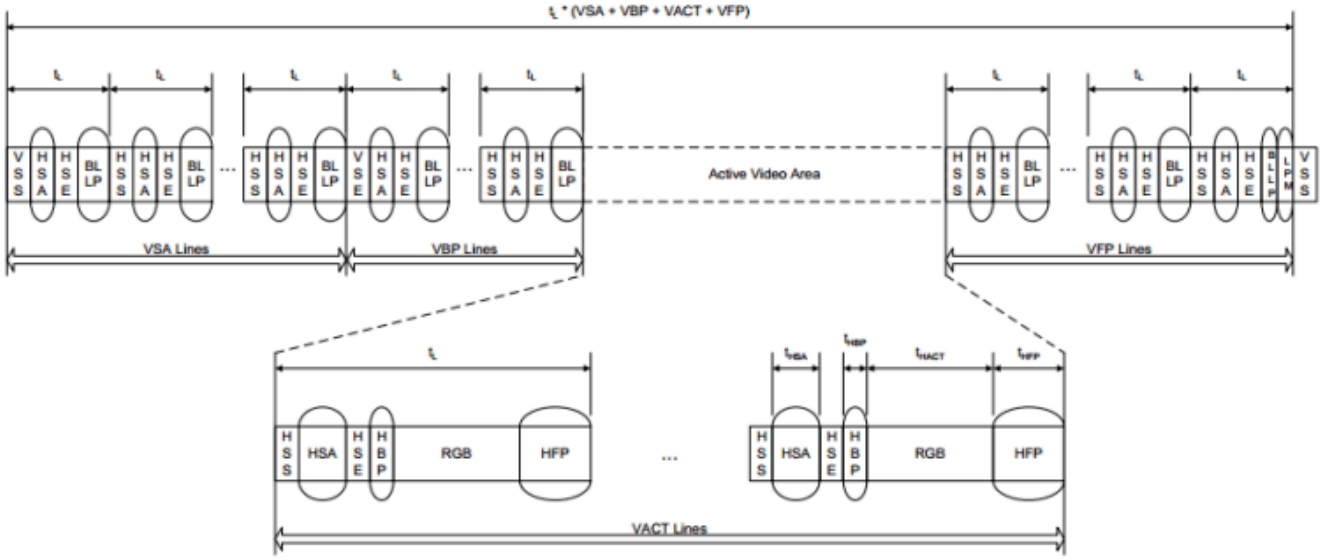
Vertical display timing (video mode)

18.2.4.2 Horizontal Display Timing (Video Mode)



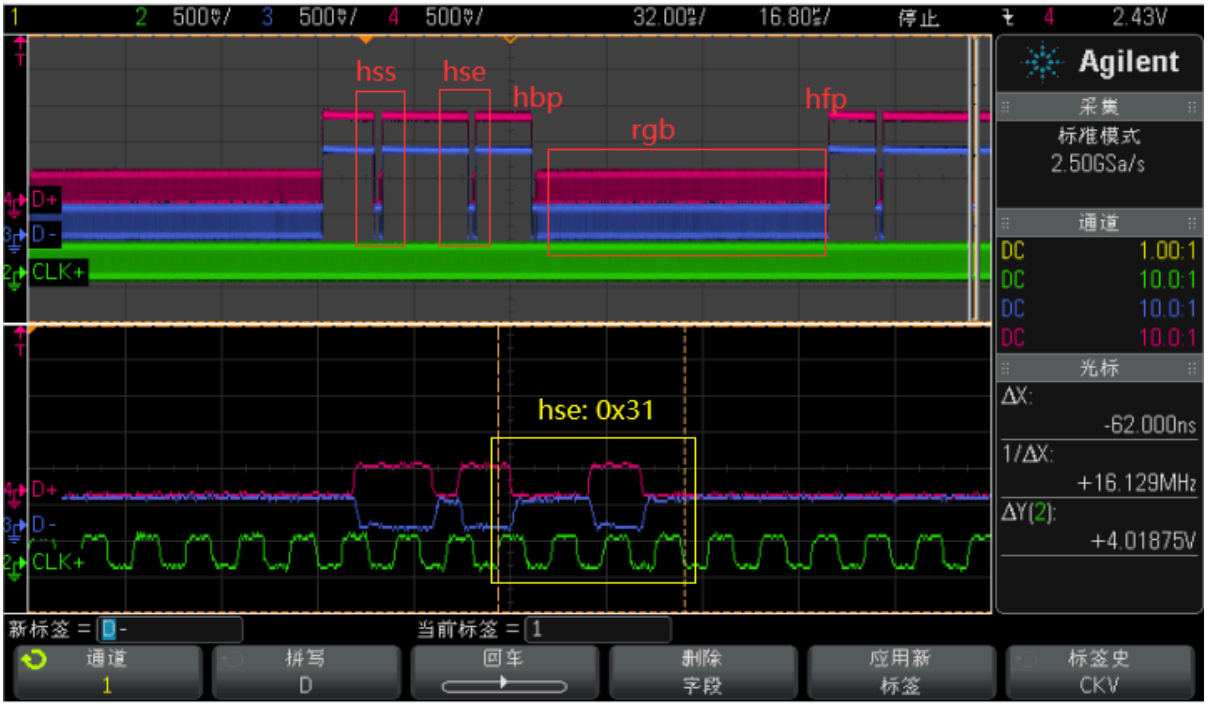
Horizontal display timing (video mode)

18.2.4.3 Non-Burst Mode with Sync Pulses

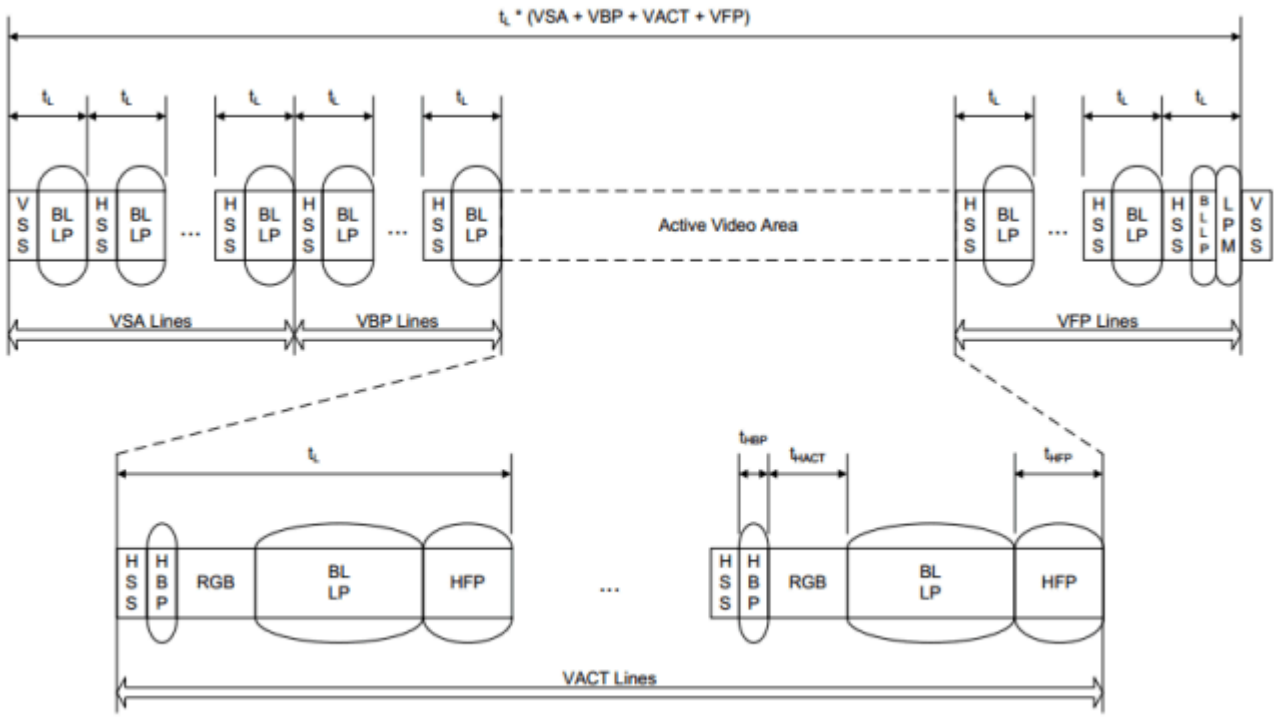


Non-Burst Mode with Sync Pulses

The Non-Burst Sync Pulse waveform captured by an oscilloscope is shown below:

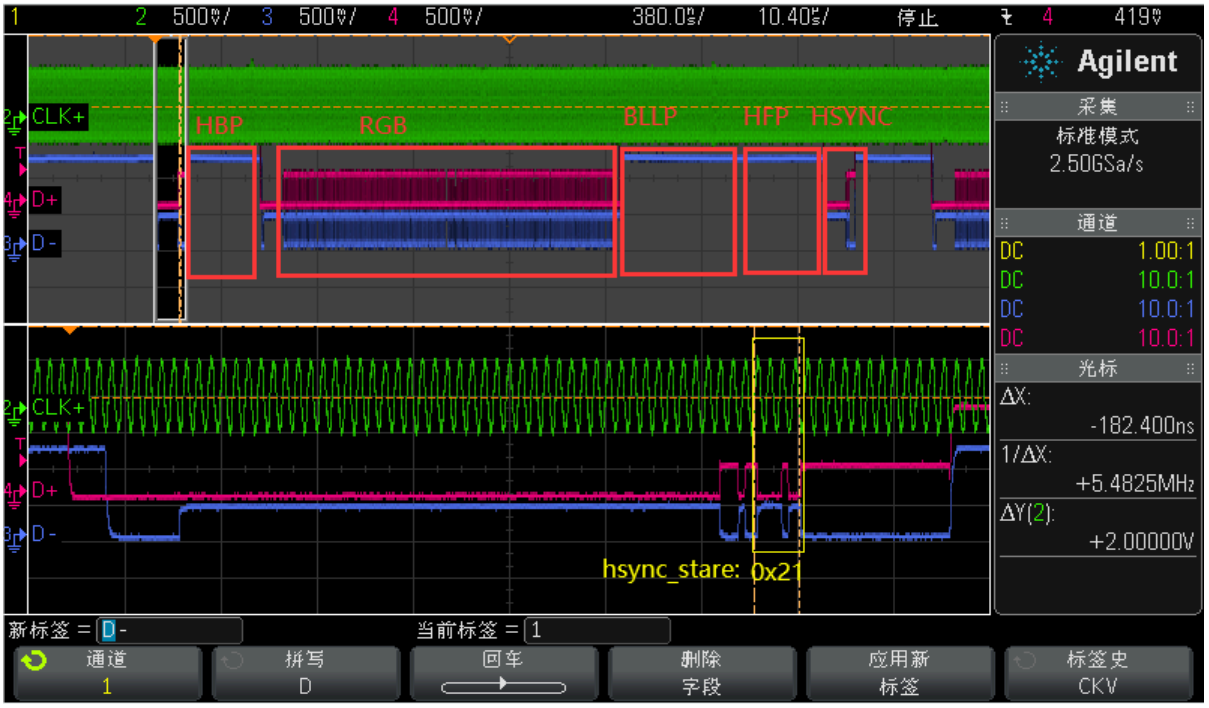


18.2.4.4 Burst Mode



Video burst mode

The Video Burst waveform captured by the oscilloscope is shown below:



19. Platform limitations

19.1 RK3588

19.1.1 VBLANK

Note for RK3588 DSI interface display evaluation: The maximum configurable value for VFP, VSYNC, and VBP cannot exceed 1023. Pay special attention to displays that change frame rates via VFP—especially when achieving low refresh rates by increasing VFP—as this VFP upper limit must be considered.

20. FAQ

20.1 Check VOP Timing and Connector Information

```
console:/ # cat /d/dri/0/summary
Video Port0: DISABLED
Video Port1: DISABLED
Video Port2: DISABLED
Video Port3: ACTIVE
   Connector: DSI-2
      bus_format[100a]: RGB888_1X24
      overlay_mode[0] output_mode[0] color_space[0]
   Display mode: 1080x1920p60
      clk[132000] real_clk[132000] type[48] flag[a]
      H: 1080 1095 1099 1129
      V: 1920 1935 1937 1952
   Cluster3-win0: ACTIVE
      win_id: 6
      format: AB24 little-endian (0x34324241) SDR[0] color_space[0] glb_alpha[0xff]
      rotate: xmirror: 0 ymirror: 0 rotate_90: 0 rotate_270: 0
      csc: y2r[0] r2y[0] csc mode[0]
      zpos: 0
      src: pos[0, 0] rect[1080 x 1920]
      dst: pos[0, 0] rect[1080 x 1920]
      buf[0]: addr: 0x000000000376e000 pitch: 4352 offset: 0
```

20.2 View DSI2-related Clock Tree

When MIPI operates in non-DSC mode, the MIPI interface transfers 4 pixels per clock cycle. The relevant clocks must satisfy the following relationship:

video_timing_pixel_rate / 4 = mipi_pixel_clock × K = dclk_out × K = dclk_core

When MIPI operates in DSC mode, the relevant clocks must satisfy the following relationship:

mipi_pixel_clock = cds_clk / 2

Note! By default, K = 1. When MIPI is configured for dual-channel display mode, K = 2.

```
console:/ # cat /d/clk/clk_summary | grep vop
   clk_vop_pmu                0    0    0   24000000    0    0  50000
       dclk_vop3              1    2    0   33000000    0    0  50000
       dclk_vop1_src          0    1    0  594000000    0    0  50000
         dclk_vop1            0    1    0  594000000    0    0  50000
       dclk_vop0_src          0    1    0  594000000    0    0  50000
         dclk_vop0            0    1    0  594000000    0    0  50000
         aclk_vop_low_root    1    1    0  396000000    0    0  50000
         hclk_vop_root        2    4    0  198000000    0    0  50000
           hclk_vop           1    3    0  198000000    0    0  50000
       aclk_vop_root          1    1    0  500000000    0    0  50000
         aclk_vop_doby        0    0    0  500000000    0    0  50000
         aclk_vop_sub_src     1    1    0  500000000    0    0  50000
           aclk_vop           1    4    0  500000000    0    0  50000
         pclk_vop_root        3    5    0  100000000    0    0  50000
       dclk_vop2_src          1    1    0  148500000    0    0  50000
         dclk_vop2            1    2    0  148500000    0    0  50000
console:/ # cat /d/clk/clk_summary | grep dsi
           pclk_dsihost1      1    2    0  100000000    0    0  50000
           pclk_dsihost0      1    2    0  100000000    0    0  50000
   clk_dsihost1               1    2    0  351000000    0    0  50000
   clk_dsihost0               1    2    0  351000000    0    0  50000
console:/ # cat /d/clk/clk_summary | grep mipi
       mipi1_clk_src          0    0    0   33000000    0    0  50000
         mipi1_pixclk         0    0    0   33000000    0    0  50000
       mipi0_clk_src          0    0    0  148500000    0    0  50000
         mipi0_pixclk         0    0    0  148500000    0    0  50000
   clk_usbdpphy_mipidcpphy_ref  5    5    0   24000000    0    0  50000
   clk_mipi_camaraout_m4      0    0    0   24000000    0    0  50000
   clk_mipi_camaraout_m3      0    0    0   24000000    0    0  50000
   clk_mipi_camaraout_m2      0    0    0   24000000    0    0  50000
   clk_mipi_camaraout_m0      0    0    0   24000000    0    0  50000
       clk_mipi_camaraout_m1  0    0    0   37125000    0    0  50000
           pclk_mipi_dcphy1   1    1    0  100000000    0    0  50000
           pclk_mipi_dcphy0   1    1    0  100000000    0    0  50000
console:/ #
```

20.3 How to Check the Specified DSI Lane Rate

There are two methods for specifying the DSI lane rate: one is automatic calculation by the Driver:

```
dmesg | grep dsi

[   77.369812] dw-mipi-dsi2 fde20000.dsi: [drm:dw_mipi_dsi2_encoder_enable] final DSI-Link bandwidth: 879 x 4 Mbps
```

One method is to manually specify via the following properties:

```
&dsi0 {
    ...
    rockchip,lane-rate = <1000>;
    ...
}
```

20.4 The MIPI DSI2 HOST does not have a built-in color bar function; use the following commands to enable VOP2 to output color bars

Select the corresponding command based on the display routing:

20.4.1 RK3588

```
vp0:
io -4 0xfdd90c08 0x1 && io -4 0xfdd90000 0xffffffff

vp1:
io -4 0xfdd90d08 0x1 && io -4 0xfdd90000 0xffffffff

vp2:
io -4 0xfdd90e08 0x1 && io -4 0xfdd90000 0xffffffff

vp3:
io -4 0xfdd90f08 0x1 && io -4 0xfdd90000 0xffffffff
```

20.4.2 RK3576

```
vp0:
io -4 0x27d00c08 0x1 && io -4 0x27d00000 0xffffffff

vp1:
io -4 0x27d00d08 0x1 && io -4 0x27d00000 0xffffffff

vp2:
io -4 0x27d00e08 0x1 && io -4 0x27d00000 0xffffffff
```

20.5 How to determine whether MIPI DSI2 and the panel are communicating normally when a display anomaly occurs

uboot:

```
--- a/drivers/video/drm/rockchip_panel.c
+++ b/drivers/video/drm/rockchip_panel.c
@@ -260,6 +260,7 @@ static void panel_simple_prepare(struct rockchip_panel *panel)
   struct rockchip_panel_priv *priv = dev_get_priv(panel->dev);
   struct mipi_dsi_device *dsi = dev_get_parent_platdata(panel->dev);
   int ret;
+   u8 mode;

   if (priv->prepared)
      return;
@@ -285,6 +286,8 @@ static void panel_simple_prepare(struct rockchip_panel *panel)
   if (plat->delay.init)
      mdelay(plat->delay.init);

+   mipi_dsi_dcs_get_power_mode(dsi, &mode);
+   printf("===>mode: 0x%x\n", mode);
   if (plat->on_cmds) {
      if (priv->cmd_type == CMD_TYPE_SPI)
         ret = rockchip_panel_send_spi_cmds(panel->state,
@@ -298,6 +301,8 @@ static void panel_simple_prepare(struct rockchip_panel *panel)
         printf("failed to send on cmds: %d\n", ret);
   }

+   mipi_dsi_dcs_get_power_mode(dsi, &mode);
+   printf("===>mode: 0x%x\n", mode);
   priv->prepared = true;
 }
```

kernel

```
--- a/drivers/gpu/drm/panel/panel-simple.c
+++ b/drivers/gpu/drm/panel/panel-simple.c
@@ -506,6 +506,7 @@ static int panel_simple_prepare(struct drm_panel *panel)
   unsigned int delay;
   int err;
   int hpd_asserted;
+   u8 mode;

   if (p->prepared)
      return 0;
@@ -554,6 +555,8 @@ static int panel_simple_prepare(struct drm_panel *panel)
      }
   }

+   mipi_dsi_dcs_get_power_mode(p->dsi, &mode);
+   printk("===>mode: 0x%x\n", mode);
```

```
    if (p->desc->init_seq)
        if (p->dsi)
            panel_simple_xfer_dsi_cmd_seq(p, p->desc->init_seq);
@@ -561,6 +564,9 @@ static int panel_simple_prepare(struct drm_panel *panel)
    if (p->desc->delay.init)
        msleep(p->desc->delay.init);

+   mipi_dsi_dcs_get_power_mode(p->dsi, &mode);
+   printk("===>mode: 0x%x\n", mode);
+
    p->prepared = true;

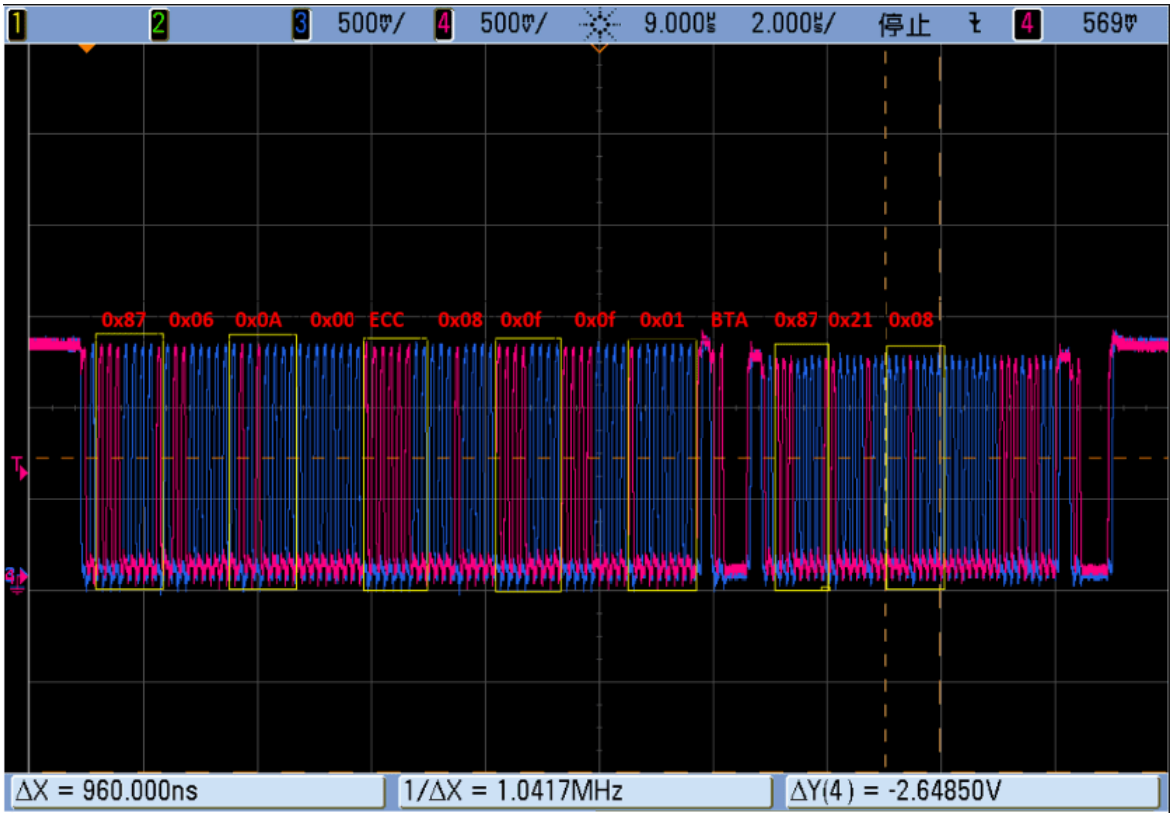
    return 0;
```

If communication is normal, the following log will be printed; otherwise, check the panel timing to ensure the panel is ready:

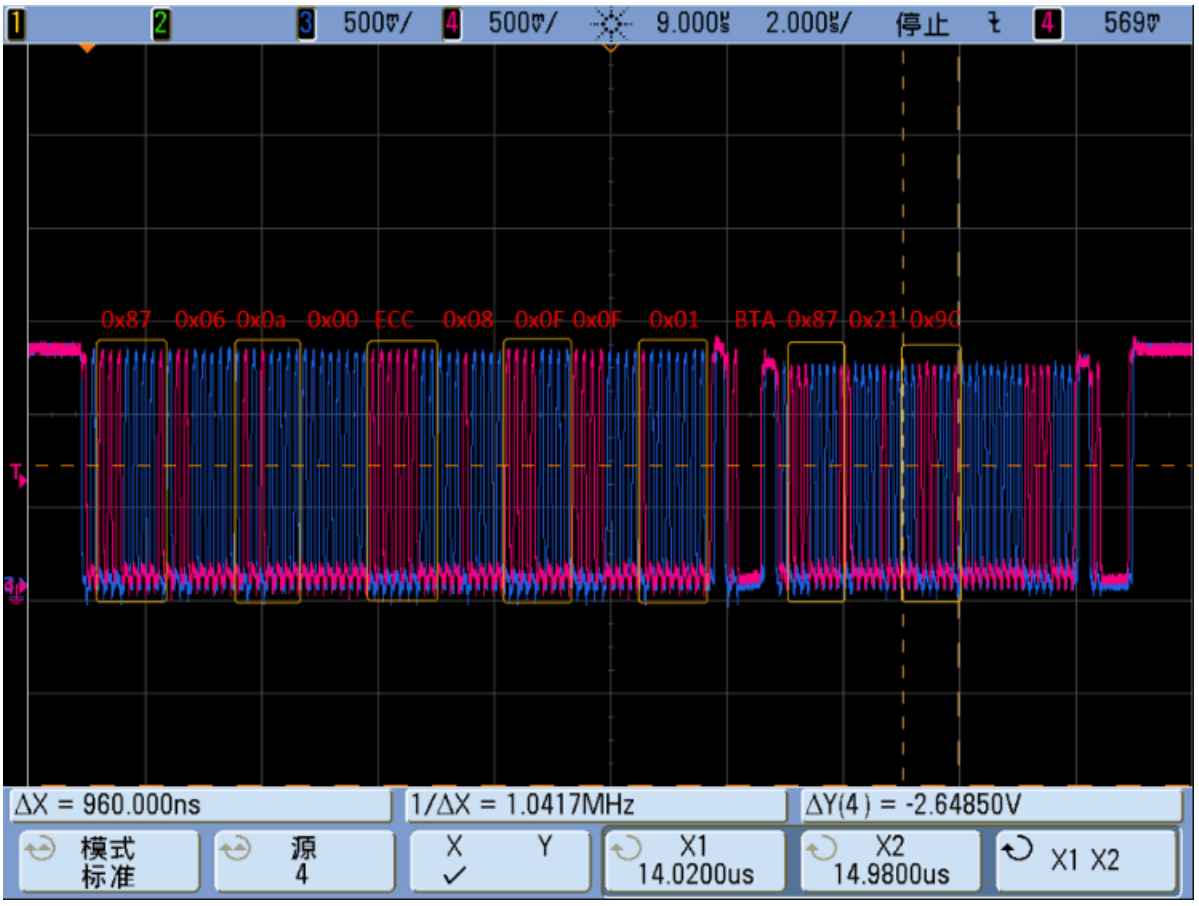
```
===> mode: 0x8
===> mode: 0x9c
```

20.5.1 Two Readback Waveforms for get_power_mode

20.5.1.1 Normal Panel Return of 0x08 (0x11 and 0x29 Not Issued)



20.5.1.2 Normal Panel Return of 0x9c (0x11 and 0x29 Issued)



20.6 DRM Driver Failed to Bind

Use the following command to check whether a specific component failed to bind. For example, if the DP PHY is not enabled, DP binding will fail:

```
/sys/kernel/debug/device_component/display-subsystem
master name                          status
----------------------------------------------------------------
display-subsystem                       bound

device name                          status
----------------------------------------------------------------
fdd90000.vop                          bound
fde20000.dsi                         bound
fde30000.dsi                         bound
fde50000.dp                          bound
```

20.7 Backlight driver probe failure

```
console:/ # dmesg | grep backlight
[    3.164274] pwm-backlight: probe of backlight failed with error -16
```

20.8 How to Configure TE for Command Mode Display Modules

To prevent image tearing, the frame refresh rate of the display controller must match the frame readout rate from the display module's GRAM. The RK3588 and RK3576 only support external TE signal feedback from the display module.

20.8.1 Hardware TE

MIPI DSI provides dedicated pins that can be multiplexed as TE.

```
&dsi0 {
    ...
    /* Connect the display module TE signal to MIPI_TE0 */
    pinctrl-names = "default";
    pinctrl-0 = <&mipi_te0>;
    ...
};

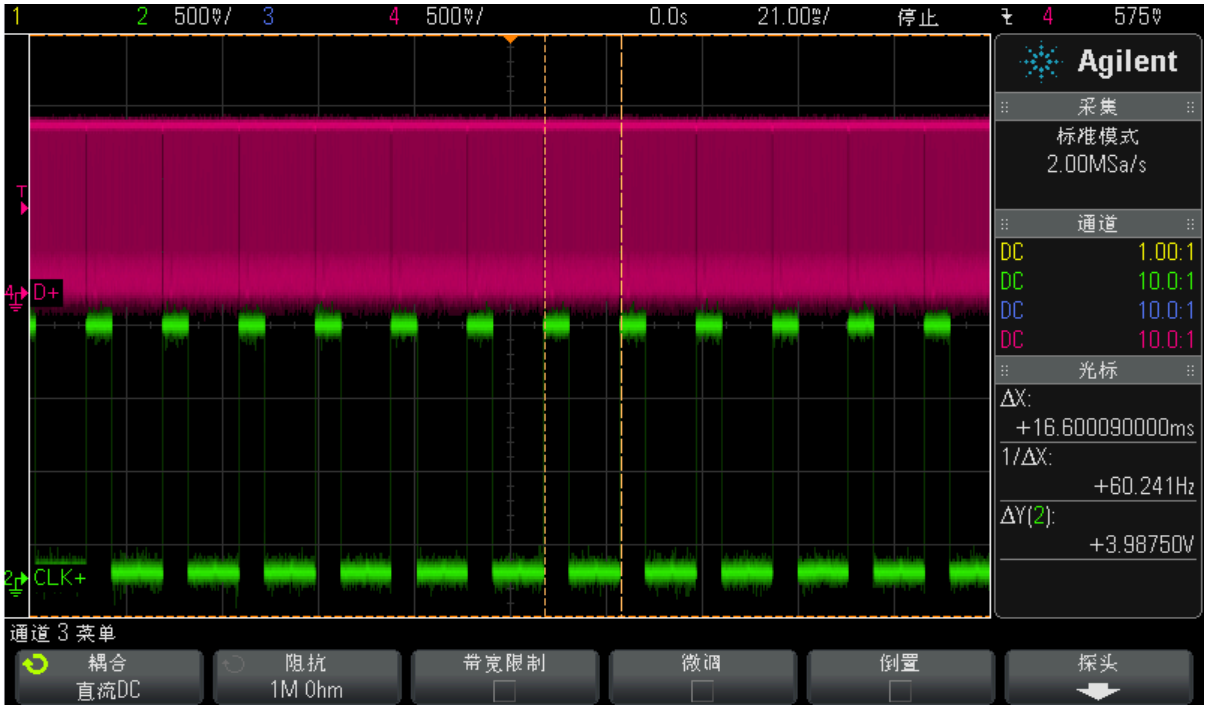
&dsi1 {
    ...
    /* Connect the display module TE signal to MIPI_TE1; only RK3588 supports DSI1 */
    pinctrl-names = "default";
    pinctrl-0 = <&mipi_te1>;
    ...
};
```

20.8.2 Software TE

In some cases, the hardware TE signal may be repurposed for other functions. Therefore, an available GPIO can be configured as an input to notify the display system of the TE Interrupt.

```
&dsi0 {
    status = "okay";
    te-gpios = <&gpio0 RK_PC4 GPIO_ACTIVE_LOW>;
    pinctrl-names = "default";
    pinctrl-0 = <&lcd_te_gpio>;
};
```

The following figure captures the waveform where the field frequency of the MIPI DSI transmitted data signal matches the frequency of the display module TE signal:



20.9 Dual-Channel MIPI Primary/Secondary Sequence Switching

If the hardware design has reversed the connections of the two MIPI ports in a dual-channel configuration, software correction can be applied using the following configuration:

```
&dsi0 {
    ...
    rockchip,data-swap;
    rockchip,dual-channel = <&dsi1>;
    ...
};

&dsi1 {
    status = "okay";
};
```

20.10 Debug Nodes

During MIPI DSI signal testing, signal adjustments are required. The relevant debug node paths are as follows:

```
dcphy0:
cd /sys/devices/platform/feda0000.phy/
dcphy1:
cd /sys/devices/platform/fedb0000.phy/
```

20.10.1 Patch

```
diff --git a/drivers/phy/rockchip/phy-rockchip-samsung-dcphy.c b/drivers/phy/rockchip/phy-rockchip-samsung-dcphy.c
index cd6b615228d9.26bb1d030d08 100644
--- a/drivers/phy/rockchip/phy-rockchip-samsung-dcphy.c
+++ b/drivers/phy/rockchip/phy-rockchip-samsung-dcphy.c
@@ -219,6 +219,25 @@
 #define RX_S0D3_DESKEW_CON4       (0xF50)
 #define RX_S0D3_ADI_STAT0     (0XFEC)
 #define MIPI_DCPHY_MAX_REGISGER       RX_S0D3_ADI_STAT0
+#define REG_400M_MASK     GENMASK(6, 4)
+#define REG_400M(x)       UPDATE(x, 6, 4)
+#define REG_645M_MASK     GENMASK(2, 0)
+#define REG_645M(x)       UPDATE(x, 2, 0)
+
+#define BIAS_CON4     0x0010
+#define I_MUX_SEL_MASK    GENMASK(6, 5)
+#define I_MUX_SEL(x)      UPDATE(x, 6, 5)
+#define CAP_PEAKING_MASK  GENMASK(14, 8)
+#define RES_UP_MASK       GENMASK(7, 4)
+#define RES_UP(x)     UPDATE(x, 7, 4)
+#define RES_DN_MASK       GENMASK(3, 0)
+#define RES_DN(x)     UPDATE(x, 3, 0)
+#define T_HS_ZERO_MASK    GENMASK(15, 8)
+#define T_HS_PREPARE_MASK GENMASK(7, 0)
+#define T_HS_EXIT_MASK    GENMASK(15, 8)
+#define T_HS_TRAIL_MASK       GENMASK(7, 0)
+#define T_TA_GET_MASK     GENMASK(7, 4)
+#define T_TA_GO_MASK      GENMASK(3, 0)

 struct samsung_mipi_dphy_timing {
   unsigned int max_lane_mbps;
@@ -1780,10 +1799,10 @@ static struct v4l2_subdev *get_remote_sensor(struct v4l2_subdev *sd);
 static int samsung_mipi_dcphy_power_on(struct phy *phy)
 {
   struct samsung_mipi_dcphy *samsung = phy_get_drvdata(phy);
-  enum phy_mode mode = phy_get_mode(phy);
   int on = 0;
   struct v4l2_subdev *sensor_sd = NULL;

+  samsung->mode = phy_get_mode(phy);
   pm_runtime_get_sync(samsung->dev);
   reset_control_assert(samsung->apb_rst);
   udelay(1);
@@ -1801,7 +1820,7 @@ static int samsung_mipi_dcphy_power_on(struct phy *phy)
                RKMODULE_SET_QUICK_STREAM, &on);
   }

-  switch (mode) {
+  switch (samsung->mode) {
   case PHY_MODE_MIPI_DPHY:
     samsung_mipi_dphy_power_on(samsung);
```

```
        break;
@@ -2381,6 +2400,467 @@ static const struct regmap_config samsung_mipi_dcphy_regmap_config = {
    .max_register = MIPI_DCPHY_MAX_REGISGER,
 };

+static ssize_t
+reg_645m_show(struct device *device, struct device_attribute *attr, char *buf)
+{
+   struct samsung_mipi_dcphy *samsung = dev_get_drvdata(device);
+   unsigned int val;
+   unsigned int level;
+
+   regmap_read(samsung->regmap, BIAS_CON2, &val);
+   level = (val & REG_645M_MASK);
+
+   return sprintf(buf, "%d\n", level);
+
+}
+
+static ssize_t
+reg_645m_store(struct device *device, struct device_attribute *attr,
+           const char *buf, size_t count)
+{
+   struct samsung_mipi_dcphy *samsung = dev_get_drvdata(device);
+   unsigned long val;
+
+   if (kstrtoul(buf, 10, &val))
+       return -EINVAL;
+
+
+   if (val > 7)
+       val = 7;
+
+   regmap_update_bits(samsung->regmap, BIAS_CON2, REG_645M_MASK, REG_645M(val));
+
+   return count;
+}
+static DEVICE_ATTR_RW(reg_645m);
+
+static ssize_t
+reg_400m_show(struct device *device, struct device_attribute *attr, char *buf)
+{
+   struct samsung_mipi_dcphy *samsung = dev_get_drvdata(device);
+   unsigned int val;
+   unsigned int level;
+
+   regmap_read(samsung->regmap, BIAS_CON2, &val);
+   level = (val & REG_400M_MASK) >> 4;
+
+   return sprintf(buf, "%d\n", level);
+
+}
+
+
```

```
+static ssize_t
+reg_400m_store(struct device *device, struct device_attribute *attr,
+           const char *buf, size_t count)
+{
+   struct samsung_mipi_dcphy *samsung = dev_get_drvdata(device);
+   unsigned long val;
+
+   if (kstrtoul(buf, 10, &val))
+      return -EINVAL;
+
+
+   if (val > 7)
+      val = 7;
+
+   regmap_update_bits(samsung->regmap, BIAS_CON2, REG_400M_MASK, REG_400M(val));
+
+   return count;
+}
+static DEVICE_ATTR_RW(reg_400m);
+
+static ssize_t
+cap_peaking_show(struct device *device, struct device_attribute *attr, char *buf)
+{
+   struct samsung_mipi_dcphy *samsung = dev_get_drvdata(device);
+   unsigned int val;
+   unsigned int level;
+
+   regmap_read(samsung->regmap, COMBO_MD0_ANA_CON0, &val);
+   level = (val & CAP_PEAKING_MASK) >> 12;
+
+   return sprintf(buf, "%d\n", level);
+
+}
+
+static ssize_t
+cap_peaking_store(struct device *device, struct device_attribute *attr,
+           const char *buf, size_t count)
+{
+   struct samsung_mipi_dcphy *samsung = dev_get_drvdata(device);
+   unsigned long val;
+
+   if (kstrtoul(buf, 10, &val))
+      return -EINVAL;
+
+
+   if (val > 7)
+      val = 7;
+
+   regmap_update_bits(samsung->regmap, DPHY_MC_ANA_CON0, CAP_PEAKING_MASK,
+            EDGE_CON_DIR(samsung->mode == PHY_MODE_MIPI_DPHY ? 0 : 1) |
+            EDGE_CON(val) | EDGE_CON_EN);
+   regmap_update_bits(samsung->regmap, COMBO_MD0_ANA_CON0, CAP_PEAKING_MASK,
+            EDGE_CON_DIR(samsung->mode == PHY_MODE_MIPI_DPHY ? 0 : 1) |
```

```
+            EDGE_CON(val) | EDGE_CON_EN);
+   regmap_update_bits(samsung->regmap, COMBO_MD1_ANA_CON0, CAP_PEAKING_MASK,
+            EDGE_CON_DIR(samsung->mode == PHY_MODE_MIPI_DPHY ? 0 : 1) |
+            EDGE_CON(val) | EDGE_CON_EN);
+   regmap_update_bits(samsung->regmap, COMBO_MD2_ANA_CON0, CAP_PEAKING_MASK,
+            EDGE_CON_DIR(samsung->mode == PHY_MODE_MIPI_DPHY ? 0 : 1) |
+            EDGE_CON(val) | EDGE_CON_EN);
+   regmap_update_bits(samsung->regmap, DPHY_MD3_ANA_CON0, CAP_PEAKING_MASK,
+            EDGE_CON_DIR(samsung->mode == PHY_MODE_MIPI_DPHY ? 0 : 1) |
+            EDGE_CON(val) | EDGE_CON_EN);
+
+   return count;
+}
+static DEVICE_ATTR_RW(cap_peaking);
+
+static ssize_t
+res_up_show(struct device *device, struct device_attribute *attr, char *buf)
+{
+   struct samsung_mipi_dcphy *samsung = dev_get_drvdata(device);
+   unsigned int val;
+   unsigned int level;
+
+   regmap_read(samsung->regmap, COMBO_MD0_ANA_CON0, &val);
+   level = (val & RES_UP_MASK) >> 4;
+
+   return sprintf(buf, "%d\n", level);
+
+}
+
+static ssize_t
+res_up_store(struct device *device, struct device_attribute *attr,
+          const char *buf, size_t count)
+{
+   struct samsung_mipi_dcphy *samsung = dev_get_drvdata(device);
+   unsigned long val;
+
+   if (kstrtoul(buf, 10, &val))
+      return -EINVAL;
+
+
+   if (val > 15)
+      val = 15;
+
+   regmap_update_bits(samsung->regmap, DPHY_MC_ANA_CON0, RES_UP_MASK, RES_UP(val));
+   regmap_update_bits(samsung->regmap, COMBO_MD0_ANA_CON0, RES_UP_MASK, RES_UP(val));
+   regmap_update_bits(samsung->regmap, COMBO_MD1_ANA_CON0, RES_UP_MASK, RES_UP(val));
+   regmap_update_bits(samsung->regmap, COMBO_MD2_ANA_CON0, RES_UP_MASK, RES_UP(val));
+   regmap_update_bits(samsung->regmap, DPHY_MD3_ANA_CON0, RES_UP_MASK, RES_UP(val));
+
+   return count;
+}
+static DEVICE_ATTR_RW(res_up);
+
```

```
+static ssize_t
+res_dn_show(struct device *device, struct device_attribute *attr, char *buf)
+{
+	struct samsung_mipi_dcphy *samsung = dev_get_drvdata(device);
+	unsigned int val;
+	unsigned int level;
+
+	regmap_read(samsung->regmap, COMBO_MD0_ANA_CON0, &val);
+	level = (val & RES_DN_MASK);
+
+	return sprintf(buf, "%d\n", level);
+
+}
+
+static ssize_t
+res_dn_store(struct device *device, struct device_attribute *attr,
+	     const char *buf, size_t count)
+{
+	struct samsung_mipi_dcphy *samsung = dev_get_drvdata(device);
+	unsigned long val;
+
+	if (kstrtoul(buf, 10, &val))
+		return -EINVAL;
+
+
+	if (val > 15)
+		val = 15;
+
+	regmap_update_bits(samsung->regmap, DPHY_MC_ANA_CON0, RES_DN_MASK, RES_DN(val));
+	regmap_update_bits(samsung->regmap, COMBO_MD0_ANA_CON0, RES_DN_MASK, RES_DN(val));
+	regmap_update_bits(samsung->regmap, COMBO_MD1_ANA_CON0, RES_DN_MASK, RES_DN(val));
+	regmap_update_bits(samsung->regmap, COMBO_MD2_ANA_CON0, RES_DN_MASK, RES_DN(val));
+	regmap_update_bits(samsung->regmap, DPHY_MD3_ANA_CON0, RES_DN_MASK, RES_DN(val));
+
+	return count;
+}
+static DEVICE_ATTR_RW(res_dn);
+
+static ssize_t
+output_voltage_show(struct device *device, struct device_attribute *attr, char *buf)
+{
+	struct samsung_mipi_dcphy *samsung = dev_get_drvdata(device);
+	unsigned int val;
+	unsigned int level;
+
+	regmap_read(samsung->regmap, BIAS_CON4, &val);
+	level = (val & I_MUX_SEL_MASK) >> 5;
+
+	return sprintf(buf, "%d\n", level);
+
+}
+
+static ssize_t
```

```
+output_voltage_store(struct device *device, struct device_attribute *attr,
+		const char *buf, size_t count)
+{
+	struct samsung_mipi_dcphy *samsung = dev_get_drvdata(device);
+	unsigned long val;
+
+	if (kstrtoul(buf, 10, &val))
+		return -EINVAL;
+
+
+	if (val > 3)
+		val = 3;
+
+	regmap_update_bits(samsung->regmap, BIAS_CON4,
+			I_MUX_SEL_MASK, I_MUX_SEL(val));
+
+	return count;
+}
+static DEVICE_ATTR_RW(output_voltage);
+
+static ssize_t
+hs_exit_show(struct device *device, struct device_attribute *attr, char *buf)
+{
+	struct samsung_mipi_dcphy *samsung = dev_get_drvdata(device);
+	unsigned int val;
+	unsigned int hs_exit;
+
+	regmap_read(samsung->regmap, COMBO_MD0_TIME_CON2, &val);
+	hs_exit = (val & GENMASK(15, 8)) >> 8;
+
+	return sprintf(buf, "%d\n", hs_exit);
+
+}
+
+static ssize_t hs_exit_store(struct device *device, struct device_attribute *attr,
+		const char *buf, size_t count)
+{
+	struct samsung_mipi_dcphy *samsung = dev_get_drvdata(device);
+	unsigned long val;
+	unsigned long hs_exit;
+
+	if (kstrtoul(buf, 10, &val))
+		return -EINVAL;
+
+	hs_exit = T_HS_EXIT(val);
+	regmap_update_bits(samsung->regmap, COMBO_MD0_TIME_CON2, T_HS_EXIT_MASK, hs_exit);
+	regmap_update_bits(samsung->regmap, COMBO_MD1_TIME_CON2, T_HS_EXIT_MASK, hs_exit);
+	regmap_update_bits(samsung->regmap, COMBO_MD2_TIME_CON2, T_HS_EXIT_MASK, hs_exit);
+
+	if (!samsung->c_option) {
+		regmap_update_bits(samsung->regmap, DPHY_MC_TIME_CON2, T_HS_EXIT_MASK, hs_exit);
+		regmap_update_bits(samsung->regmap, DPHY_MD3_TIME_CON2, T_HS_EXIT_MASK, hs_exit);
+	}
```

```
+
+	return count;
+}
+static DEVICE_ATTR_RW(hs_exit);
+
+static ssize_t
+hs_trail_or_post_3_show(struct device *device, struct device_attribute *attr, char *buf)
+{
+	struct samsung_mipi_dcphy *samsung = dev_get_drvdata(device);
+	unsigned int val;
+	unsigned int hs_trail;
+
+	regmap_read(samsung->regmap, COMBO_MD0_TIME_CON2, &val);
+	hs_trail = val & GENMASK(7, 0);
+
+	return sprintf(buf, "%d\n", hs_trail);
+
+}
+
+static ssize_t hs_trail_or_post_3_store(struct device *device, struct device_attribute *attr,
+			const char *buf, size_t count)
+{
+	struct samsung_mipi_dcphy *samsung = dev_get_drvdata(device);
+	unsigned long val;
+	unsigned long hs_trail;
+
+	if (kstrtoul(buf, 10, &val))
+		return -EINVAL;
+
+	hs_trail = T_HS_TRAIL(val);
+	regmap_update_bits(samsung->regmap, COMBO_MD0_TIME_CON2, T_HS_TRAIL_MASK, hs_trail);
+	regmap_update_bits(samsung->regmap, COMBO_MD1_TIME_CON2, T_HS_TRAIL_MASK, hs_trail);
+	regmap_update_bits(samsung->regmap, COMBO_MD2_TIME_CON2, T_HS_TRAIL_MASK, hs_trail);
+
+	if (!samsung->c_option) {
+		regmap_update_bits(samsung->regmap, DPHY_MC_TIME_CON2, T_HS_TRAIL_MASK, hs_trail);
+		regmap_update_bits(samsung->regmap, DPHY_MD3_TIME_CON2, T_HS_TRAIL_MASK, hs_trail);
+	}
+
+	return count;
+}
+static DEVICE_ATTR_RW(hs_trail_or_post_3);
+
+static ssize_t
+hs_zero_or_prebegin_3_show(struct device *device, struct device_attribute *attr, char *buf)
+{
+	struct samsung_mipi_dcphy *samsung = dev_get_drvdata(device);
+	unsigned int val;
+	unsigned int hs_zero;
+
+	regmap_read(samsung->regmap, COMBO_MD0_TIME_CON1, &val);
+	hs_zero = (val & GENMASK(15, 8)) >> 8;
+
```

```
+	return sprintf(buf, "%d\n", hs_zero);
+
+}
+
+static ssize_t hs_zero_or_prebegin_3_store(struct device *device, struct device_attribute *attr,
+		const char *buf, size_t count)
+{
+	struct samsung_mipi_dcphy *samsung = dev_get_drvdata(device);
+	unsigned long val;
+	unsigned long hs_zero;
+
+	if (kstrtoul(buf, 10, &val))
+		return -EINVAL;
+
+	hs_zero = T_HS_ZERO(val);
+	regmap_update_bits(samsung->regmap, COMBO_MD0_TIME_CON1,
+			T_HS_ZERO_MASK, hs_zero);
+	regmap_update_bits(samsung->regmap, COMBO_MD1_TIME_CON1,
+			T_HS_ZERO_MASK, hs_zero);
+	regmap_update_bits(samsung->regmap, COMBO_MD2_TIME_CON1,
+			T_HS_ZERO_MASK, hs_zero);
+
+	if (!samsung->c_option) {
+		regmap_update_bits(samsung->regmap, DPHY_MC_TIME_CON1,
+				T_HS_ZERO_MASK, hs_zero);
+		regmap_update_bits(samsung->regmap, DPHY_MD3_TIME_CON1,
+				T_HS_ZERO_MASK, hs_zero);
+	}
+
+	return count;
+}
+static DEVICE_ATTR_RW(hs_zero_or_prebegin_3);
+
+static ssize_t
+hs_prepare_or_prepare_3_show(struct device *device, struct device_attribute *attr, char *buf)
+{
+	struct samsung_mipi_dcphy *samsung = dev_get_drvdata(device);
+	unsigned int val;
+	unsigned int hs_prepare;
+
+	regmap_read(samsung->regmap, COMBO_MD0_TIME_CON1, &val);
+	hs_prepare = val & GENMASK(7, 0);
+
+	return sprintf(buf, "%d\n", hs_prepare);
+}
+
+static ssize_t hs_prepare_or_prepare_3_store(struct device *device, struct device_attribute *attr,
+		const char *buf, size_t count)
+{
+	struct samsung_mipi_dcphy *samsung = dev_get_drvdata(device);
+	unsigned long val;
+	unsigned long hs_prepare;
+
```

```
+	if (kstrtoul(buf, 10, &val))
+		return -EINVAL;
+
+	hs_prepare = T_HS_PREPARE(val);
+	regmap_update_bits(samsung->regmap, COMBO_MD0_TIME_CON1,
+			T_HS_PREPARE_MASK, hs_prepare);
+	regmap_update_bits(samsung->regmap, COMBO_MD1_TIME_CON1,
+			T_HS_PREPARE_MASK, hs_prepare);
+	regmap_update_bits(samsung->regmap, COMBO_MD2_TIME_CON1,
+			T_HS_PREPARE_MASK, hs_prepare);
+
+	if (!samsung->c_option) {
+		regmap_update_bits(samsung->regmap, DPHY_MC_TIME_CON1,
+				T_HS_PREPARE_MASK, hs_prepare);
+		regmap_update_bits(samsung->regmap, DPHY_MD3_TIME_CON1,
+				T_HS_PREPARE_MASK, hs_prepare);
+	}
+
+	return count;
+}
+static DEVICE_ATTR_RW(hs_prepare_or_prepare_3);
+
+static ssize_t
+lpx_show(struct device *device, struct device_attribute *attr, char *buf)
+{
+	struct samsung_mipi_dcphy *samsung = dev_get_drvdata(device);
+	unsigned int val;
+	unsigned int lpx;
+
+	regmap_read(samsung->regmap, COMBO_MD0_TIME_CON0, &val);
+	lpx = (val & GENMASK(11, 4)) >> 4;
+
+	return sprintf(buf, "%d\n", lpx);
+
+}
+
+static ssize_t lpx_store(struct device *device, struct device_attribute *attr,
+			 const char *buf, size_t count)
+{
+	struct samsung_mipi_dcphy *samsung = dev_get_drvdata(device);
+	unsigned long val;
+	unsigned long lpx = 0;
+
+	if (kstrtoul(buf, 10, &val))
+		return -EINVAL;
+
+	lpx |= T_LPX(val);
+	/*  T_LP_EXIT_SKEW/T_LP_ENTRY_SKEW unconfig */
+	regmap_write(samsung->regmap, COMBO_MD0_TIME_CON0, lpx);
+	regmap_write(samsung->regmap, COMBO_MD1_TIME_CON0, lpx);
+	regmap_write(samsung->regmap, COMBO_MD2_TIME_CON0, lpx);
+
+	if (!samsung->c_option) {
```

```
+		regmap_write(samsung->regmap, DPHY_MC_TIME_CON0, lpx);
+		regmap_write(samsung->regmap, DPHY_MD3_TIME_CON0, lpx);
+	}
+
+	return count;
+}
+static DEVICE_ATTR_RW(lpx);
+
+static struct attribute *samsung_mipi_dcphy_cts_attrs[] = {
+	&dev_attr_lpx.attr,
+	&dev_attr_hs_prepare_or_prepare_3.attr,
+	&dev_attr_hs_zero_or_prebegin_3.attr,
+	&dev_attr_hs_trail_or_post_3.attr,
+	&dev_attr_hs_exit.attr,
+	&dev_attr_output_voltage.attr,
+	&dev_attr_res_up.attr,
+	&dev_attr_res_dn.attr,
+	&dev_attr_cap_peaking.attr,
+	&dev_attr_reg_400m.attr,
+	&dev_attr_reg_645m.attr,
+	NULL
+};
+
+static const struct attribute_group samsung_mipi_dcphy_cts_attr_group = {
+	.attrs = samsung_mipi_dcphy_cts_attrs,
+};
+
+static int samsung_mipi_dcphy_cts_sysfs_add(struct samsung_mipi_dcphy *samsung)
+{
+	struct device *dev = samsung->dev;
+	int ret;
+
+	ret = sysfs_create_group(&dev->kobj, &samsung_mipi_dcphy_cts_attr_group);
+	if (ret) {
+		dev_err(dev, "failed to register sysfs. err: %d\n", ret);
+		return ret;
+	}
+
+	return 0;
+}
+
+
 static int samsung_mipi_dcphy_probe(struct platform_device *pdev)
 {
	struct device *dev = &pdev->dev;
@@ -2474,6 +2954,7 @@ static int samsung_mipi_dcphy_probe(struct platform_device *pdev)
	samsung->stream_off = samsung_dcphy_rx_stream_off;
	mutex_init(&samsung->mutex);
	pm_runtime_enable(dev);
+	samsung_mipi_dcphy_cts_sysfs_add(samsung);

	return 0;
 }
```

```
diff --git a/drivers/phy/rockchip/phy-rockchip-samsung-dcphy.h b/drivers/phy/rockchip/phy-rockchip-samsung-dcphy.h
index b5e827d7cfc1.a53356c0eed6 100644
--- a/drivers/phy/rockchip/phy-rockchip-samsung-dcphy.h
+++ b/drivers/phy/rockchip/phy-rockchip-samsung-dcphy.h
@@ -48,6 +48,7 @@ struct samsung_mipi_dcphy {
   struct clk *pclk;
   struct regmap *regmap;
   struct regmap *grf_regmap;
+  int mode;
   struct reset_control *m_phy_rst;
   struct reset_control *s_phy_rst;
   struct reset_control *apb_rst;
```

20.10.2 Drive Strength

20.10.2.1 Output Voltage (400mV/200mV/530mV) Selection Pin

```
echo level > output_voltage
```

The Driver uses the following default configurations:

1. D-PHY: 2'b00
2. C-PHY: 2'b10

Refer to the following levels:

1. 2b'00 : 400mV
2. 2b'01 : 200mV
3. 2b'10 : 530mV
4. 2b'11 : 530mV

20.10.2.2 LP Common-Mode Voltage

```
echo level > reg_645m
```

The level references are as follows:

Reference voltage - 645mV for LP Function Control Pin

1. 3'b000: 605mV
2. 3'b001: 625mV
3. 3'b010: 635mV
4. 3'b011: 645mV
5. 3'b100: 655mV
6. 3'b101: 665mV

7. 3'b110: 685mV
8. 3'b111: 725mV

20.10.2.3 HS Common-Mode Voltage

```
echo level > reg_400m
```

The levels are as follows:

1. 3'b000: 380mV / 230mV
2. 3'b001: 390mV / 220mV
3. 3'b010: 400mV / 210mV
4. 3'b011: 410mV / 200mV
5. 3'b100: 420mV / 190mV
6. 3'b101: 430mV / 180mV
7. 3'b110: 440mV / 170mV
8. 3'b111: 450mV / 160mV

20.10.2.4 HS Differential Voltage

The high-speed differential voltage is jointly determined by configurable pull-up and pull-down resistors with the following resistance values:

$$Vp - Vn(differentialvoltage) = VREGx100/(Rup + 100 + Rdn)$$

Where VREG corresponds to the common-mode voltage (configured in the REG_400M Register field).

20.10.2.4.1 High-Speed Driver Up Resistor Control

```
echo level > res_up
```

The level references are as follows:

1. 4'b0000: 43 ohm
2. 4'b0001: 46 ohm
3. 4'b0010: 49 ohm
4. 4'b0011: 52 ohm
5. 4'b0100: 56 ohm
6. 4'b0101: 60 ohm
7. 4'b0110: 66 ohm
8. 4'b0111: 73 ohm
9. 4'b1000: 30 ohm

10. 4'b1001: 31.2 ohm
11. 4'b1010: 32.5 ohm
12. 4'b1011: 34 ohm
13. 4'b1100: 35.5 ohm
14. 4'b1101: 37 ohm
15. 4'b1110: 39 ohm
16. 4'b1111: 41 ohm

20.10.2.4.2 High-Speed Driver Down Resistor Control

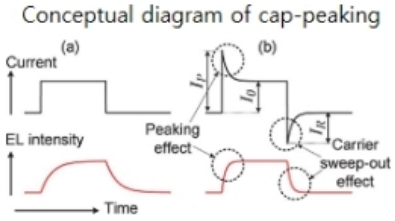
```
echo level > res_dn
```

The level settings are as follows:

1. 4'b0000: 43 ohm
2. 4'b0001: 46 ohm
3. 4'b0010: 49 ohm
4. 4'b0011: 52 ohm
5. 4'b0100: 56 ohm
6. 4'b0101: 60 ohm
7. 4'b0110: 66 ohm
8. 4'b0111: 73 ohm
9. 4'b1000: 30 ohm
10. 4'b1001: 31.2 ohm
11. 4'b1010: 32.5 ohm
12. 4'b1011: 34 ohm
13. 4'b1100: 35.5 ohm
14. 4'b1101: 37 ohm
15. 4'b1110: 39 ohm
16. 4'b1111: 41 ohm

20.10.2.5 Cap Peaking

- **Related register**



Reg. Name	Offset	Field name	Range	Access Tag	RTL Tag	Default Value (D-PHY)	Default Value (C-PHY)
M%_COMBO_MD#_ANA_CON0	0x???8		[31:0]			0x0000_7333	0x0000_1188
		RESERVED0	[31:16]	RWi	RWi	0x0	0x0
		RESERVED1	[15]	RW	RW	0x0	0x0
		EDGE_CON	[14:12]	RW	RW	0x7	0x1
		RESERVED2	[11:10]	RW	RW	0x0	0x0
		EDGE_CON_DIR	[9]	RW	RW	0x1	0x0
		EDGE_CON_EN	[8]	RW	RW	0x1	0x1
		RES_UP	[7:4]	RW	RW	0x3	0x8
		RES_DN	[3:0]	RW	RW	0x3	0x8

Note

- All M%_COMBO_MD#_ANA_CON0, M%_DPHY_MD#_ANA_CON0, and M%_DPHY_MC_ANA_CON0 should have the same setting value.

Setting guide

EDGE_CON: Cap peaking control (thermometer)
- 3'b111: High-strength cap-peaking
- 3'b011: Mid-strength cap-peaking
- 3'b001: Low-strength cap-peaking
- 3'b000: Cap-peaking off (unrelated to EDGE_CON_EN and EDGE_CON_DIR)

EDGE_CON_DIR: Cap peaking direction control
- 1'b0: Backward Cap Peaking (for slower transition) for DPHY
  Forward Cap Peaking (for faster transition) for CPHY
- 1'b1: Forward Cap Peaking (for faster transition) for DPHY
  Backward Cap Peaking (for slower transition) for CPHY

EDGE_CON_EN: Cap peaking enable
- 1'b0: Disable
- 1'b1: Enable

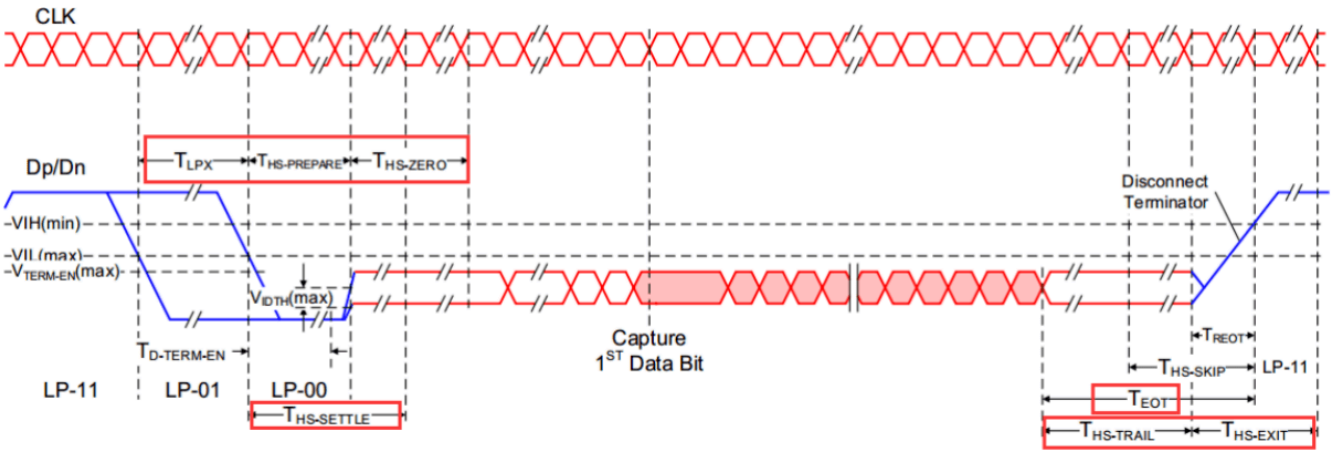
RES_UP: High-speed driver up resistor control
RES_DN: High-speed driver down resistor control
(The resistance according to the code is the same.)
- 4'b0000: 42Ω
- 4'b0001: 44Ω
- 4'b0010: 46Ω
- 4'b0011: 49Ω (default for DPHY)
- 4'b0100: 52Ω
- 4'b0101: 58Ω
- 4'b0110: 64Ω
- 4'b0111: 73Ω
- 4'b1000: 30Ω (default for CPHY)
- 4'b1001: 31.2Ω
- 4'b1010: 32.5Ω
- 4'b1011: 34Ω
- 4'b1100: 35.5Ω
- 4'b1101: 37Ω
- 4'b1110: 39Ω
- 4'b1111: 41Ω

```
echo level > cap_peaking

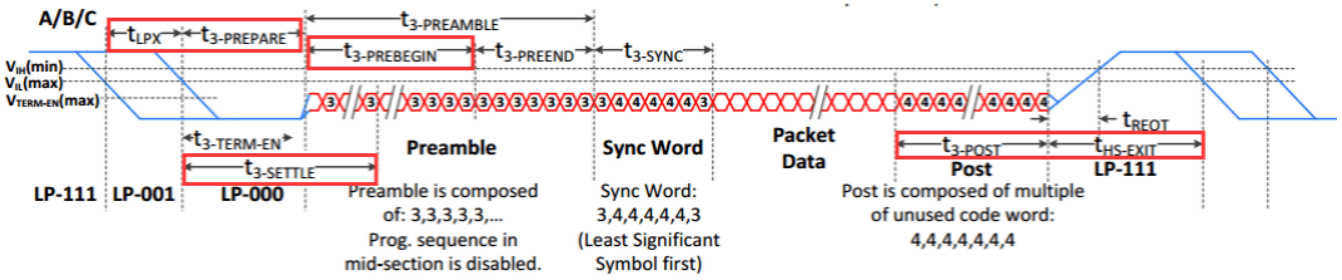
level: 0~7
```

20.10.3 Signal Timing

The signal parameters highlighted in the red box in the figure below are adjustable.



D-PHY High-Speed Data Transmission in Burst



C-PHY High-Speed Data Transmission in Burst

The following signal adjustment count values can be modified based on the readback results, for example, Tlpx:

```
cat lpx
```

20.10.3.1 Tlpx

```
echo count > lpx
```

20.10.3.2 Ths_prepare_or_prepare_3

```
echo count > hs_prepare_or_prepare_3
```

20.10.3.3 Ths_zero_or_prebegin_3

```
echo count > hs_zero_or_prebegin_3
```

20.10.3.4 Ths_trail_or_post_3

```
echo count > hs_trail_or_post_3
```

20.10.3.5 Ths_exit

```
echo count > hs_exit
```